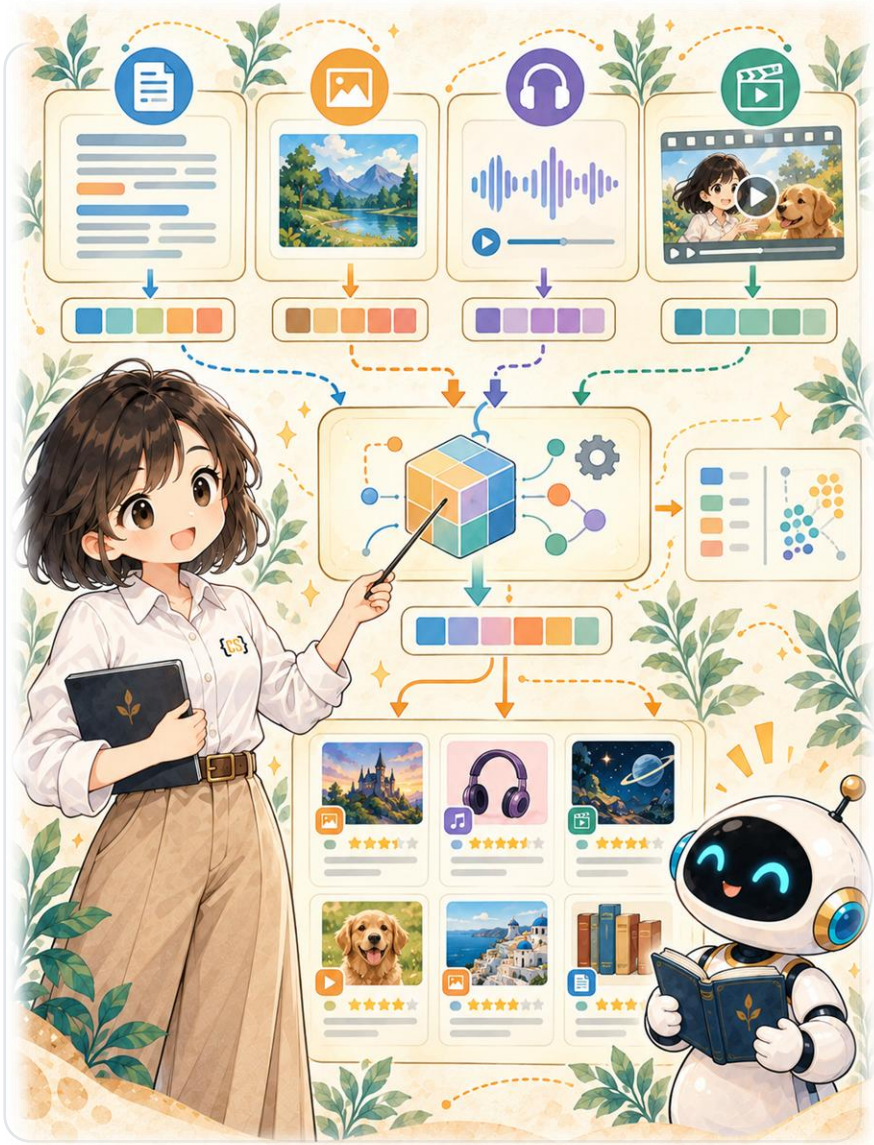
TOPIC 4:

Multimodal Recommender System

204729

Recommender Systems

ผู้ช่วยศาสตราจารย์ ดร.ดุษฎี ประเสริฐธิติพงษ์



COMPUTER SCIENCE
Chiang Mai University

บทนี้ตอบคำถามว่า “ภาพ เสียง วิดีโอ ช่วยแนะนำภาพยนตร์ได้อย่างไร”



ชุดข้อมูลเดิมจากบทที่ 3: MovieMini-12 (12 เรื่อง • 8 ผู้ใช้)

ผู้ใช้หลัก u1 และผู้สมัคร 7 เรื่อง ⟶ เทียบผลกับบทที่ 3 ได้โดยตรง

1 **พื้นฐาน (4.1–4.2)**

เหตุผลของหลายสื่อ • นิวรอน • Gradient Descent • Embedding

2 **ตัวเข้ารหัส (4.3–4.4)**

CNN และ Transformer ทีละขั้น พร้อมตัวเลขคำนวณมือ

3 **หัวใจของบท (4.5–4.7)**

โปรไฟล์ผู้ใช้ • Multimodal Fusion • ประเมินผล

4 **ภาพใหญ่ (4.8–4.10)**

ระบบจริง • ไฮเปอร์พารามิเตอร์ • Lab • บทวิเคราะห์

วัตถุประสงค์ประจำบท

1 **อธิบาย**

ข้อความ ภาพ เสียง วิดีโอ ให้หลักฐานต่างกันอย่างไร

2 **อธิบาย**

วิวัฒนาการ Feature Extraction สามยุค

3 **คำนวณด้วยมือ**

นิวรอน ReLU Gradient Descent และ Embedding

4 **คำนวณขั้นตอน CNN**

Conv 1D/2D • Padding • Stride • Pooling • Receptive Field

5 **คำนวณขั้นตอน Transformer**

Self-Attention • Multi-Head • Positional Enc. • LayerNorm • FFN

6 **สร้างโปรไฟล์ผู้ใช้**

ค่าเฉลี่ย • ถ่วงคะแนน • Rocchio • ลดตามเวลา • หลายจุดสนใจ • Attention

7 **เลือกและคำนวณ Fusion**

Late • z-score • Borda/RRF • Early • Gate • CLIP • VBPR

8 **ประเมินผล**

HR@3 • MRR • nDCG@3 แบบ Leave-One-Out และการถอดสื่อ

ความรู้พื้นฐานที่ควรมีก่อนเรียนบทนี้

ความรู้	ใช้ในหัวข้อ	ทบทวนได้ที่
ผลคูณจุด ความยาวเวกเตอร์ ความคล้ายโคไซน์	4.1–4.7	บทที่ 3 หัวข้อ 3.3
TF-IDF และ L^2 Normalization	4.1, 4.5	บทที่ 3 หัวข้อ 3.5
Static Word Embedding (เวกเตอร์ 12 มิติ)	4.3, 4.4	บทที่ 3 หัวข้อ 3.7
โปรไฟล์ค่าเฉลี่ย ถ่วงคะแนน Rocchio	4.5	บทที่ 3 หัวข้อ 3.9
HR@K, MRR, nDCG@K และ Leave-One-Out	4.7	บทที่ 2 หัวข้อ 2.4–2.8
การคูณเมทริกซ์ exp และอนุพันธ์อย่างง่าย	4.2–4.6	ภาคผนวกของหนังสือ

ทิปการเรียน อ่านกล่อง “อธิบายแบบเข้าใจง่าย” และตัวอย่างการคำนวณข้อแรกของแต่ละหัวข้อก่อน แล้วจึงกลับมาดูรายละเอียด

4.1

ทำไมต้องใช้ข้อมูลหลายรูปแบบ

Why Multimodal • MovieMini-12 • ขั้นตอนหกขั้นของระบบ



4.1.1 สิ่งที่เรื่องย่อบอกไม่ได้

i7 Ballerina กับ i8 Furiosa เรื่องย่อแทบไม่มีคำร่วมกัน แต่เป็นแอ็กชันจังหวะเร็วเหมือนกัน

ข้อมูลที่ใช้แทนภาพยนตร์	ที่มาของเวกเตอร์	cos(i7, i8)
TF-IDF ของเรื่องย่อ	บทที่ 3 หัวข้อ 3.5	0.0000
ข้อความ เทคนิค CNN (Static)	ตารางที่ 4.12	0.4493
ข้อความ เทคนิค Transformer	ตารางที่ 4.15	0.4486
ภาพโปสเตอร์ (Transformer)	ตารางที่ 4.16	−0.5187
เสียงเพลงประกอบ (Transformer)	ตารางที่ 4.16	0.9642
ตัวอย่างภาพยนตร์ (Transformer)	ตารางที่ 4.16	0.9968

อ่านตารางได้สามข้อ

- **นับคำ** มองไม่เห็นความคล้ายเลย (0.0000)
- **ความหมาย** (CNN/Transformer) เห็นปานกลาง ≈ 0.45
- **เสียง/วิดีโอ** สูงมาก 0.96–0.99 แต่ **โปสเตอร์** ติดลบ −0.5187

ข้อสรุปหลักของบท แต่ละสื่อเห็นความคล้ายคนละด้าน ⟶ ต้องตัดสินใจว่าจะเชื่อสื่อใดมากน้อยเพียงใด (Fusion, หัวข้อ 4.6)

สมการ (4.1) ความคล้ายโคไซน์ (Cosine Similarity)

$$\cos(\mathbf{a}, \mathbf{b}) = \frac{\mathbf{a} \cdot \mathbf{b}}{\|\mathbf{a}\| \, \|\mathbf{b}\|} = \frac{\sum_{k=1}^{d} a_k b_k}{\sqrt{\sum_{k=1}^{d} a_k^2} \sqrt{\sum_{k=1}^{d} b_k^2}}$$

สมการ (4.1)

วัด “ทิศ” ของเวกเตอร์ ไม่สนความยาว

องค์ประกอบของสูตร

- a, b เวกเตอร์ของรายการสองรายการ
- a_k, b_k ค่ามิติที่ k
- d จำนวนมิติของเวกเตอร์
- $\|a\|$ ความยาว (L^2 Norm) = $\sqrt{(\sum a_k^2)}$

การตีความ

- มีค่าระหว่าง −1 ถึง 1 ยิ่งมากยิ่งคล้าย
- ถ้าเวกเตอร์ปรับความยาวเป็น 1 แล้ว ($\|a\| = \|b\| = 1$) โคไซน์ = **ผลคูณจุด** a•b
- มิติที่เป็น 0 ในเวกเตอร์ใดเวกเตอร์หนึ่ง ข้ามได้เลย

ตัวอย่างการคำนวณ 4.1 : โคไซน์ข้อความ i7–i8 (Transformer)

โจทย์ หาโคไซน์ระหว่างเวกเตอร์ข้อความ 12 มิติของ i7 Ballerina และ i8 Furiosa (ตารางที่ 4.15 ปรับความยาวเป็น 1 แล้ว)

1 หามิติที่ไม่เป็นศูนย์พร้อมกัน

- มี 3 มิติ: crime, action, family
- มิติอื่นมีอย่างน้อยหนึ่งเรื่องเป็น 0 ⟶ ผลคูณเป็น 0

2 คูณทีละมิติแล้วบวกกัน

มิติ	i7	i8	i7 × i8
crime	0.9065	0.0449	0.0407
action	0.4121	0.9755	0.4020
family	0.0769	0.0772	0.0059
รวม (a•b)			0.4486

3 หารด้วยผลคูณความยาว

- $\|a\| = \|b\| = 1$ (ปรับความยาวแล้ว)
- cos = 0.4486 / (1 × 1) = 0.4486

ตีความ ค่าเกือบทั้งหมดมาจาก action (0.4020) แต่ i7 เน้น crime ส่วน i8 เน้นเอาชีวิตรอด ⟶ คล้ายระดับกลาง

ที่มาของข้อมูล • ตัวอย่างการคำนวณ 4.1 (i7–i8)

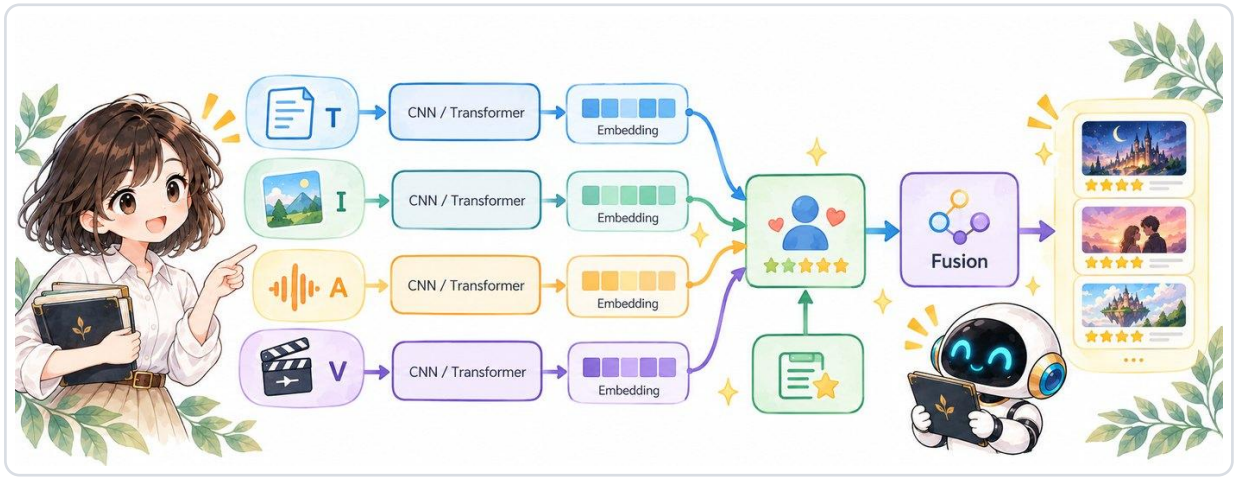
ข้อมูลมาจากไหน

- ชื่อเรื่องและแนว: ตารางที่ 4.4
- เรื่องย่อ: ตารางที่ 4.6 → ตัดคำ → คลังแนวคิด 12 มิติ → เฉลี่ยถ่วง IDF → ปรับ L^2
- i7: 20 โทเคน คำอาชญากรรม (assassin, killers ×2, criminal, cult, murdered, revenge, violent, underworld) → **crime**
- i8: 18 โทเคนในคลัง คำเอาชีวิตรอด (survivor, warlord, horde, fight, survival, road, chase) → **action**
- Transformer อ่าน chase ใน i8 เป็น "การไล่ล่า" (action 1)

มิติ	ตัวเศษ i7	÷ 56.2196	÷ $\|\|e\|\|$ = 0.6643
mind	1.2332	0.0219	0.0330
crime	33.8534	0.6022	0.9065
action	15.3894	0.2737	0.4121
family	2.8718	0.0511	0.0769
music	1.4359	0.0255	0.0384

$$i7 \cdot i8 = 0.0407 + 0.4020 + 0.0059 = 0.4486$$

4.1.2 ขั้นตอนการทำงานหกขั้นของระบบ



หัวข้อ	รับเข้า	ส่งออก
4.2 การสกัดคุณลักษณะ	ข้อมูลนำเข้าของสี่สื่อ	แนวคิดตัวเข้ารหัส นิวรอน การฝัง
4.3 CNN / 4.4 Transformer	เทนเซอร์ / ลำดับโทเคนหรือแพตช์	เวกเตอร์รายการของแต่ละสื่อ
4.5 โปรไฟล์ผู้ใช้	เวกเตอร์รายการ + คะแนนผู้ใช้	โปรไฟล์และคะแนนรายสื่อ
4.6 Fusion	คะแนนรายสื่อ	คะแนนรวมและ Top-K
4.7 ประเมินผล	Top-K ของทุกผู้ใช้	HR@3, MRR, nDCG@3

4.1.3 ชุดข้อมูล MovieMini-12 : บัญชีภาพยนตร์

รหัส	ชื่อเรื่อง (ปี)	แนว	รหัส	ชื่อเรื่อง (ปี)	แนว
i1	Toy Story 5 (2026)	Animation, Family	i7	Ballerina (2025)	Action, Thriller
i2	Inside Out 2 (2024)	Animation, Family	i8	Furiosa: A Mad Max Saga	Action, Sci-Fi
i3	The Wild Robot (2024)	Animation, Sci-Fi	i9	Conclave (2024)	Drama, Thriller
i4	It Ends with Us (2024)	Drama, Romance	i10	Dune: Part Two (2024)	Action, Sci-Fi
i5	Wicked (2024)	Fantasy, Musical	i11	Mickey 17 (2025)	Sci-Fi, Thriller
i6	Bridget Jones: Mad About the Boy	Drama, Romance	i12	Project Hail Mary (2026)	Drama, Sci-Fi

กุญแจเชื่อม รหัส i1–i12 เชื่อมทุกตาราง ถ้าจับคู่รหัสผิด ระบบอาจนำเรื่องย่อของเรื่องหนึ่งไปรวมกับเสียงของอีกเรื่อง **โดยไม่มีข้อความเตือนใด ๆ**

แถบสีฟ้า = แอนิเมชันสามเรื่องที่ u1 ชอบ

MovieMini-12 : คะแนนผู้ใช้และข้อตกลงของบท

ผู้ใช้	ชอบ (4–5)	ไม่ชอบ (1–2)	กลาง (3)
u1	i1 (5), i2 (5), i3 (4)	i4 (1), i6 (1)	–
u2	i4 (5), i5 (4), i6 (5)	i1 (2)	–
u3	i10 (5), i11 (5), i12 (4)	i7 (1)	–
u4	i1 (4), i7 (5), i8 (5)	i5 (1)	–
u5	i3 (4), i9 (5)	i10 (2)	i6 (3)
u6	i2 (5), i5 (4)	–	i11 (3)
u7	i11 (4), i12 (5)	i8 (2)	i3 (3)
u8	i4 (4), i6 (5)	i2 (2)	i11 (3)

ผู้ใช้หลัก u1

- ผู้สมัคร (Candidate) 7 เรื่อง: i5, i7, i8, i9, i10, i11, i12

ข้อตกลงที่ใช้ตลอดบท

- ลำดับสื่อ T, I, A, V (ข้อความ ภาพ เสียง วิดีโอ)
- เวกเตอร์รายการ v_i^m • โปรไฟล์ p_u^m
- Rating (1–5) ≠ Score ของระบบ (ติดลบได้)
- เสมอกันเรียงตามรหัส • ใช้ K = 3
- แสดงทศนิยม 4 ตำแหน่ง

ที่มาของข้อมูล : คะแนนของผู้ใช้ 8 คน (ครบทุกช่อง)

ผู้ใช้	ชอบ (4–5)	ไม่ชอบ (1–2)	กลาง (3)	ยังไม่เคยให้คะแนน
u1	i1 (5), i2 (5), i3 (4)	i4 (1), i6 (1)	—	i5, i7, i8, i9, i10, i11, i12
u2	i4 (5), i5 (4), i6 (5)	i1 (2)	—	i2, i3, i7, i8, i9, i10, i11, i12
u3	i10 (5), i11 (5), i12 (4)	i7 (1)	—	i1, i2, i3, i4, i5, i6, i8, i9
u4	i1 (4), i7 (5), i8 (5)	i5 (1)	—	i2, i3, i4, i6, i9, i10, i11, i12
u5	i3 (4), i9 (5)	i10 (2)	i6 (3)	i1, i2, i4, i5, i7, i8, i11, i12
u6	i2 (5), i5 (4)	—	i11 (3)	i1, i3, i4, i6, i7, i8, i9, i10, i12
u7	i11 (4), i12 (5)	i8 (2)	i3 (3)	i1, i2, i4, i5, i6, i7, i9, i10
u8	i4 (4), i6 (5)	i2 (2)	i11 (3)	i1, i3, i5, i7, i8, i9, i10, i12

ช่อง "ยังไม่เคยให้คะแนน" ไม่ใช่คะแนนศูนย์ • ใช้เป็นรายการผู้สมัคร เช่น u1 = i5, i7–i12 • เรื่องที่ชอบใช้สร้างโปรไฟล์และเป็นกรณีทดสอบ 20 กรณี

ที่มาของข้อมูล : เรื่องย่อของภาพยนตร์ (ตารางที่ 4.6) 1/2

รหัส	ชื่อเรื่อง	เรื่องย่อ (Synopsis)
i1	Toy Story 5	A cowboy toy and a space ranger toy lead the toys of a girl bedroom against a glowing tablet that steals every playtime, and the toys must prove to the girl that friendship and imagination still matter more than a screen, so the toys win the girl back.
i2	Inside Out 2	Inside the mind of a teenage girl, five emotions meet an anxious new emotion that seizes control, while each night a dream replays a memory, a dream of friendship and a dream of the future, until a frightening dream blurs belief, memory and self.
i3	The Wild Robot	A shipwrecked robot boots up on a wild island where animals fear the machine, until she raises an orphaned gosling as her own child, learns the language of the wild world, and gives up her programming to protect the family and the island she now calls home.
i4	It Ends with Us	A florist falls in love with a charming surgeon, but when his love turns violent she must choose between the boy she remembers from her first love in a small town and her own safety, so that the cycle of violence ends with her and never reaches her daughter.
i5	Wicked	A green-skinned woman with a gift for magic joins a popular friend to chase a dream of meeting the wizard, sharing music, a rival and a first love, until ambition and a dream of a better land pull the two friends apart and the whole city brands her wicked.
i6	Bridget Jones: Mad About the Boy	A widowed mother of two children keeps talking to the memory of her late husband, until a younger man and a kind teacher bring love into her life again, and she discovers that a new love does not erase a memory, so both of them can live under one roof.

เรื่องย่อคือที่มาของเวกเตอร์ข้อความ 12 มิติ: ตัดคำ → คลังแนวคิด → เฉลี่ยถ่วง IDF → ปรับ L^2 (บทที่ 3 หัวข้อ 3.7)

ที่มาของข้อมูล : เรื่องย่อของภาพยนตร์ (ตารางที่ 4.6) 2/2

รหัส	ชื่อเรื่อง	เรื่องย่อ (Synopsis)
i7	Ballerina	A ballerina trained as an assassin by a secret school of killers hunts the criminal cult that murdered her father, and she cuts a violent path of revenge through hotels, clubs and rival killers of the underworld.
i8	Furiosa: A Mad Max Saga	Snatched from a fertile place as a child, a survivor crosses a scorched wasteland ruled by a warlord and his biker horde, and a road chase across the wasteland becomes a fight for fuel, water, freedom and survival.
i9	Conclave	After the death of the pope, a cardinal must run the secret election of the next pope behind the sealed doors of a chapel, where ambition, a hidden secret of each candidate and opposing factions turn a holy vote into a thriller, until one final secret about the chosen candidate changes the church.
i10	Dune: Part Two	On a desert planet, a young heir of a fallen dynasty allies with the desert tribes, adopts their ways, rides the giant sand worms and leads a war of rebellion to topple an empire, while a prophecy of the desert world and his visions of the future weigh on him.
i11	Mickey 17	A disposable worker on an icy planet dies on every dangerous mission and comes back as a fresh printed copy, until a copy that should be dead returns alive, and two copies of one man must hide from the colony leader, the alien creatures of the planet and the fear of losing his own memory.
i12	Project Hail Mary	A lone science teacher wakes on a spaceship far from Earth with no memory of his mission, finds out that a microbe is dimming the sun, and travels through space with an alien friend from another star to save humanity, while space and time carry him ever farther from home.

เรื่องย่อคือที่มาของเวกเตอร์ข้อความ 12 มิติ: ตัดคำ → คลังแนวคิด → เฉลี่ยถ่วง IDF → ปรับ L^2 (บทที่ 3 หัวข้อ 3.7)

ที่มาของข้อมูล : เส้นทางจากตารางต้นทางสู่ตัวอย่างการคำนวณ (ตารางที่ 4.7)

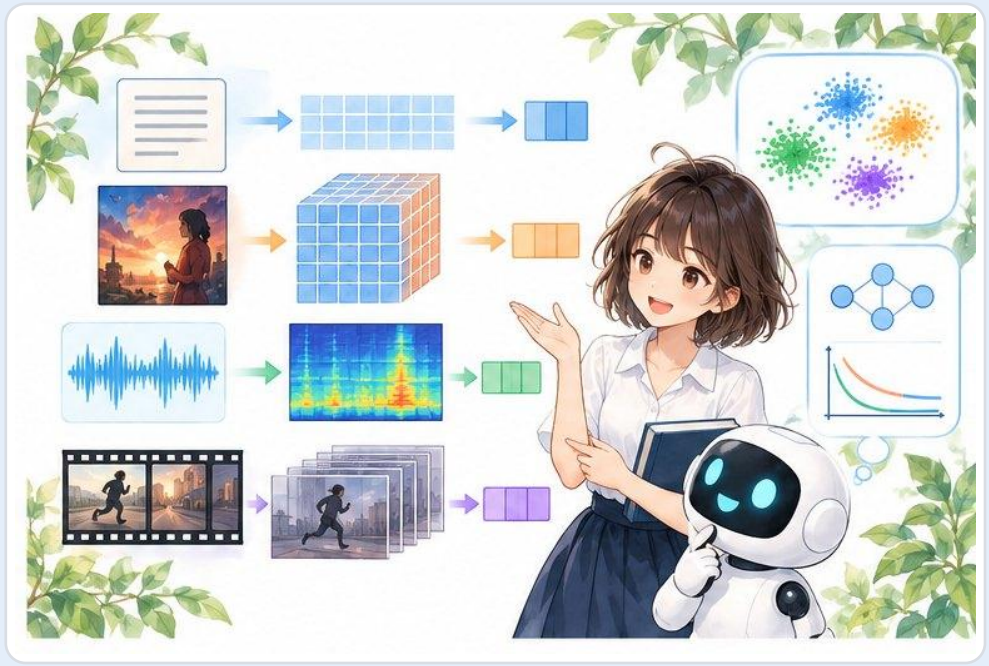
ข้อมูลต้นทาง	สิ่งที่ได้	ขั้นตอนการได้มา	ใช้ในตัวอย่างการคำนวณ
ตารางที่ 4.4 บัญชีภาพยนตร์	รหัส i1–i12 (กุญแจเชื่อม) • แนว	อ่านตรงจากตาราง	ทุกตัวอย่าง
ตารางที่ 4.5 คะแนนผู้ใช้	ชอบ / ไม่ชอบ / ผู้สมัคร / 20 กรณีทดสอบ	จัดกลุ่มตามช่วงคะแนน	4.4, 4.25–4.38, 4.40, 4.41
ตารางที่ 4.6 เรื่องย่อ	เวกเตอร์ข้อความ 12 มิติ (ตารางที่ 4.12, 4.15)	ตัดคำ → คลังแนวคิด → ถ่วง IDF → L^2	4.1, 4.5, 4.12, 4.14–4.17, 4.21, 4.23, 4.25–4.28
สื่อ (รูปแบบในตารางที่ 4.8)	เวกเตอร์ภาพ เสียง วิดีโอ 3 มิติ (ตารางที่ 4.13, 4.16)	ค่าจำลองตามแนว → ลบค่าเฉลี่ยคลัง → L^2	4.3, 4.4, 4.24, 4.29–4.31, 4.37, 4.40
ตารางที่ 4.19 คะแนนรายสื่อ u1	ข้อมูลนำเข้าของ Fusion	ผลของโปรไฟล์ค่าเฉลี่ย (4.5)	4.32–4.38
ค่าสมมติเพื่อการสอน	น้ำหนัก ตัวกรอง เมทริกซ์ฉาย วันที่	กำหนดให้คำนวณด้วยมือได้	4.2–4.13, 4.15–4.22, 4.28, 4.31, 4.39, 4.40

ทุกตัวอย่างการคำนวณมีสไลด์ "ที่มาของข้อมูล" หรือ "ขั้นตอนโดยละเอียด" ต่อท้าย เพื่อย้อนตรวจตัวเลขทุกตัวได้

4.2

การสกัดคุณลักษณะ (Feature Extraction) ในยุคต่าง ๆ

Neuron • Gradient Descent • Embedding • Encoder



4.2.1 คุณลักษณะคืออะไร และทำไมต้องสกัด

นิยาม 4.2

- Feature ตัวเลขที่สรุปสิ่งสำคัญของข้อมูล
- Feature Extraction แปลงข้อมูลตัวเลขจำนวนมาก ⟶ เวกเตอร์สั้น
- Embedding ผลลัพธ์ เวกเตอร์ยาวคงที่ที่วัดความคล้ายได้

สื่อ	สิ่งที่เก็บ	รูปร่างเทนเซอร์	จำนวนตัวเลข
ข้อความ (T)	เรื่องย่อ 60 โทเคน	60 × 768	46,080
ภาพ (I)	โปสเตอร์ 224×224 RGB	224 × 224 × 3	150,528
เสียง (A)	คลิป 10.24 วินาที	Log-Mel 128 × 1,024	131,072
วิดีโอ (V)	ตัวอย่างหนัง 16 เฟรม	16 × 224 × 224 × 3	2,408,448

ตารางที่ 4.8 ขนาดข้อมูลนำเข้าต่อภาพยนตร์หนึ่งเรื่อง

$$N_{in} = \prod_{k=1}^{K} n_k \qquad (4.2)$$

$$\rho = \frac{N_{in}}{d} \qquad (4.3)$$

- K จำนวนแกน • n_k ขนาดแกนที่ k
- d มิติของเวกเตอร์ เช่น 768
- **ρ** อัตราการย่อ: กี่ค่านำเข้าถูกสรุปเหลือ 1 ค่า

เทียบพิกเซลตรง ๆ ไม่ได้ เพราะทั้งช้าและไม่มีความหมาย ⟶ ต้องย่อเป็นเวกเตอร์สั้น

ตัวอย่างการคำนวณ 4.2 : จำนวนตัวเลขนำเข้าและอัตราการย่อ

โจทย์ ตัวเข้ารหัสคืนเวกเตอร์ d = 768 มิติ จงหาจำนวนตัวเลขนำเข้าและอัตราการย่อของ “โปสเตอร์” และ “ตัวอย่างภาพยนตร์”

1 โปสเตอร์ (3 แกน)

- หนึ่งช่องสี: 224 × 224 = 50,176
- สามช่องสี: 50,176 × 3 = **150,528** ค่า

2 ตัวอย่างภาพยนตร์ (4 แกน)

- หนึ่งเฟรม = 150,528 ค่า (เท่าโปสเตอร์)
- 16 เฟรม: 16 × 150,528 = **2,408,448** ค่า

3 อัตราการย่อ

$$\rho_I = \frac{150{,}528}{768} = 196 \qquad \rho_V = \frac{2{,}408{,}448}{768} = 3{,}136$$

ตีความ เวกเตอร์เหลือเพียง ≈0.51% (ภาพ) และ ≈0.03% (วิดีโอ) — ไม่ใช่การบีบอัดไฟล์ แต่ **เก็บเฉพาะสิ่งที่จำเป็นต่อการเปรียบเทียบ**

ที่มาของข้อมูล • ตัวอย่างการคำนวณ 4.2 (อัตราการย่อ)

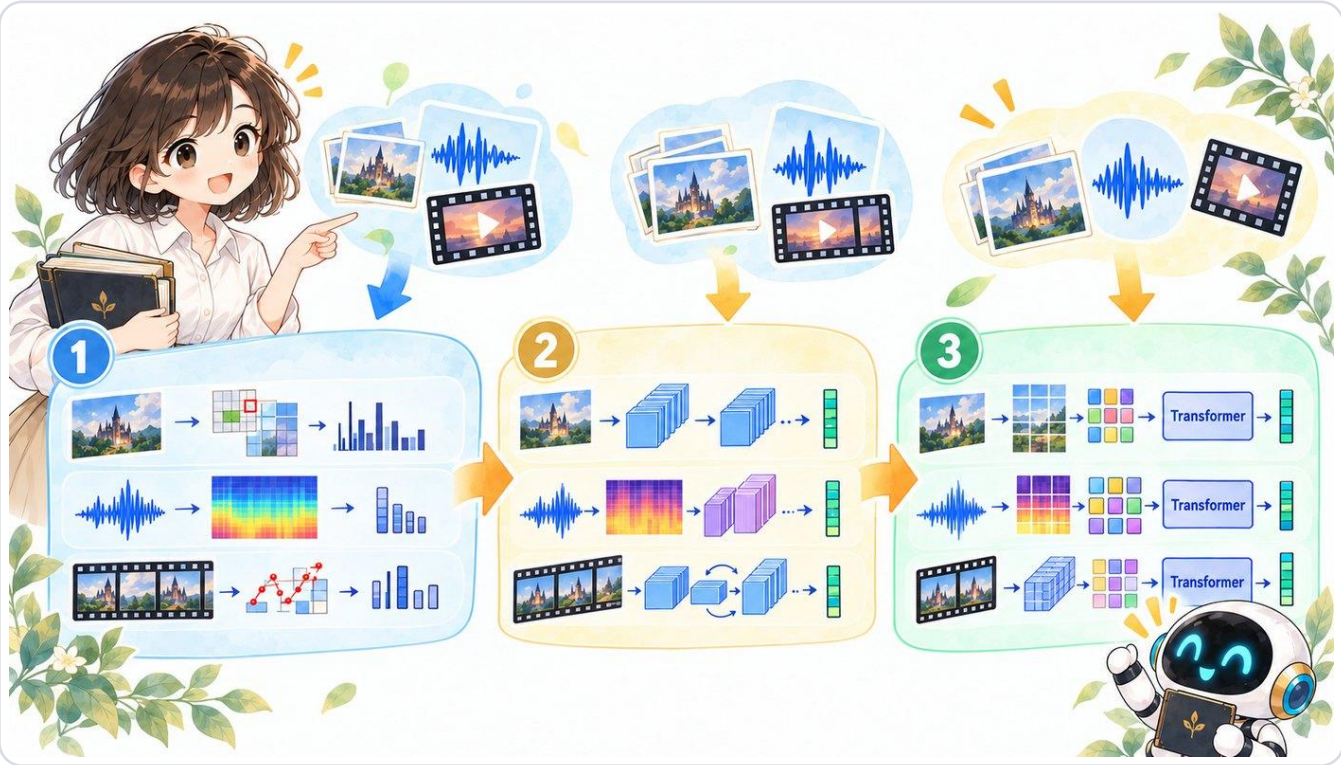
ข้อมูลมาจากไหน

- รูปร่างเทนเซอร์ของทุกเรื่องใน MovieMini-12: ตารางที่ 4.8
- โปสเตอร์ 224 × 224 × 3 (RGB)
- ตัวอย่างภาพยนตร์ 16 เฟรม (เลือกจากตัวอย่าง 2 นาที ดูตัวอย่าง 4.13)
- d = 768 ตาม ViT-B/16 และ BERT-base

สื่อ	คูณทุกแกน	Nin	ρ = Nin/768
ข้อความ	60 × 768	46,080	60
ภาพ	224 × 224 × 3	150,528	196
เสียง	128 × 1,024	131,072	170.67
วิดีโอ	16 × 224 × 224 × 3	2,408,448	3,136

ตรวจ: 196 × 768 = 150,528 และ 3,136 × 768 = 2,408,448

4.2.2 สามยุคของการสกัดคุณลักษณะ



ยุคหลังใช้ **ข้อมูลมากขึ้น** • **การคำนวณมากขึ้น** และได้ **ความเข้าใจเชิงความหมายสูงขึ้น** บทนี้เน้นยุคที่ 2 และ 3

1 ออกแบบด้วยมือ (1990–2011)

SIFT, HOG, MFCC ⟶ ตัวบรรยาย (Descriptor) อธิบายง่าย แต่จับความหมายสูงได้จำกัด

2 เรียนรู้เชิงลึก CNN (2012–2020)

AlexNet, ResNet, 3D CNN ⟶ Feature Map ⟶ Embedding

3 Transformer หลายรูปแบบ (2021–)

ViT, AST, CLIP: โทเคน/แพตช์ + Attention ⟶ ปริภูมิร่วม

สรุปสามยุคสำหรับสี่สื่อ (ตารางที่ 4.9)

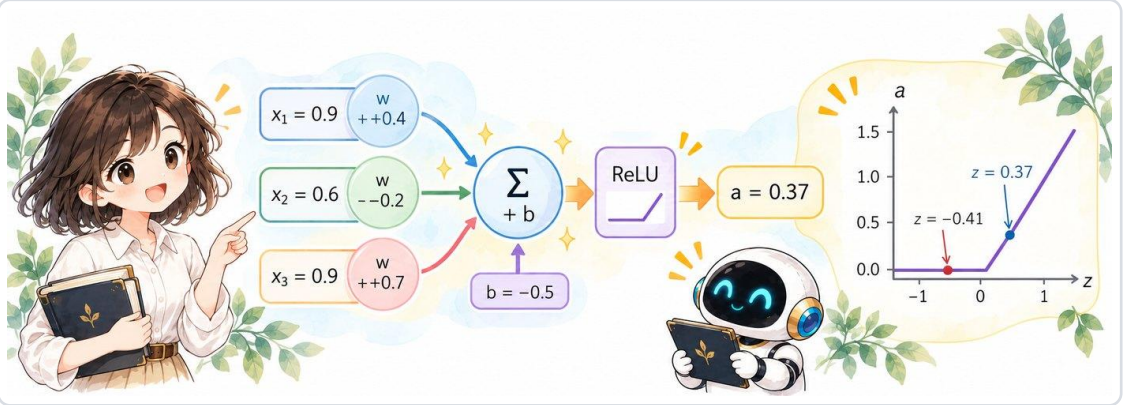
สื่อ	ยุค 1 ออกแบบด้วยมือ	ยุค 2 เรียนรู้เชิงลึก (CNN)	ยุค 3 Transformer
ข้อความ	Bag-of-Words, TF-IDF, BM25	Word2Vec, GloVe + Text CNN	BERT, Sentence-BERT, MiniLM
ภาพ	SIFT, HOG, LBP, GIST	AlexNet, VGG, ResNet, EfficientNet	ViT, Swin, DINOv2, CLIP
เสียง	MFCC, Chroma, Spectral Centroid	CNN บนสเปกโทรแกรม, VGGish, PANNs	AST, wav2vec 2.0, HuBERT, CLAP
วิดีโอ	STIP, HOF, Dense Trajectories	2D CNN รายเฟรม, C3D, I3D, SlowFast	TimeSformer, ViViT, VideoMAE
ผลลัพธ์	ตัวบรรยาย (Descriptor)	Feature Map ⟶ Embedding	เวกเตอร์ตามบริบท / ปริภูมิร่วม

อธิบายแบบเข้าใจง่าย ยุค 1 = พ่อครัวชิมตามรายการตรวจที่เขียนไว้ • ยุค 2 = ชิมมาหลายหมื่นจานจนรู้เอง • ยุค 3 = นักวิจารณ์ที่ชิม ดู และฟังพร้อมกัน

4.2.3 นิวรอน : หน่วยพื้นฐานของการเรียนรู้คุณลักษณะ

$$z = \sum_{k=1}^{n} w_k x_k + b = \mathbf{w} \cdot \mathbf{x} + b \qquad (4.4)$$

$$a = \mathrm{ReLU}(z) = \max(0, z) \qquad (4.5)$$



องค์ประกอบ

- x_k ค่านำเข้า • w_k น้ำหนัก (สำคัญ/ทิศทาง)
- b ค่าเอนเอียง ใช้เลื่อนเกณฑ์การตอบสนอง
- z ผลรวมถ่วงน้ำหนัก • a ค่าส่งออก
- ReLU ตัดค่าลบเป็น 0 ส่งค่าบวกผ่าน

นิวรอน = กรรมการหนึ่งคน

- ดูหลักฐานหลายข้อ ให้น้ำหนักแต่ละข้อ
- ถ้าคะแนนรวมไม่ถึงเกณฑ์ ⟶ “ไม่ยกป้าย” (ReLU = 0)
- โครงข่าย = คณะกรรมการหลายพันคนซ้อนหลายชั้น
- ReLU สำหรับ CNN • GELU สำหรับ Transformer

ตัวอย่างการคำนวณ 4.3 : นิวรอนตรวจจับโปสเตอร์แอนิเมชัน

โจทย์ w = (0.4, −0.2, 0.7), b = −0.5 จงหาค่าส่งออกเมื่อป้อนเวกเตอร์ภาพ i1 = (0.9, 0.6, 0.9) และ i7 = (0.2, 0.3, 0.1)

1 คำนวณ z ของ i1 Toy Story 5

- 0.4 × 0.9 = 0.36
- −0.2 × 0.6 = −0.12
- 0.7 × 0.9 = 0.63
- z = 0.36 − 0.12 + 0.63 − 0.5 = **0.37**

2 ผ่าน ReLU

- a = max(0, 0.37) = 0.37 ⟶ นิวรอน “ตื่นตัว”

3 ทำซ้ำกับ i7 Ballerina

- z = 0.4(0.2) − 0.2(0.3) + 0.7(0.1) − 0.5
- = 0.08 − 0.06 + 0.07 − 0.5 = **−0.41**
- a = max(0, −0.41) = 0 ⟶ นิวรอน “เงียบ”

ตีความ น้ำหนักบวกขนาดใหญ่ของมิติที่ 3 (0.7) ทำให้ตอบสนองต่อ i1 ที่มีค่ามิติ 3 สูง = “การเรียนรู้คุณลักษณะ”

ที่มาของข้อมูล • ตัวอย่างการคำนวณ 4.3 (นิวรอน)

ข้อมูลมาจากไหน

- เวกเตอร์ภาพ: ตารางที่ 4.16 (ก่อนลบค่าเฉลี่ย)
- i1 แอนิเมชันที่ u1 ให้ 5 คะแนน (ตารางที่ 4.5) • i7 แอ็กชันโทนมืด
- I3 สูงในแอนิเมชัน (0.9, 0.9, 0.8) → w3 = 0.7 ทำให้นิวรอนไวต่อแอนิเมชัน
- w, b เป็นค่าสมมติ

เรื่อง	(I1, I2, I3)	z	ReLU
i1	(0.9, 0.6, 0.9)	0.37	0.37
i2	(0.8, 0.8, 0.9)	0.29	0.29
i3	(0.7, 0.5, 0.8)	0.24	0.24
i4	(0.5, 0.7, 0.2)	−0.30	0.00
i5	(0.7, 0.6, 0.5)	0.01	0.01
i6	(0.6, 0.8, 0.3)	−0.21	0.00
i7	(0.2, 0.3, 0.1)	−0.41	0.00
i8	(0.6, 0.9, 0.3)	−0.23	0.00
i9	(0.4, 0.3, 0.1)	−0.33	0.00
i10	(0.5, 0.8, 0.1)	−0.39	0.00
i11	(0.3, 0.2, 0.1)	−0.35	0.00
i12	(0.3, 0.3, 0.1)	−0.37	0.00

z = 0.4•I1 − 0.2•I2 + 0.7•I3 − 0.5 → ตื่นตัวชัดเฉพาะ i1, i2, i3 (i5 ได้เพียง 0.01)

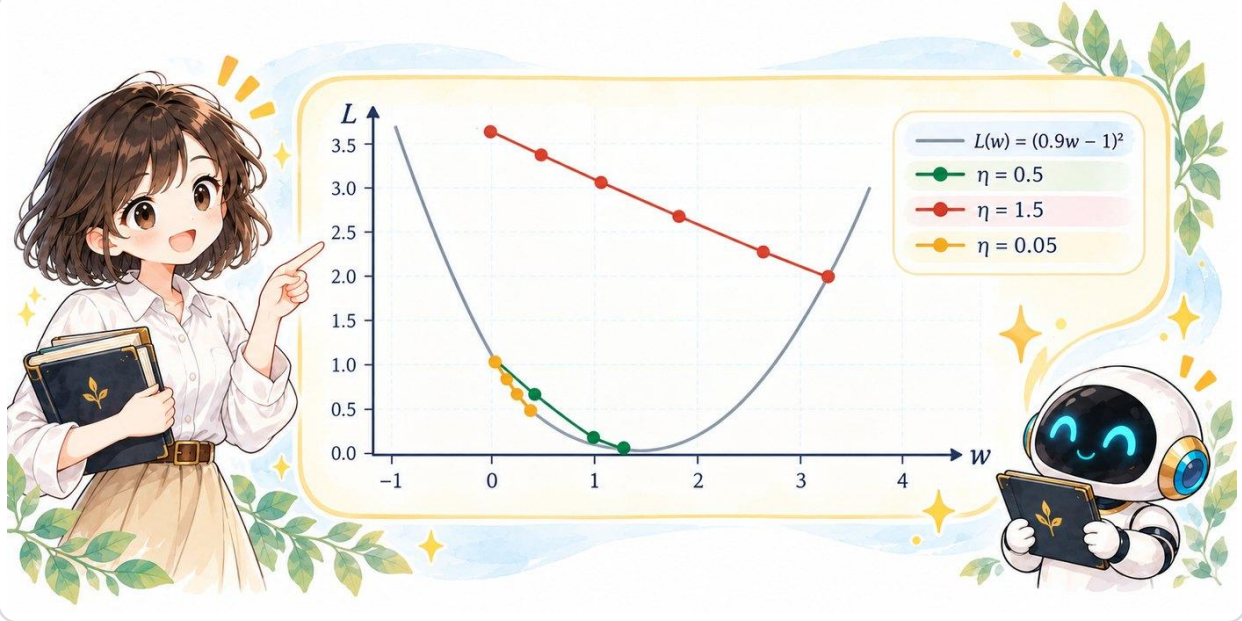
4.2.4 ค่าสูญเสีย (Loss) และการลดตามเกรเดียนต์

$$L(w) = (\hat{y} - y)^2 \tag{4.6}$$

$$w_{t+1} = w_t - \eta \frac{\partial L}{\partial w}(w_t) \tag{4.7}$$

องค์ประกอบ

- $\hat{y}$ ค่าที่ทาย • y ค่าเป้าหมาย
- ∂L/∂w ความชัน บอกว่า L เพิ่มขึ้นทางใด
- η อัตราการเรียนรู้ (ขนาดก้าว)
- ก้าว **สวนทาง** ความชัน



เดินลงเขาในหมอก

- รู้เพียงพื้นใต้เท้าลาดไปทางไหน แล้วก้าวลงทีละก้าว
- η = ความยาวก้าว: สั้นไป⟶ช้า ยาวไป⟶กระโดดข้ามหุบ

ตัวอย่างการคำนวณ 4.4 : ปรับน้ำหนักด้วย Gradient Descent สามรอบ

โจทย์ $x = 0.9$, $\hat{y} = wx$, เป้าหมาย $y = 1$, เริ่ม $w_0 = 0$, $\eta = 0.5$ จงคำนวณการปรับน้ำหนักสามรอบ

1 เขียน L และเกรเดียนต์

$$L(w) = (0.9w - 1)^2 \qquad \frac{\partial L}{\partial w} = 2(0.9w - 1) \times 0.9$$

กฎลูกโซ่: อนุพันธ์ของ $(\cdot)^2$ คือ $2(\cdot)$ แล้วคูณอนุพันธ์ของ $0.9w - 1$ ซึ่งเท่ากับ 0.9

รอบ	w	$L = (0.9w - 1)^2$	เกรเดียนต์	w ใหม่ = w − 0.5 × เกรเดียนต์
1	0.0000	1.0000	−1.8000	0.0000 − 0.5(−1.8000) = 0.9000
2	0.9000	0.0361	−0.3420	0.9000 − 0.5(−0.3420) = 1.0710
3	1.0710	0.0013	−0.0650	1.0710 − 0.5(−0.0650) = 1.1035

3 เทียบคำตอบจริง

- L = 0 เมื่อ w = 1/0.9 = **1.1111**
- สามรอบได้ 1.1035 ใกล้แล้ว

ตีความ ก้าวสั้นลงเองเมื่อใกล้จุดต่ำสุด • ลอง η: 0.5 ⟶ w=1.1035 (L≈0) | 1.5 ⟶ w=4.3602 (L=8.55 แกว่ง) | 0.05 ⟶ w=0.2487 (ช้า)

ที่มาของข้อมูล • ตัวอย่างการคำนวณ 4.4 (Gradient Descent)

ข้อมูลมาจากไหน

- x = 0.9 = มิติที่ 1 ของภาพ i1 (ตารางที่ 4.16)
- y = 1 แทน "ชอบ" เพราะ u1 ให้ i1 = 5 (ตารางที่ 4.5)
- w0 = 0, η = 0.5 เป็นค่าสมมติ
- กฎลูกโซ่: dL/dw = 2(0.9w − 1) × 0.9

รอบ	w	$\hat{y}$	e	L	dL/dw	wใหม่
1	0.0000	0.0000	−1.0000	1.0000	−1.8000	0.9000
2	0.9000	0.8100	−0.1900	0.0361	−0.3420	1.0710
3	1.0710	0.9639	−0.0361	0.0013	−0.0650	1.1035

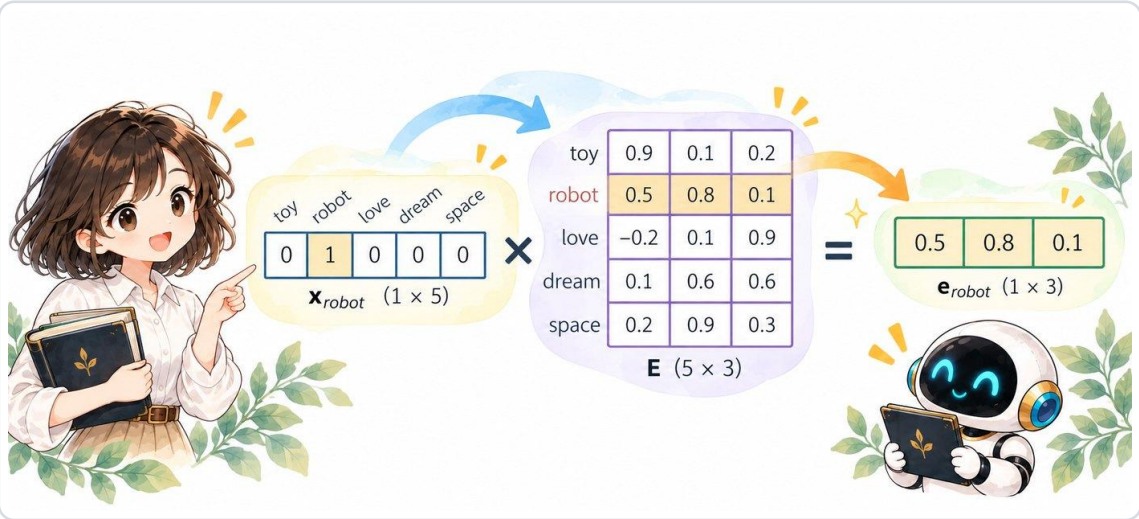
หลัง 3 รอบ w = 1.1035 ใกล้คำตอบ 1/0.9 = 1.1111

4.2.5 การฝัง (Embedding) : เปลี่ยนสัญลักษณ์เป็นเวกเตอร์

$$\mathbf{e}_w = \mathbf{x}_w \mathbf{E} = \mathbf{E}[\, id(w), : \,] \qquad (4.8)$$

ทำไมต้องฝัง

- ใช้เลขลำดับตรง ๆ (toy=1, robot=2) ระบบจะเข้าใจผิดว่า robot “มากกว่า” toy สองเท่า
- x_w เวกเตอร์หนึ่งร้อน (One-hot) ยาว |V|
- Ɛ ตารางฝัง |V| × d แต่ละแถว = หนึ่งสัญลักษณ์
- one-hot × Ɛ = **หยิบแถว** (Lookup)
- ใช้กับรหัสผู้ใช้/รายการ (VBPR) แพตช์ และตำแหน่งด้วย



ขนาดเวกเตอร์ d มาจากไหน

- BERT-base, ViT-B/16 = 768 • ResNet-50 = 2,048 • MiniLM = 384
- บทนี้ใช้ 12 มิติ (ข้อความ) และ 3 มิติ (ภาพ เสียง วิดีโอ) เพื่อคำนวณด้วยมือ

ตัวอย่างการคำนวณ 4.5 : เปิดแถวของตารางฝังสำหรับ robot

โจทย์ คลังศัพท์ toy, robot, love, dream, space จงหาเวกเตอร์ฝังของ robot และโคไซน์ระหว่าง toy กับ robot

ตารางฝัง E (5×3) และเวกเตอร์หนึ่งร้อน

คำ (id)	มิติ 1	มิติ 2	มิติ 3	x_robot
toy (1)	0.9	0.1	0.2	0
robot (2)	0.5	0.8	0.1	1
love (3)	−0.2	0.1	0.9	0
dream (4)	0.1	0.6	0.6	0
space (5)	0.2	0.9	0.3	0

ตีความ ระบบจริง “เปิดแถว” ตามรหัส เร็วกว่าคูณเมทริกซ์

1 x_robot × E ทีละคอลัมน์

- คอลัมน์ 1: $0(0.9) + 1(0.5) + 0(-0.2) + 0(0.1) + 0(0.2) = 0.5$
- คอลัมน์ 2 = 0.8 • คอลัมน์ 3 = 0.1 $\longrightarrow$ $e_{robot} = (0.5, 0.8, 0.1)$ = แถวที่ 2

2 โคไซน์กับ toy = (0.9, 0.1, 0.2)

- ผลคูณจุด $= 0.9(0.5) + 0.1(0.8) + 0.2(0.1) = 0.55$
- $\|toy\| = \sqrt{0.86} = 0.9274$ • $\|robot\| = \sqrt{0.90} = 0.9487$
- $\cos = 0.55 / (0.9274 \times 0.9487) = 0.6252$

ที่มาของข้อมูล • ตัวอย่างการคำนวณ 4.5 (ตารางฝัง)

ข้อมูลมาจากไหน

- ห้าคำมาจากเรื่องย่อ (ตารางที่ 4.6): toy (i1 ×2), robot (i3), love (i4, i5, i6), dream (i2 ×4, i5 ×2), space (i1, i12 ×2)
- เลขแถวเป็นเพียงรหัส → ต้องใช้เวกเตอร์หนึ่งร้อน
- ค่าใน E เป็นค่าสมมติ (ภาพที่ 4.6)

คำ	x	x × คอลัมน์ 1	x × คอลัมน์ 2	x × คอลัมน์ 3
toy	0	0	0	0
robot	1	0.5	0.8	0.1
love	0	0	0	0
dream	0	0	0	0
space	0	0	0	0
รวม		0.5	0.8	0.1

cos(toy, robot) = 0.55 / (0.9274 × 0.9487) = 0.6252

4.2.6 ตัวเข้ารหัส (Encoder) และเวกเตอร์ที่ใช้ในบทนี้

$$\mathbf{v} = f_\theta(x), \qquad \mathbf{v} \in \mathbb{R}^d \tag{4.9}$$

องค์ประกอบและแนวปฏิบัติ

- x ข้อมูลนำเข้า (เทนเซอร์) • f_θ CNN หรือ Transformer
- θ น้ำหนักทั้งหมด ได้จากการฝึก • v เวกเตอร์ d มิติ
- ใช้ Pretrained Model (ResNet/ImageNet, BERT) ตัดชั้นจำแนกทิ้ง
- เรียกว่า Transfer Learning

เวกเตอร์ขนาดย่อสำหรับการสอน

- **ข้อความ 12 มิติ**: technology, mind, space, romance, crime, action, family, music, fantasy, toys, society, ambition
- **ภาพ เสียง วิดีโอ 3 มิติ**: เรียกว่ามิติ 1, 2, 3
- CNN ⟶ หัวข้อ 4.3.7 • Transformer ⟶ 4.4.8

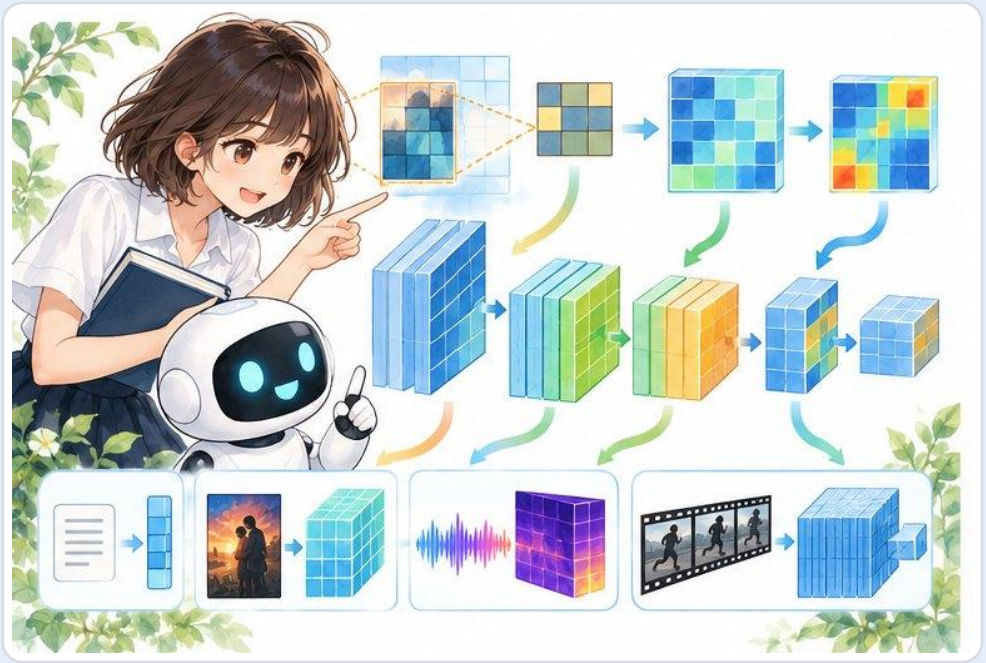
ข้อควรระวัง

- เป็นข้อมูลเพื่อการสอน ขั้นตอนเหมือนระบบจริงทุกประการ ต่างเพียงความยาวเวกเตอร์
- ห้ามสรุปว่าผู้ชมจริงชอบภาพยนตร์ลักษณะใด

4.3

โครงข่ายประสาทแบบคอนโวลูชัน (CNN)

Convolution • Padding • Stride • ReLU • Pooling • Receptive Field



4.3.1 แนวคิดของ CNN : สามแนวคิดหลัก

นิวรอนหนึ่งตัวดูโปสเตอร์ทั้งใบต้องมีน้ำหนัก 150,528 ตัว ถ้าชั้นแรกมี 1,000 นิวรอน ⟶ น้ำหนัก **เกิน 150 ล้านตัว** ช้าและเรียนรู้ยาก

1 มองทีละบริเวณ

Local Connectivity: ดูหน้าต่างเล็ก เช่น 3×3 พิกเซล หรือ 3 คำติดกัน

2 ใช้น้ำหนักร่วมกัน

Weight Sharing: ตัวกรองชุดเดียวเลื่อนทั่วข้อมูล ตรวจลวดลายได้ทุกตำแหน่ง

3 ซ้อนเป็นลำดับชั้น

Hierarchy: ชั้นต้นเห็นขอบ/มุม ชั้นหลังรวมเป็นลวดลายซับซ้อน



CNN = ส่องไฟฉายไล่ดูโปสเตอร์

- ตัวกรองหนึ่งตัว = ไฟฉายที่ไวต่อลวดลายหนึ่งแบบ
- ภาพ ⟶ 7×7×2,048 ⟶ เฉลี่ยเป็นเวกเตอร์เดียว

ตัวอย่างการคำนวณ 4.6 : พารามิเตอร์ของ FC เทียบกับ Conv

$$P_{FC} = (N_{in} + 1) \times N_{out} \quad (4.10)$$

$$P_{conv} = (k_h \times k_w \times C_{in} + 1) \times C_{out} \quad (4.11)$$

โจทย์ ชั้นแรกรับโปสเตอร์ 224×224×3 ส่งออก 64 ช่อง: เทียบ FC 64 นิวรอน กับ Conv ตัวกรอง 3×3 จำนวน 64 ตัว

1 ชั้นเชื่อมต่อเต็ม

- N_{in} = 150,528 • N_{out} = 64
- (150,528 + 1) × 64
- = 9,633,856 ตัว

2 ชั้นคอนโวลูชัน

- 3×3×3 = 27 น้ำหนัก + 1 = 28
- 28 × 64 = 1,792 ตัว
- ไม่ขึ้นกับขนาดภาพ

3 อัตราส่วน

- 9,633,856 / 1,792
- ≈ 5,376 เท่า

ตีความ มองทีละบริเวณ + ใช้น้ำหนักร่วมกัน ลดพารามิเตอร์หลายพันเท่า ทั้งที่ยังส่งออก 64 ช่องเท่าเดิม ⟶ เรียนรู้จากข้อมูลน้อยได้ดีกว่า

ขั้นตอนโดยละเอียด • ตัวอย่างการคำนวณ 4.6 (จำนวนพารามิเตอร์)

ข้อมูลมาจากไหน

- โปสเตอร์ 224 × 224 × 3 จากตารางที่ 4.8
- ตัวกรอง 3 × 3 × 64 ตามชั้นแรกของ VGG
- FC 64 นิวรอน เพื่อให้ส่งออก 64 ค่า เท่ากัน

ชั้น	ต่อหนึ่งหน่วย	× 64	รวม
เชื่อมต่อเต็ม	150,528 + 1 = 150,529	150,529 × 64	9,633,856
คอนโวลูชัน	3×3×3 + 1 = 28	28 × 64	1,792

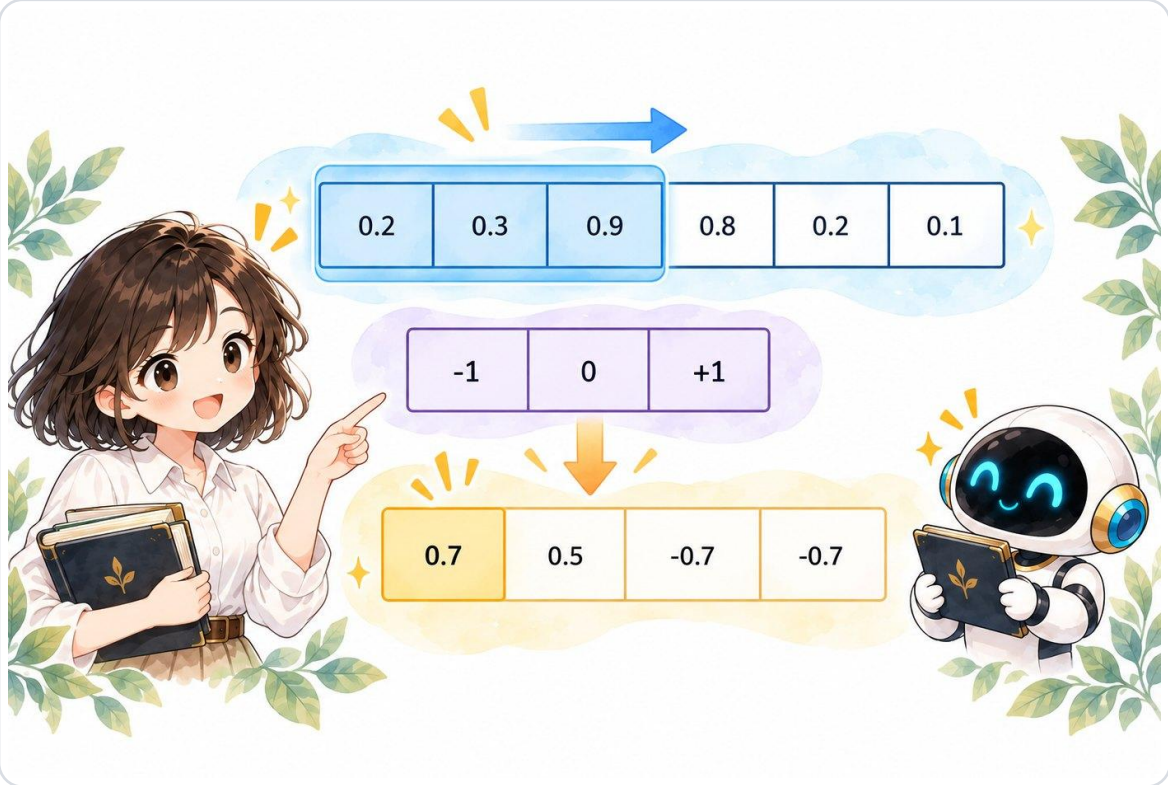
9,633,856 / 1,792 ≈ 5,376 เท่า

4.3.2 คอนโวลูชันหนึ่งมิติ (1D Convolution)

$$y_t = \sum_{j=0}^{k-1} w_j \, x_{t \cdot s + j} + b \qquad (4.12)$$

องค์ประกอบ

- x สัญญาณนำเข้า (หลังเติมขอบ) • w_j น้ำหนักช่องที่ j
- k ความยาวตัวกรอง (Kernel Size)
- s ระยะก้าว (Stride) • b ค่าเอนเอียง
- y_t ค่าส่งออกตำแหน่ง t (เริ่ม 0)
- ไม่กลับด้านตัวกรอง = Cross-correlation แต่ทุกไลบรารีเรียก “Convolution”



ตัวกรอง (−1, 0, +1) = “ขวาลบซ้าย” ⟶ ตรวจจับ **ขอบ**

ตัวอย่างการคำนวณ 4.7 : คอนโวลูชันหนึ่งมิติตรวจจับขอบ

โจทย์ x = (0.2, 0.3, 0.9, 0.8, 0.2, 0.1), w = (−1, 0, 1), b = 0, s = 1, ไม่เติมขอบ จงหาผลลัพธ์ y

1 ทาบตัวกรองแล้วเลื่อนทีละช่อง

t	หน้าต่างที่ทาบ (x_t, x_t+1, x_t+2)	(−1)•x_t + 0•x_t+1 + (1)•x_t+2	y_t
0	(0.2, 0.3, 0.9)	−0.2 + 0 + 0.9	0.7
1	(0.3, 0.9, 0.8)	−0.3 + 0 + 0.8	0.5
2	(0.9, 0.8, 0.2)	−0.9 + 0 + 0.2	−0.7
3	(0.8, 0.2, 0.1)	−0.8 + 0 + 0.1	−0.7

2 รวมผล

- y = (0.7, 0.5, −0.7, −0.7)
- ยาว 4 ช่อง
- สั้นกว่านำเข้า 2 ช่อง

ตีความ ค่าบวก = มืด⟶สว่าง (ขอบขึ้น) • ค่าลบ = สว่าง⟶มืด • ใน CNN จริงค่าตัวกรอง **ได้จากการฝึก** ไม่มีคนกำหนด

ขั้นตอนโดยละเอียด • ตัวอย่างการคำนวณ 4.7 (คอนโวลูชันหนึ่งมิติ)

ข้อมูลมาจากไหน

- x = ความสว่างของพิกเซลหกช่องบนโปสเตอร์หนึ่งแถว (ค่าสมมติ ภาพที่ 4.8)
- w = (−1, 0, +1) = ขวาลบซ้าย → ตัวตรวจจับขอบ
- b = 0, s = 1, ไม่เติมขอบ → ทาบได้ 4 ตำแหน่ง

t	หน้าต่าง	ขวา − ซ้าย	y_t
t = 0	(0.2, 0.3, 0.9)	(−1)(0.2) + 0(0.3) + 1(0.9)	0.7
t = 1	(0.3, 0.9, 0.8)	(−1)(0.3) + 0(0.9) + 1(0.8)	0.5
t = 2	(0.9, 0.8, 0.2)	(−1)(0.9) + 0(0.8) + 1(0.2)	−0.7
t = 3	(0.8, 0.2, 0.1)	(−1)(0.8) + 0(0.2) + 1(0.1)	−0.7

y = (0.7, 0.5, −0.7, −0.7)

ตัวอย่างการคำนวณ 4.8 : ความยาวผลลัพธ์ เมื่อเปลี่ยน p และ s

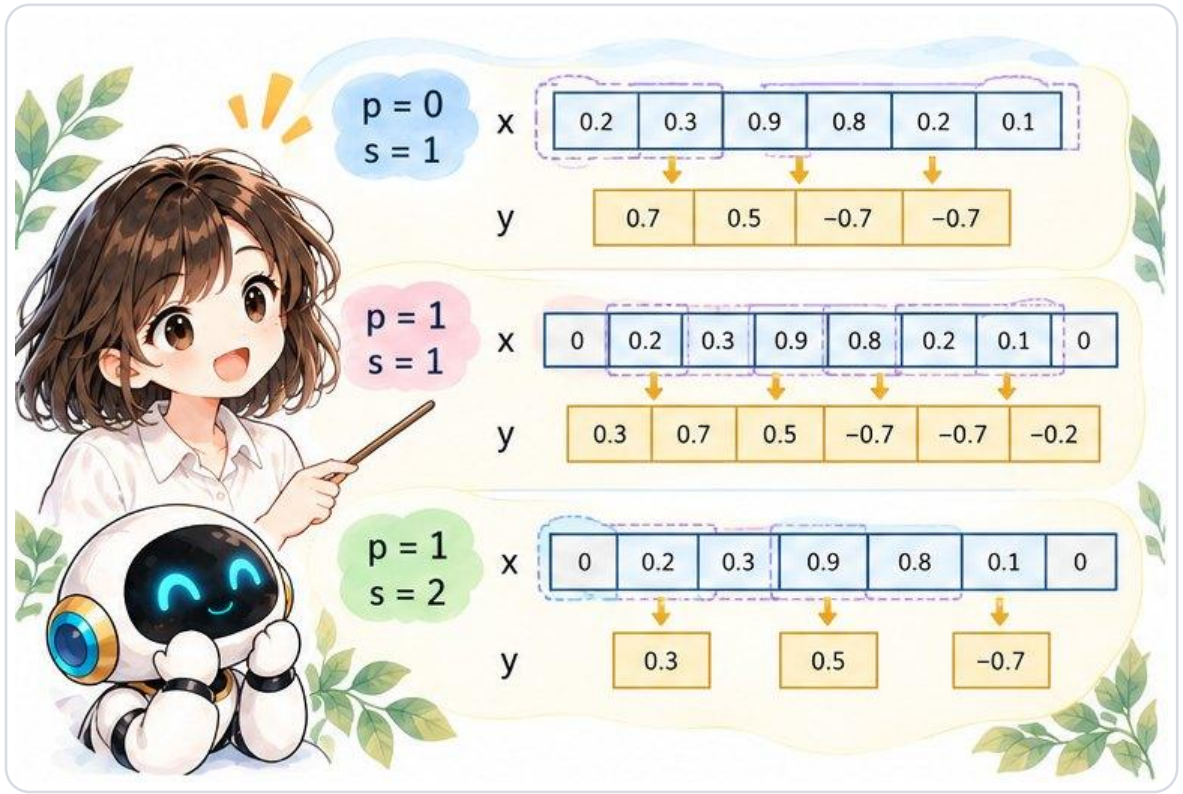
$$L_{out} = \left\lfloor \frac{L_{in} + 2p - k}{s} \right\rfloor + 1 \qquad (4.13)$$

โจทย์ $L_{in} = 6$, $k = 3$ จงหาความยาวผลลัพธ์สามกรณี

1. ไม่เติมขอบ p=0, s=1 : $\lfloor (6 + 0 - 3)/1 \rfloor + 1 = 4$
   $y = (0.7, 0.5, -0.7, -0.7)$

2. Same Padding p=1, s=1 : $\lfloor (6 + 2 - 3)/1 \rfloor + 1 = 6$
   (เท่าเดิม) $y = (0.3, 0.7, 0.5, -0.7, -0.7, -0.2)$

3. p=1, s=2 : $\lfloor (6 + 2 - 3)/2 \rfloor + 1 = \lfloor 2.5 \rfloor + 1 = 3$
   $y = (0.3, 0.5, -0.7)$



ตีความ Padding ให้ขอบภาพถูกมองครบ • Stride 2 ย่อขนาดแทน Pooling ได้

ขั้นตอนโดยละเอียด • ตัวอย่างการคำนวณ 4.8 (การเติมขอบและระยะก้าว)

ข้อมูลมาจากไหน

- แถวพิกเซล 6 ช่องและตัวกรอง 3 ช่องจากตัวอย่าง 4.7
- สามกรณีตามภาพที่ 4.9
- เติมขอบ p = 1 → แถวยาว 8 ช่อง

กรณี	แทนค่าในสมการ (4.13)	Lout	หน้าต่างเริ่มที่
p=0, s=1	$\lfloor (6+0-3)/1 \rfloor + 1$	4	0, 1, 2, 3
p=1, s=1	$\lfloor (6+2-3)/1 \rfloor + 1$	6	0–5
p=1, s=2	$\lfloor (6+2-3)/2 \rfloor + 1 = \lfloor 2.5 \rfloor + 1$	3	0, 2, 4

p=1, s=1 ได้ (0.3, 0.7, 0.5, −0.7, −0.7, −0.2)

4.3.3 ReLU และ Pooling

$$\text{MaxPool}(\mathbf{y})_t = \max_{0 \le j < q} y_{t \cdot s + j} \qquad \text{AvgPool}(\mathbf{y})_t = \frac{1}{q} \sum_{j=0}^{q-1} y_{t \cdot s + j} \tag{4.14}$$

$$\text{GAP}(\mathbf{Y})_c = \frac{1}{HW} \sum_{r=1}^{H} \sum_{u=1}^{W} Y_{r,u,c} \tag{4.15}$$

ไฮเปอร์พารามิเตอร์

- Max Pool 2×2 ระยะก้าว 2 (AlexNet, VGG) ย่อครึ่งหนึ่งพอดี
- ใช้ GAP ครั้งเดียวที่ปลายโครงข่าย ⟶ เวกเตอร์ยาวคงที่

หน้าที่ของแต่ละขั้น

- ReLU ตัดค่าลบ เหลือเฉพาะตำแหน่งที่ “พบลวดลาย”
- Max Pooling เก็บสัญญาณเด่นสุดในหน้าต่าง q
- Avg Pooling เก็บภาพรวมของบริเวณ
- GAP เฉลี่ยทั้งแผนที่ต่อช่อง ⟶ เวกเตอร์ยาว C
- ทำให้ไม่อ่อนไหวต่อการเลื่อนตำแหน่งเล็กน้อย

ตัวอย่างการคำนวณ 4.9 : ReLU และ Pooling บนผลของ 4.7

โจทย์ นำ y = (0.7, 0.5, −0.7, −0.7) ผ่าน ReLU แล้วทำ Max/Average Pooling (q = 2, s = 2) และ Global Average Pooling

1 ReLU

- max(0, 0.7) = 0.7 • max(0, 0.5) = 0.5
- max(0, −0.7) = 0 • max(0, −0.7) = 0
- a = **(0.7, 0.5, 0, 0)**

2 แบ่งหน้าต่างละสองค่า

- หน้าต่าง 1 = (0.7, 0.5)
- หน้าต่าง 2 = (0, 0)

3 Max / Average Pooling

- Max: (max(0.7,0.5), max(0,0)) = **(0.7, 0)**
- Avg: ((0.7+0.5)/2, 0) = **(0.6, 0)**

4 Global Average Pooling

- (0.7 + 0.5 + 0 + 0)/4 = 0.3

ตีความ Max: “ครึ่งแรกมีขอบชัดสุด 0.7” • GAP: “ทั้งแถวมีขอบแบบนี้เฉลี่ย 0.3” = วิธีเปลี่ยน Feature Map เป็นเวกเตอร์

ขั้นตอนโดยละเอียด • ตัวอย่างการคำนวณ 4.9 (ReLU และ Pooling)

ข้อมูลมาจากไหน

- y = ผลของตัวอย่าง 4.7 (แถวพิกเซลโปสเตอร์ภาพที่ 4.8)
- q = 2, s = 2 ตาม AlexNet/VGG
- ความยาวหลัง Pooling = $\lfloor (4 - 2)/2 \rfloor + 1 = 2$

ขั้น	t=0	t=1	t=2	t=3
y	0.7	0.5	−0.7	−0.7
ReLU	0.7	0.5	0	0
Max	max = 0.7		max = 0	
Average	(0.7+0.5)/2 = 0.6		(0+0)/2 = 0	

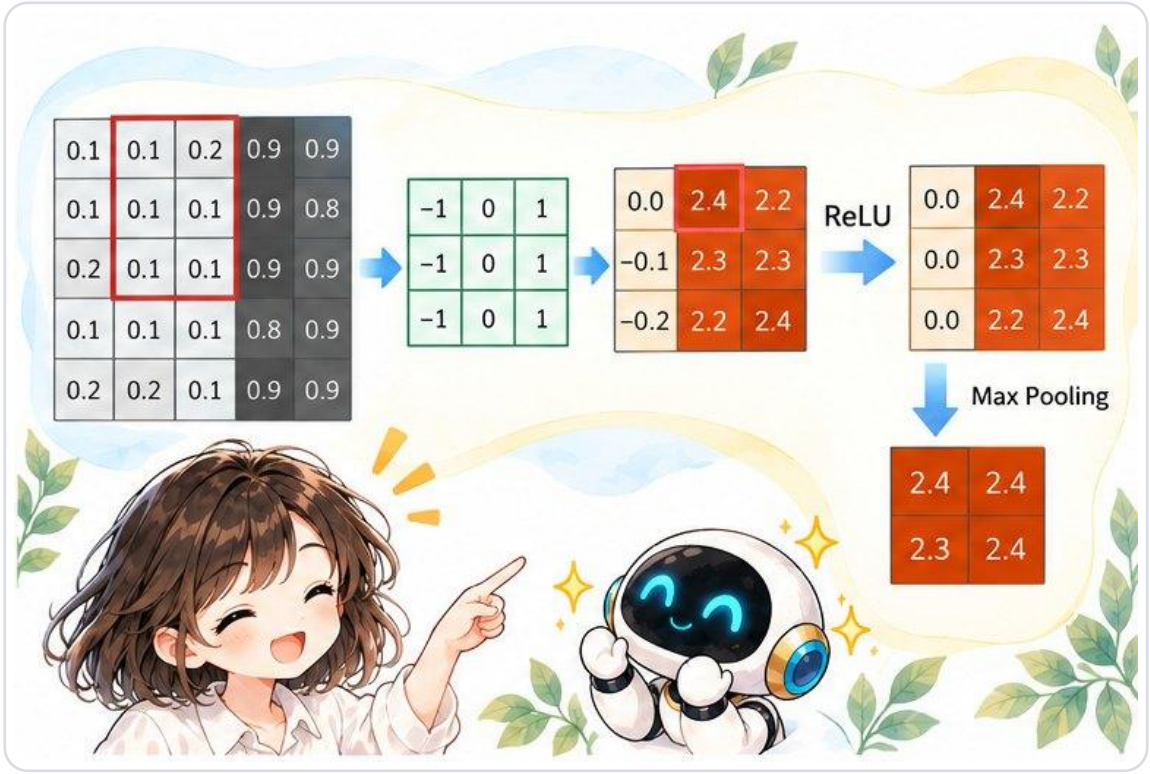
GAP = (0.7 + 0.5 + 0 + 0)/4 = 0.3

4.3.4 คอนโวลูชันสองมิติและหลายช่องสัญญาณ

$$Y_{r,u} = \sum_{c=1}^{C_{in}} \sum_{i=0}^{k-1} \sum_{j=0}^{k-1} K_{i,j,c} X_{r+i,\, u+j,\, c} + b \qquad (4.16)$$

องค์ประกอบ

- X ภาพ/แผนที่ H × W × C_{in}
- K ตัวกรอง k × k × C_{in} (ลึกเท่าจำนวนช่อง)
- $Y_{r,u}$ ค่าส่งออกที่แถว r คอลัมน์ u
- ตัวกรองเลื่อนทั้งแนวนอนและแนวตั้ง แล้วรวมทุกช่องเป็นค่าเดียว
- ตัวกรอง C_{out} ตัว ⟶ เทนเซอร์ H_{out} × W_{out} × C_{out}



ตัวอย่างการคำนวณ 4.10 : คอนโวลูชันสองมิติบนภาพ 5 × 5

โจทย์ ภาพขาวดำ 5×5 (ซ้ายมืด ขวาสว่าง) ตัวกรอง 3×3 ทุกแถว = (−1, 0, 1), b = 0 จงหา Y(0,0), Y(0,1) และขนาดผลลัพธ์

ภาพนำเข้า X

	c0	c1	c2	c3	c4
r0	0.1	0.1	0.2	0.9	0.9
r1	0.1	0.1	0.1	0.9	0.8
r2	0.2	0.1	0.1	0.9	0.9
r3	0.1	0.1	0.1	0.8	0.9
r4	0.2	0.2	0.1	0.9	0.9

ตัวกรอง K

−1	0	1
−1	0	1
−1	0	1

สีเหลือง = หน้าต่างที่ (0,0)

1 ทาบที่ (0, 0)

- ขวาลบซ้ายแต่ละแถว:
- 0.2−0.1 = 0.1
- 0.1−0.1 = 0 • 0.1−0.2 = −0.1
- รวม = **0.0**

2 เลื่อนไปขวา (0, 1)

- แถว (0.1,0.2,0.9) ⟶ 0.8
- (0.1,0.1,0.9) ⟶ 0.8
- (0.1,0.1,0.9) ⟶ 0.8
- รวม = **2.4** ตรงขอบพอดี

3 ขนาดผลลัพธ์

- $\lfloor (5+0-3)/1 \rfloor + 1 = 3 \longrightarrow 3\times3$

4 Y ครบ ⟶ ReLU ⟶ MaxPool 2×2

0.0	2.4	2.2
−0.1	2.3	2.3
−0.2	2.2	2.4

⟶ (2.4, 2.4)

(2.3, 2.4)

ขั้นตอนโดยละเอียด • ตัวอย่างการคำนวณ 4.10 (ครบ 9 ตำแหน่ง)

ข้อมูลมาจากไหน

- ภาพ 5 × 5 ค่าสมมติ: ครึ่งซ้ายมืด (0.1–0.2) ครึ่งขวาสว่าง (0.8–0.9)
- ตัวกรอง = (−1, 0, +1) ซ้อนสามแถว
- ทุกตำแหน่ง: รวม "ขวา − ซ้าย" ของสามแถว

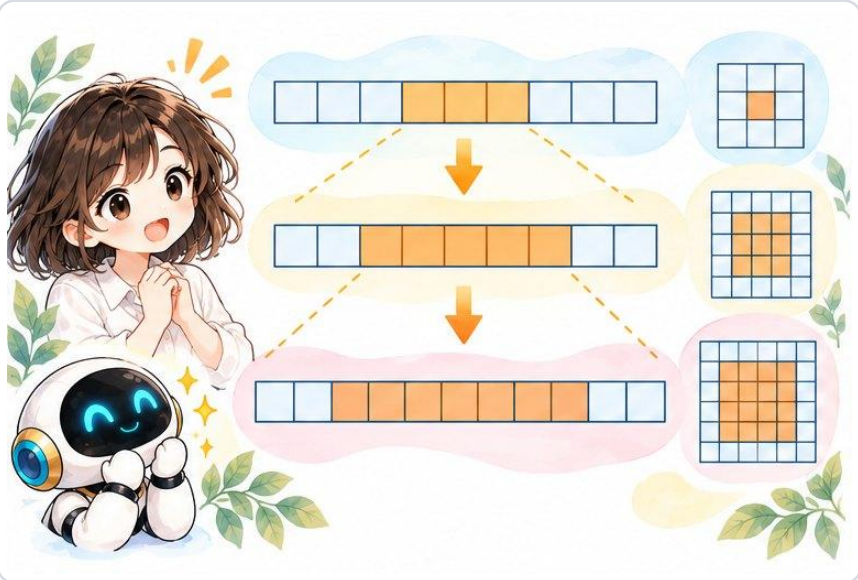
(r, u)	สามแถว: ขวา − ซ้าย	Y
(0, 0)	(0.2 − 0.1) + (0.1 − 0.1) + (0.1 − 0.2)	0.0
(0, 1)	(0.9 − 0.1) + (0.9 − 0.1) + (0.9 − 0.1)	2.4
(0, 2)	(0.9 − 0.2) + (0.8 − 0.1) + (0.9 − 0.1)	2.2
(1, 0)	(0.1 − 0.1) + (0.1 − 0.2) + (0.1 − 0.1)	−0.1
(1, 1)	(0.9 − 0.1) + (0.9 − 0.1) + (0.8 − 0.1)	2.3
(1, 2)	(0.8 − 0.1) + (0.9 − 0.1) + (0.9 − 0.1)	2.3
(2, 0)	(0.1 − 0.2) + (0.1 − 0.1) + (0.1 − 0.2)	−0.2
(2, 1)	(0.9 − 0.1) + (0.8 − 0.1) + (0.9 − 0.2)	2.2
(2, 2)	(0.9 − 0.1) + (0.9 − 0.1) + (0.9 − 0.1)	2.4

ReLU + Max Pool 2×2 → (2.4, 2.4), (2.3, 2.4)

4.3.5 ซ้อนหลายชั้น : ระยะมองเห็น (Receptive Field)

$$r_L = 1 + L(k-1) \qquad (4.17)$$

L จำนวนชั้น • k ขนาดตัวกรอง • r_L ความกว้างที่เห็น (พิกเซล)



โจทย์ ตัวอย่าง 4.11 : 3×3 สามชั้น เทียบ 7×7 หนึ่งชั้น (C ช่องเข้า-ออก ไม่นับ bias)

1 ระยะมองเห็น

- $r_3 = 1 + 3(3-1) = 7$
- = 7×7 หนึ่งชั้น

2 จำนวนน้ำหนัก

- $3\times(3\cdot3\cdot C\cdot C) = 27C^2$
- $7\cdot7\cdot C\cdot C = 49C^2$

3 C = 64 : 110,592 เทียบ 200,704 ตัว (น้อยกว่าเกือบครึ่ง)

ตีความ ชั้นเล็กซ้อนกัน + ReLU คั่น 3 ครั้ง ⟶ แทนลวดลายซับซ้อนกว่า • ข้อจำกัด: ระยะมองเห็นขยายช้า ⟶ นำไปสู่ Transformer

ขั้นตอนโดยละเอียด • ตัวอย่างการคำนวณ 4.11 (ระยะมองเห็น)

ข้อมูลมาจากไหน

- แนวคิดจาก VGG
- C = 64 ช่อง เท่าชั้นแรกในตัวอย่าง 4.6
- ไม่นับค่าเอนเอียง

เรื่อง	แทนค่า	ผล
r หลัง 1 ชั้น	1 + 1×2	3
r หลัง 2 ชั้น	1 + 2×2	5
r หลัง 3 ชั้น	1 + 3×2	7
น้ำหนัก 3 ชั้น 3×3	3 × 9 × 64 × 64	110,592
น้ำหนัก 1 ชั้น 7×7	49 × 64 × 64	200,704

ระยะมองเห็นเท่ากัน แต่ใช้น้ำหนักเพียง 55.1%

สถาปัตยกรรม CNN ที่สำคัญ และขั้นตอนสกัดเวกเตอร์

สถาปัตยกรรม	ปี	แนวคิดเด่น	เวกเตอร์ท้าย
LeNet-5	1998	คอนโวลูชันสลับ Pooling (ตัวเลขเขียนมือ)	84
AlexNet	2012	ReLU, Dropout, GPU ชนะ ImageNet	4,096
VGG-16	2015	ซ้อน 3×3 อย่างเดียว ลึก 16 ชั้น	4,096
ResNet-50	2016	ทางลัด y = F(x) + x ลึก 50 ชั้น	2,048
EfficientNet-B0	2019	ปรับลึก กว้าง ความละเอียดพร้อมกัน	1,280

Residual Connection

- บวกข้อมูลนำเข้ากลับเข้าไป y = F(x) + x ฝึกได้ลึกนับร้อยชั้น และเป็นชิ้นส่วนเดียวกับที่ Transformer ใช้

ขั้นตอนวิธี 4.1 สกัดเวกเตอร์ด้วย CNN

1. เลือก Pretrained Model ให้ตรงสื่อ
2. เตรียมข้อมูลให้ตรงตอนฝึก (224×224, normalize ImageNet)
3. ส่งผ่านชั้น Conv ทั้งหมด **ตัดชั้นจำแนกทิ้ง**
4. GAP ⟶ เวกเตอร์ยาวคงที่ เช่น 2,048
5. บันทึกเวกเตอร์ล่วงหน้า (ออฟไลน์)

4.3.6 การประยุกต์ CNN กับข้อความ ภาพ เสียง และวิดีโอ

สื่อ	จัดข้อมูลเป็น	คอนโวลูชัน	แบบจำลองที่นิยม	ไลบรารี Python
ข้อความ	ลำดับเวกเตอร์คำ $L \times d$	1 มิติ ตามแกนคำ	Text CNN (Kim, 2014)	torch Conv1d, keras Conv1D
ภาพ	พิกเซล $H \times W \times 3$	2 มิติ	ResNet, EfficientNet	torchvision, timm
เสียง	Log-Mel $F \times T$	2 มิติ บนความถี่ × เวลา	VGGish, PANNs CNN14	librosa, torchaudio
วิดีโอ	ชุดเฟรม $T \times H \times W \times 3$	2D รายเฟรม หรือ 3D	C3D, I3D, R(2+1)D, SlowFast	torchvision.video, decord

สิ่งที่ต่างกันมีเพียงสองข้อ

- จัดข้อมูลให้เป็นเทนเซอร์รูปร่างใด
- ตัวกรองเลื่อนไปตามแกนใด

สี่สื่อ หนึ่งวิธีคิด

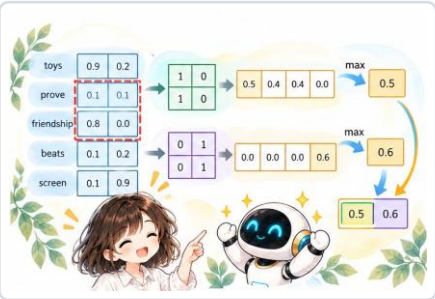
- ทุกสื่อคือ “ตาราง” ที่แกนต่างกัน → เลื่อนตัวกรอง → ReLU → Pooling → เฉลี่ยเป็นเวกเตอร์

ตัวอย่างการคำนวณ 4.12 : Text CNN ขนาดจิ๋ว

โจทย์ “toys prove friendship beats screen” เวกเตอร์คำ 2 มิติ • F1 ดูคอลัมน์ซ้าย F2 ดูคอลัมน์ขวา (น้ำหนัก 1) กว้าง 2 คำ b = −0.5

คำ	มิติ 1	มิติ 2
toys	0.9	0.2
prove	0.1	0.1
friendship	0.8	0.0
beats	0.1	0.2
screen	0.1	0.9

คู่คำติดกัน	F1 (มิติ 1)	ReLU	F2 (มิติ 2)	ReLU
toys–prove	0.9+0.1−0.5 = 0.5	0.5	0.3−0.5 = −0.2	0
prove–friendship	0.1+0.8−0.5 = 0.4	0.4	0.1−0.5 = −0.4	0
friendship–beats	0.8+0.1−0.5 = 0.4	0.4	0.2−0.5 = −0.3	0
beats–screen	0.1+0.1−0.5 = −0.3	0	0.2+0.9−0.5 = 0.6	0.6
Max-over-time		h1 = 0.5		h2 = 0.6



- เวกเตอร์ประโยค h = (0.5, 0.6) แต่ละมิติตอบว่า “มีวลีแบบที่ตัวกรองชอบหรือไม่” โดยไม่สนตำแหน่ง
- ข้อจำกัด: ตัวกรองเห็นเพียง 2 คำติดกัน ความหมายที่ขึ้นกับคำไกล ๆ จะมองไม่เห็น

ที่มาของข้อมูล • ตัวอย่างการคำนวณ 4.12 (Text CNN)

ข้อมูลมาจากไหน

- ประโยคสรุปใจความเรื่องย่อ i1 Toy Story 5 (ตารางที่ 4.6): toys ×3, prove, friendship, screen
- เวกเตอร์คำ 2 มิติ (ภาพที่ 4.12): มิติ 1 = ของเล่น/มิตรภาพ • มิติ 2 = หน้าจอ
- F1 = มิติ 1 ของสองคำ − 0.5 • F2 = มิติ 2 ของสองคำ − 0.5

คู่คำ	F1	ReLU	F2	ReLU
toys–prove	0.9 + 0.1 − 0.5 = 0.5	0.5	0.2 + 0.1 − 0.5 = −0.2	0.0
prove–friendship	0.1 + 0.8 − 0.5 = 0.4	0.4	0.1 + 0.0 − 0.5 = −0.4	0.0
friendship–beats	0.8 + 0.1 − 0.5 = 0.4	0.4	0.0 + 0.2 − 0.5 = −0.3	0.0
beats–screen	0.1 + 0.1 − 0.5 = −0.3	0.0	0.2 + 0.9 − 0.5 = 0.6	0.6

h = (max F1, max F2) = (0.5, 0.6)

ตัวอย่างการคำนวณ 4.13 : ติดตามรูปร่างข้อมูลผ่าน CNN ของสี่สื่อ

$$T = 1 + \left\lfloor \frac{N - n_{win}}{n_{hop}} \right\rfloor \qquad (4.18)$$

$$\Delta = \frac{F_{total}}{T_v}, \quad t_j = \frac{\Delta}{2} + j\,\Delta \qquad (4.19)$$

สื่อ	การคำนวณรูปร่างทีละขั้น	เวกเตอร์
ข้อความ (Text CNN)	60 โทเคน × 300 มิติ • k = 3, 100 ตัวกรอง: (60−3)/1 + 1 = 58 ⟶ 58×100 ⟶ Max-over-time	100
ภาพ (ResNet-50)	224×224×3 • ย่อครึ่ง 5 ครั้ง: $224/2^5 = 7$ ⟶ 7×7×2,048 ⟶ GAP	2,048
เสียง (PANNs)	N = 10.24 × 16,000 = 163,840 • T = 1 + (163,840−400)/160 ปัดลง = 1,022 ⟶ เติมเป็น 1,024 ⟶ 128×1,024	2,048
วิดีโอ (รายเฟรม)	2 นาที × 24 fps = 2,880 เฟรม • Δ = 2,880/16 = 180 ⟶ t = 90, 270, ..., 2,790 ⟶ 16×2,048	2,048

ตีความ ไม่ว่าข้อมูลนำเข้าจะมีรูปร่างแบบใด ปลายทางคือ **เวกเตอร์ยาวคงที่หนึ่งเส้นต่อหนึ่งรายการ**

ขั้นตอนโดยละเอียด • ตัวอย่างการคำนวณ 4.13 (รูปร่างข้อมูล)

ข้อมูลมาจากไหน

- รูปแบบข้อมูลนำเข้าของทุกเรื่อง: ตารางที่ 4.8
- สถาปัตยกรรม: ตารางที่ 4.11
- เสียง 16,000 Hz • วิดีโอ 24 เฟรม/วินาที

สื่อ	คำนวณ	ผล
ข้อความ	$\lfloor (60-3)/1 \rfloor + 1$	58 × 100 → 100
ภาพ	$224/2^5 = 7$	7×7×2,048 → 2,048
เสียง	10.24 × 16,000	N = 163,840
เสียง	$1 + \lfloor 163{,}440/160 \rfloor$	1,022 → 1,024
วิดีโอ	120 × 24	2,880 เฟรม
วิดีโอ	2,880/16 • 90 + 180j	Δ = 180

ทุกสื่อจบที่เวกเตอร์ยาวคงที่หนึ่งเส้นต่อเรื่อง

4.3.7 เวกเตอร์ข้อความ 12 มิติจากเทคนิค CNN (ตารางที่ 4.12)

รหัส	tech	mind	spac	rom	crim	act	fam	mus	fant	toys	soc	amb
i1	.2212	.1327	.0822	·	.0885	.1327	.3482	·	.0885	.8787	·	·
i2	·	.9641	·	.0266	·	.0391	.2001	·	·	·	·	.1679
i3	.3654	.1741	.0523	·	·	.1218	.7835	·	.4525	·	·	·
i4	·	.1940	·	.8027	.3051	.0555	.4528	·	·	·	.1294	·
i5	·	.1189	·	.3029	·	.1189	.4649	.2076	.6229	·	.2076	.4356
i6	·	.1369	·	.6710	·	·	.7270	·	·	·	.0498	·
i7	·	.0330	·	·	.9065	.4121	.0769	.0384	·	·	·	·
i8	·	·	.0933	·	.0466	.9728	.0801	·	·	·	.1865	.0401
i9	·	.1355	·	·	.1734	.0758	·	·	.0379	·	.9473	.2167
i10	·	.2055	.4034	·	·	.5857	.0229	·	.1826	·	.6390	.0990
i11	.6062	.2146	.4942	.0727	.0606	.4418	.0467	·	.0606	·	.3637	·
i12	.2656	.2486	.8730	.0181	·	.1215	.2690	·	·	.0912	.0987	·

Static Embedding ของบทที่ 3 ปรับความยาวเป็น 1 แล้ว · จุด · แทนค่า 0 · ละเลขศูนย์หน้าจุด

เวกเตอร์ภาพ เสียง วิดีโอ 3 มิติจากเทคนิค CNN (ตารางที่ 4.13)

รหัส	I1	I2	I3	A1	A2	A3	V1	V2	V3
i1	0.90	0.60	0.77	0.50	0.65	0.19	0.57	0.50	1.00
i2	0.80	0.80	0.77	0.50	0.80	0.26	0.52	0.42	1.00
i3	0.70	0.50	0.69	0.30	0.70	0.41	0.47	0.35	1.00
i4	0.50	0.70	0.24	0.30	0.70	0.34	0.47	0.35	0.00
i5	0.70	0.60	0.47	0.40	0.80	0.19	0.42	0.42	0.00
i6	0.60	0.80	0.32	0.20	0.70	0.19	0.37	0.27	0.00
i7	0.20	0.30	0.17	0.90	0.40	0.71	0.72	0.80	0.00
i8	0.60	0.90	0.32	1.00	0.45	0.64	0.77	0.87	0.00
i9	0.40	0.30	0.17	0.30	0.55	0.56	0.42	0.42	0.00
i10	0.50	0.80	0.17	0.80	0.45	0.71	0.62	0.57	0.00
i11	0.30	0.20	0.17	0.60	0.45	0.64	0.57	0.50	0.00
i12	0.30	0.30	0.17	0.50	0.60	0.64	0.52	0.42	0.00

อ่านตาราง

- I = ภาพ (ฟ้า) • A = เสียง (เขียว) • V = วิดีโอ (ส้ม)
- เรียกว่า “มิติที่ 1, 2, 3” ไม่ได้ตั้งชื่อตามความหมาย
- แอนิเมชัน i1–i3 มี V3 = 1
- ก่อนใช้ต้อง **ลบค่าเฉลี่ยคลัง + ปรับความยาว** (หัวข้อ 4.5.1)

ตัวอย่างการคำนวณ 4.14 : โคไซน์ข้อความ i1–i2 จากเทคนิค CNN

โจทย์ หาโคไซน์ระหว่างเวกเตอร์ข้อความของ i1 Toy Story 5 และ i2 Inside Out 2 (ตารางที่ 4.12)

1 มิติที่ไม่เป็นศูนย์พร้อมกัน

- mind, action, family

2 ผลคูณจุด

มิติ	i1	i2	i1 × i2
mind	0.1327	0.9641	0.1279
action	0.1327	0.0391	0.0052
family	0.3482	0.2001	0.0697
a•b			0.2028

3 หารด้วยความยาว

- $\|a\| = \|b\| = 1.0000$
- cos = 0.2028

ตีความ แอนิเมชันครอบครัวเหมือนกัน แต่ข้อความคล้ายเพียง 0.2028 เพราะ i1 เด่น toys ส่วน i2 เด่น mind ⟶ ภาพ/วิดีโอจะช่วยเห็นความใกล้กัน

ที่มาของข้อมูล • ตัวอย่างการคำนวณ 4.14 (i1–i2 เทคนิค CNN)

ข้อมูลมาจากไหน

- i1, i2 แอนิเมชันครอบครัว (ตารางที่ 4.4) • u1 ให้ 5 ทั้งคู่ (ตารางที่ 4.5)
- เวกเตอร์: Static Embedding จากเรื่องย่อ (ตารางที่ 4.6)
- i1: toy/toys 5 ครั้ง → toys .8787
- i2: mind, memory ×2, dream ×4 → mind .9641
- dream (Static) = mind 0.5 + ambition 0.5

มิติ	i1	i2	i1×i2
tech	0.2212	0.0000	0.0000
mind	0.1327	0.9641	0.1279
space	0.0822	0.0000	0.0000
romance	0.0000	0.0266	0.0000
crime	0.0885	0.0000	0.0000
action	0.1327	0.0391	0.0052
family	0.3482	0.2001	0.0697
fantasy	0.0885	0.0000	0.0000
toys	0.8787	0.0000	0.0000
ambition	0.0000	0.1679	0.0000
รวม			0.2028

cos = 0.1279 + 0.0052 + 0.0697 = 0.2028

เมื่อใดควรเลือก CNN

+ จุดเด่น

- พารามิเตอร์น้อย ฝึกเร็ว
- ใช้ข้อมูลน้อยกว่า Transformer
- รันบนอุปกรณ์เล็กได้
- มี Pretrained ให้เลือกมาก

– ข้อจำกัด

- เห็นเฉพาะบริเวณใกล้
- ความสัมพันธ์ระยะไกลต้องซ้อนชั้นมาก
- Static: คำหนึ่งคำมีความหมายเดียว

✓ ควรเลือกเมื่อ

- ข้อมูลฝึกน้อย
- ต้องการความเร็วสูง
- ลวดลายเฉพาะที่ เช่น สี พื้นผิว จังหวะเสียงสั้น ๆ

โปรแกรม 4.7 : โคไซน์ระหว่างเรื่องที่ u1 ชอบ (CNN)

สื่อ	i1–i2	i1–i3	i2–i3
T ข้อความ	0.2028	0.4373	0.3294
I ภาพ	0.9127	0.9340	0.7968
A เสียง	0.8194	0.3320	0.4839
V วิดีโอ	0.9921	0.9719	0.9937

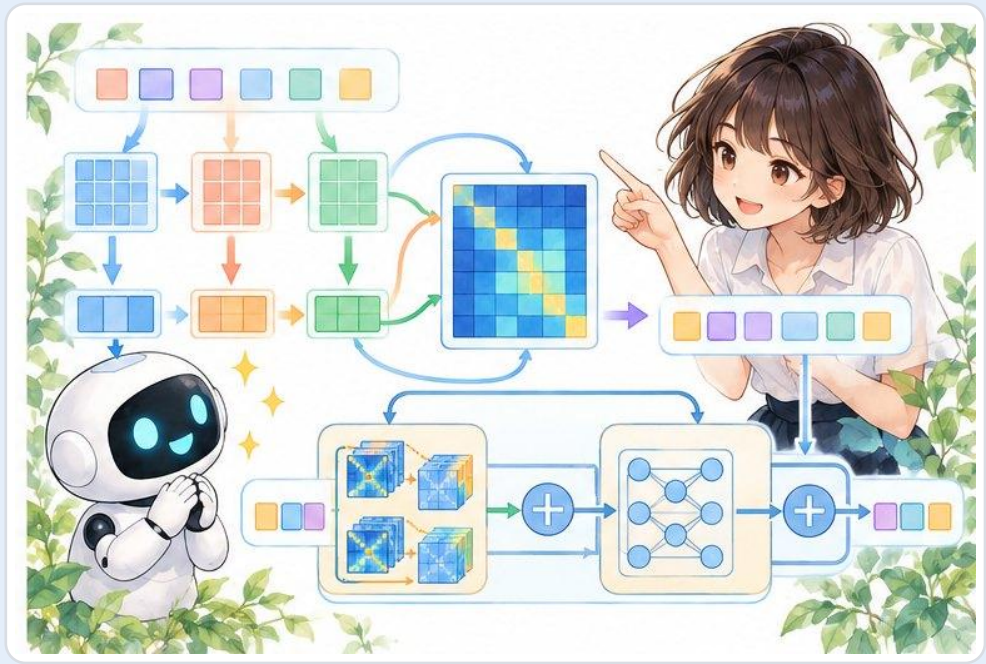
สังเกต

- ภาพและวิดีโอเห็นแอนิเมชันสามเรื่อง “คล้ายกันมาก” (0.80–0.99)
- ข้อความเห็นต่างกันมาก (0.20–0.44)

4.4

Transformer

Self-Attention • Multi-Head • Positional Encoding • Add & Norm • FFN • Pooling



4.4.1 แนวคิดของ Transformer

1 **ทุกสื่อคือลำดับของชิ้น**

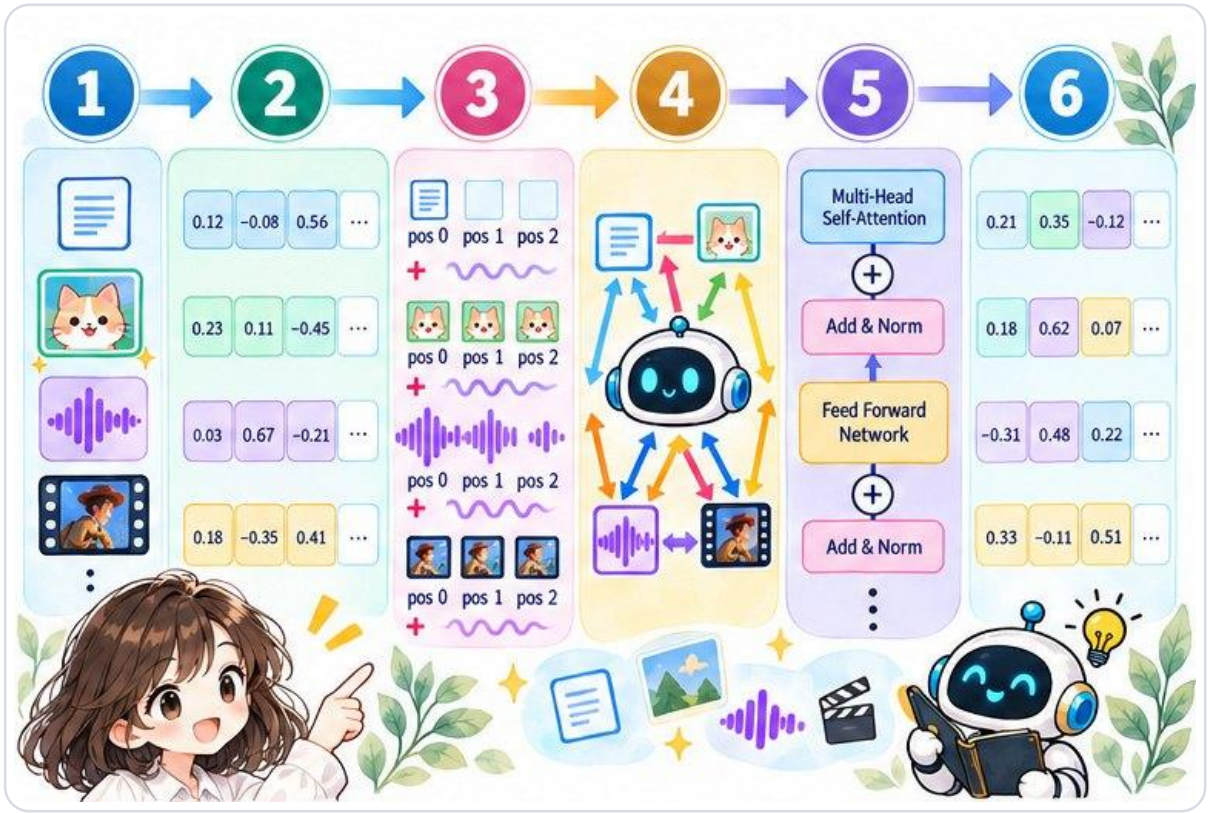
คำย่อย • แพตช์ภาพ • แพตช์สเปกโทรแกรม • ก้อนเวลา×พื้นที่

2 **ทุกชิ้นปรึกษากันตั้งแต่ชั้นแรก**

Self-Attention: เลือกเองว่าจะดึงข้อมูลจากชิ้นใด ไม่ว่าใกล้หรือไกล

3 **ซ้อนบล็อกเดียวกันหลายชั้น**

Attention + FFN พร้อมทางลัดและการปรับมาตรฐาน



ห้องประชุม CNN = กระซิบต่อกันทีละคน • Transformer = ทุกคนถามใครก็ได้ในห้องโดยตรง ราคาที่จ่ายคือจำนวนคู่สนทนา **เพิ่มเป็นกำลังสอง** ของจำนวนคน

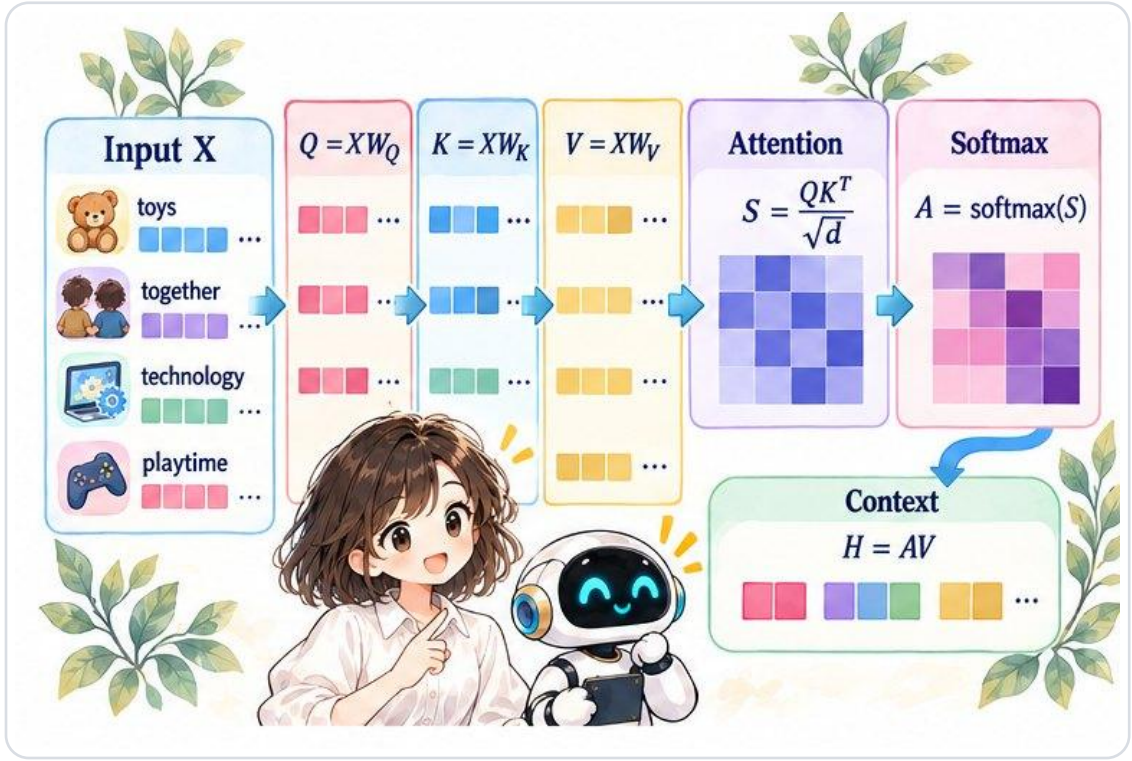
4.4.2 Self-Attention : Query, Key, Value

$$\mathbf{Q} = \mathbf{X}\mathbf{W}_Q, \quad \mathbf{K} = \mathbf{X}\mathbf{W}_K, \quad \mathbf{V} = \mathbf{X}\mathbf{W}_V \tag{4.20}$$

$$\text{Attention}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \text{softmax}\left(\frac{\mathbf{Q}\mathbf{K}^\top}{\sqrt{d_k}}\right)\mathbf{V} \tag{4.21}$$

$$\alpha_{ij} = \frac{\exp(s_{ij})}{\sum_{l=1}^{n}\exp(s_{il})}, \qquad \mathbf{h}_i = \sum_{j=1}^{n}\alpha_{ij}\,\mathbf{v}_j \tag{4.22}$$

- Query “คำถาม” ว่ากำลังมองหาอะไร • Key “ป้ายชื่อ” ว่ามีอะไรให้ • Value “เนื้อหา” ที่ส่งต่อ
- หารด้วย $\sqrt{d_k}$ กันคะแนนโตเกิน (ความแปรปรวน $d_k \longrightarrow 1$)



ถ้าไม่หาร $\sqrt{d_k}$ เมื่อ $d_k = 64$ softmax จะเทน้ำหนักให้โทเคนเดียว เกรเดียนต์ของโทเคนอื่น $\approx 0 \longrightarrow$ ฝึกยาก

ตัวอย่างการคำนวณ 4.15 : Self-Attention เมื่อ W = I

โจทย์ toy = (1, 0), friend = (0.8, 0.6), screen = (0, 1) • $W_Q = W_K = W_V = I$, $d_k = 2$ • จงหาเวกเตอร์ตามบริบทของ toy

1 คะแนนของ toy กับทุกโทเคน

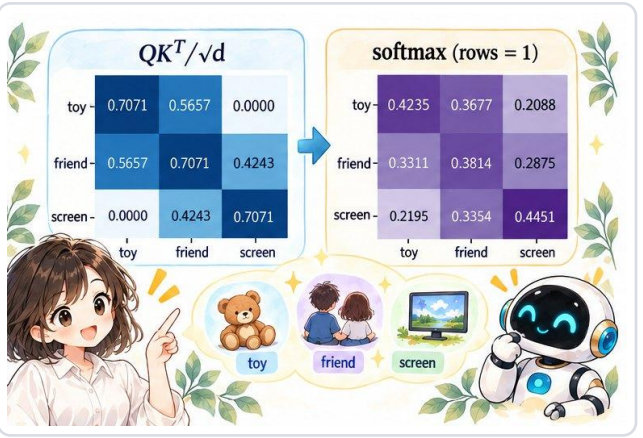
- toy•toy = 1(1)+0(0) = 1 • toy•friend = 1(0.8)+0(0.6) = 0.8 • toy•screen = 0
- หาร √2 = 1.4142 ⟶ s = (0.7071, 0.5657, 0.0000)

2 softmax

- $e^{0.7071} = 2.0281$ • $e^{0.5657} = 1.7607$ • $e^{0} = 1$
- รวม = 4.7888 ⟶ α = (0.4235, 0.3677, 0.2088)
  รวม = 1

3 รวม Value ด้วยน้ำหนัก

- h_{toy} = 0.4235(1, 0) + 0.3677(0.8, 0.6) + 0.2088(0, 1)
- = (0.7176, 0.4294)



ตีความ toy ให้น้ำหนักตัวเองมากสุด รองลงมาคือ friend ⟶ ความหมาย “ปรับตามบริบท”

ขั้นตอนโดยละเอียด • ตัวอย่างการคำนวณ 4.15 (W = I)

ข้อมูลมาจากไหน

- toy, friend, screen = สามแนวคิดหลักของเรื่องย่อ i1 (ตารางที่ 4.6)
- เวกเตอร์สมมติ: มิติ 1 = ของเล่น • มิติ 2 = หน้าจอ
- W = I → Q = K = V = เวกเตอร์ของโทเคน

ผู้ถาม	α→toy	α→friend	α→screen	h
toy	0.4235	0.3677	0.2088	(0.7176, 0.4294)
friend	0.3311	0.3814	0.2875	(0.6363, 0.5163)
screen	0.2195	0.3354	0.4451	(0.4878, 0.6464)

h(toy) = 0.4235(1,0) + 0.3677(0.8,0.6) + 0.2088(0,1) = (0.7176, 0.4294)

ตัวอย่างการคำนวณ 4.16 : เมื่อ Q, K, V ต่างกัน (1/2)

โจทย์ ใช้โทเคนชุดเดิม แต่เมทริกซ์ฉายต่างกัน จงหาเวกเตอร์ตามบริบทของ toy (ขั้นที่ 1: สร้าง Q, K, V)

W_Q

1.0	0.5
0.0	1.0

W_K

1.0	0.0
0.5	1.0

W_V

0.5	0.0
0.0	1.0

โทเคน x	Key = x W_K	Value = x W_V
toy (1, 0)	(1.0, 0.0)	(0.5, 0.0)
friend (0.8, 0.6)	(0.8+0.3, 0.6) = (1.1, 0.6)	(0.4, 0.6)
screen (0, 1)	(0.5, 1.0)	(0.0, 1.0)

Q ถาม • K ให้ป้าย • V ส่งเนื้อหา — สามบทบาทจากเวกเตอร์เดียวกัน

1 Query ของ toy

- $q_{toy} = (1, 0)\ W_Q$
- $= (1\times1.0 + 0\times0.0,\ 1\times0.5 + 0\times1.0) = \mathbf{(1.0, 0.5)}$

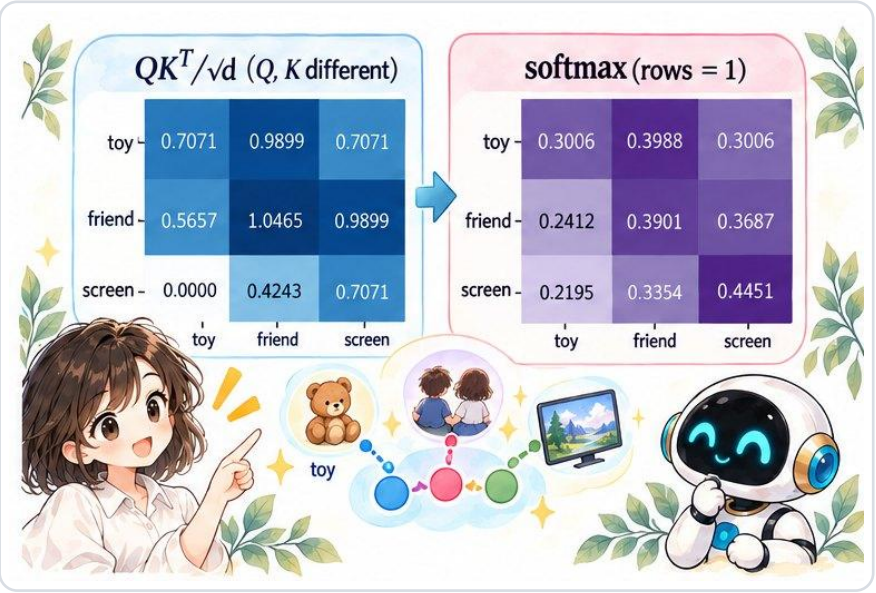
วิธีคูณเวกเตอร์แถว × เมทริกซ์

- มิติที่ j ของผล = $\sum x_i\ W_{ij}$
- เช่น friend•W_K มิติ 1 = 0.8(1.0) + 0.6(0.5) = 1.1

ตัวอย่างการคำนวณ 4.16 : คะแนน น้ำหนัก และผลลัพธ์ (2/2)

2 คะแนนและ softmax

คู่ (toy, •)	q_toy • k	÷ √2	e^s	α
toy	1(1.0) + 0.5(0.0) = 1.0	0.7071	2.0281	0.3006
friend	1(1.1) + 0.5(0.6) = 1.4	0.9899	2.6909	0.3988
screen	1(0.5) + 0.5(1.0) = 1.0	0.7071	2.0281	0.3006
รวม			6.7471	1.0000



3 รวม Value

- h_{toy} = 0.3006(0.5, 0) + 0.3988(0.4, 0.6) + 0.3006(0, 1) = (0.3098, 0.5399)

ตีความ toy หันไปให้น้ำหนัก friend มากที่สุด (0.3988) แทนตัวเอง ⟶ การฝึกเมทริกซ์ฉาย = สอนว่าโทเคนควรสนใจโทเคนใด

ขั้นตอนโดยละเอียด • ตัวอย่างการคำนวณ 4.16 (Q, K, V ต่างกัน)

ข้อมูลมาจากไหน

- โทเคนและเวกเตอร์เดิมจากตัวอย่าง 4.15 (แนวคิดหลักของเรื่องย่อ i1)
- W_Q, W_K, W_V เป็นค่าสมมติ
- friend: Q = (0.8•1 + 0.6•0, 0.8•0.5 + 0.6•1) = (0.80, 1.00)

โทเคน	Q	K	V
toy	(1.00, 0.50)	(1.00, 0.00)	(0.50, 0.00)
friend	(0.80, 1.00)	(1.10, 0.60)	(0.40, 0.60)
screen	(0.00, 1.00)	(0.50, 1.00)	(0.00, 1.00)

α(toy) = (0.3006, 0.3988, 0.3006) → h(toy) = (0.3098, 0.5399)

4.4.3 Multi-Head และตัวอย่างการคำนวณ 4.17

$$\mathrm{MHA}(\mathbf{X}) = [\mathbf{H}^{(1)} \| \mathbf{H}^{(2)} \| \cdots \| \mathbf{H}^{(h)}]\, \mathbf{W}_O, \qquad d_k = \frac{d}{h} \tag{4.23}$$

โจทย์ หัวที่ 1 = ตัวอย่าง 4.15 • หัวที่ 2 ใช้ $W_Q = W_K = [0\ 0\ ;\ 0\ 1]$ (ดูเฉพาะมิติ 2), $W_V = I$ • W_O เฉลี่ยสองหัว

1 หัวที่ 1

- จาก 4.15:
- $h^{(1)} = (0.7176,\ 0.4294)$

2 หัวที่ 2

- $q_{toy} = (1,0)W_Q = (0,\ 0) \longrightarrow$ คะแนน 0 ทุกตัว
- $\alpha = (1/3,\ 1/3,\ 1/3)$
- $h^{(2)} = (0.6000,\ 0.5333)$

3 ต่อแล้วฉาย

- $(0.7176,\ 0.4294,\ 0.6000,\ 0.5333)$
- เฉลี่ยมิติคู่กัน $\longrightarrow$ $(0.6588,\ 0.4814)$

ตีความ หัว 1 มองว่า toy ใกล้ friend • หัว 2 “ไม่มีความเห็น” • BERT-base/ViT-B: $d = 768$, $h = 12 \longrightarrow d_k = 64$ • พารามิเตอร์ Attention $\approx 4d^2 = 2{,}359{,}296$ ต่อชั้น

ขั้นตอนโดยละเอียด • ตัวอย่างการคำนวณ 4.17 (สองหัว)

ข้อมูลมาจากไหน

- หัวที่ 1 = ผลของตัวอย่าง 4.15
- หัวที่ 2: WQ = WK ดูเฉพาะมิติ 2 (หน้าจอ), WV = I
- WO = เฉลี่ยมิติคู่กัน (ค่าสมมติ)

ขั้น	การคำนวณ	ผล
q_toy (หัว 2)	(1, 0) WQ	(0, 0)
คะแนน	(0,0)·k_j	0, 0, 0
softmax	$e^0/3$	(1/3, 1/3, 1/3)
h (หัว 2)	(1.8, 1.6)/3	(0.6000, 0.5333)
ต่อสองหัว	h1 ‖ h2	(0.7176, 0.4294, 0.6000, 0.5333)
WO	เฉลี่ยมิติคู่กัน	(0.6588, 0.4814)

หัวที่ 2 "ไม่มีความเห็น" → เฉลี่ยทุกโทเคนเท่ากัน

4.4.4 การฝังตำแหน่ง และตัวอย่างการคำนวณ 4.18

$$PE_{(pos,\,2i)} = \sin\left(\frac{pos}{10000^{2i/d}}\right), \qquad PE_{(pos,\,2i+1)} = \cos\left(\frac{pos}{10000^{2i/d}}\right) \tag{4.24}$$

Self-Attention **ไม่รู้ลำดับ**: “toys beat the screen” กับ “the screen beat toys” ได้น้ำหนักเหมือนกัน ⟶ ต้องบวกเวกเตอร์ตำแหน่ง

โจทย์ หา PE ของ pos = 1 และ pos = 2 เมื่อ d = 4 แล้วบวกกับโทเคน (0.5, 0.5, 0.5, 0.5) ที่ตำแหน่ง 1

1 ตัวหาร

- i = 0: $10000^{0/4} = 1$
- i = 1: $10000^{2/4} = 100$

2 PE ของ pos 1, 2

- pos 1: (sin1, cos1, sin0.01, cos0.01) = (0.8415, 0.5403, 0.0100, 1.0000)
- pos 2: (0.9093, −0.4161, 0.0200, 0.9998)

4 บวกกับโทเคน

- (1.3415, 1.0403, 0.5100, 1.5000)



คู่แรกเปลี่ยนเร็ว คู่หลังเปลี่ยนช้า เหมือนเข็มวินาที–ชั่วโมง

ขั้นตอนโดยละเอียด • ตัวอย่างการคำนวณ 4.18 (การฝังตำแหน่ง)

ข้อมูลมาจากไหน

- d = 4 และโทเคน (0.5, 0.5, 0.5, 0.5) เป็นค่าสมมติ
- ค่าคงที่ 10,000 จาก Vaswani et al. (2017)
- ตัวหาร: คู่แรก $10000^{0} = 1$ • คู่ที่สอง 10000^0.5 = 100
- มุมเป็นเรเดียน

pos	มุม 1	มุม 2	PE
0	0 rad	0.00 rad	(0.0000, 1.0000, 0.0000, 1.0000)
1	1 rad	0.01 rad	(0.8415, 0.5403, 0.0100, 1.0000)
2	2 rad	0.02 rad	(0.9093, −0.4161, 0.0200, 0.9998)

โทเคนที่ pos = 1 → (1.3415, 1.0403, 0.5100, 1.5000)

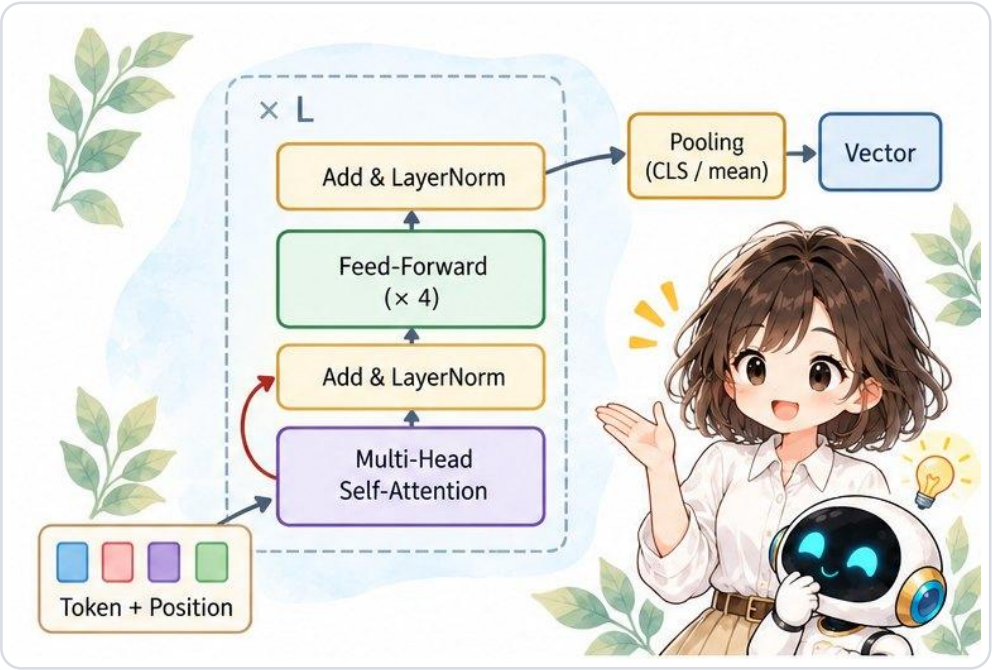
4.4.5 บล็อก Transformer Encoder : Add & Norm และ FFN

$$\mathbf{z} = LN(\mathbf{x} + MHA(\mathbf{x})), \qquad \mathbf{h} = LN(\mathbf{z} + FFN(\mathbf{z})) \tag{4.25}$$

$$LN(\mathbf{r}) = \gamma \odot \frac{\mathbf{r} - \mu}{\sqrt{\sigma^2 + \varepsilon}} + \beta \tag{4.26}$$

$$FFN(\mathbf{z}) = \mathbf{W}_2\,\mathrm{ReLU}(\mathbf{W}_1\mathbf{z} + \mathbf{b}_1) + \mathbf{b}_2 \tag{4.27}$$

- x + MHA(x) ทางลัด (Residual) แบบ ResNet • $\boldsymbol{\mu}$, $\boldsymbol{\sigma}$ ค่าเฉลี่ย/ส่วนเบี่ยงเบนของทุกมิติในเวกเตอร์
- $\boldsymbol{\gamma}$, $\boldsymbol{\beta}$ เริ่มที่ 1 และ 0 • W_1 ขยาย d ⟶ d_{ff} = 4d • W_2 ย่อกลับ



BERT-base/ViT-B: L = 12 ชั้น, d_{ff} = 3,072 • MiniLM: 6 ชั้น, d = 384 (เร็วกว่าหลายเท่า)

ตัวอย่างการคำนวณ 4.19 : Add & Norm ของโทเคนหนึ่งตัว

โจทย์ $x = (0.6, 0.2, -0.4)$ • Attention ให้ $(0.4, 0.8, -0.6)$ • FFN ให้ $(0.2929, -0.7071, 0.4142)$ • $\gamma = 1$, $\beta = 0$, และ ε

1 ทางลัดครั้งที่ 1

- $r_1 = x + \text{Attention}$
- $= (1.0, 1.0, -1.0)$

2 Layer Norm ครั้งที่ 1

- $\mu = (1 + 1 - 1)/3 = 0.3333$ • $\sigma^2 = [(0.6667)^2 + (0.6667)^2 + (-1.3333)^2]/3 = 0.8889 \longrightarrow \sigma = 0.9428$
- $z = (r_1 - \mu)/\sigma = (0.7071, 0.7071, -1.4142)$

3 ทางลัดครั้งที่ 2

- $r_2 = z + \text{FFN}(z)$
- $= (0.7071+0.2929, 0.7071-0.7071, -1.4142+0.4142)$
- $= (1.0, 0, -1.0)$

4 Layer Norm ครั้งที่ 2

- $\mu = 0$ • $\sigma = \sqrt{(2/3)} = 0.8165$
- $h = (1.2247, 0, -1.2247)$

ตีความ ทางลัดทำให้ x ไม่หาย ชั้นย่อยแค่ “เพิ่มเติม” • LN ทำให้ทุกเวกเตอร์มีค่าเฉลี่ย 0 ส่วนเบี่ยงเบน 1 $\longrightarrow$ ซ้อนหลายสิบชั้นได้

ขั้นตอนโดยละเอียด • ตัวอย่างการคำนวณ 4.19 (Add & Norm)

ข้อมูลมาจากไหน

- x, Attention และ FFN เป็นค่าสมมติในภาพที่ 4.23
- เลือกให้ผลหลังทางลัดเป็นเลขกลม (1, 1, −1) และ (1, 0, −1)
- $\gamma = 1$, $\beta = 0$ = ค่าตั้งต้นจริง

ขั้น	มิติ 1	มิติ 2	มิติ 3	สถิติ
r1 = x + Att	1.0	1.0	−1.0	$\mu = 0.3333$
r1 − μ	0.6667	0.6667	−1.3333	$\sigma = 0.9428$
z = LN(r1)	0.7071	0.7071	−1.4142	
r2 = z + FFN	1.0	0.0	−1.0	$\mu = 0$, $\sigma = 0.8165$
h = LN(r2)	1.2247	0.0000	−1.2247	เฉลี่ย 0, แปรปรวน 1

h = (1.2247, 0.0000, −1.2247)

ตัวอย่างการคำนวณ 4.20 : FFN ขยาย 2 ⟶ 4 แล้วย่อกลับ

โจทย์ $z = (1, -1)$ • ค่าเอนเอียงเป็นศูนย์ • จงหา $FFN(z) = W_2\ ReLU(W_1\ z)$

W1 (4×2)

1.0	0.0
0.0	1.0
1.0	1.0
1.0	−1.0

W2 (2×4)

0.2	0.0	0.0	0.4
0.5	0.0	0.0	−0.5

1 ขยายมิติ W1 z

- แถว 1: 1(1) + 0(−1) = 1
- แถว 2: 0(1) + 1(−1) = −1
- แถว 3: 1 − 1 = 0
- แถว 4: 1 + 1 = 2 ⟶ (1, −1, 0, 2)

2 ReLU

- ตัดค่าลบ:
- (1, −1, 0, 2) ⟶ (1, 0, 0, 2)

3 ย่อกลับ W2 •

- 0.2(1) + 0.4(2) = 1.0
- 0.5(1) − 0.5(2) = −0.5 ⟶ (1.0, −0.5)

ตีความ ขยายชั่วคราวให้ ReLU “เลือก” คุณลักษณะ แล้วพับกลับขนาดเดิม • FFN ทำกับโทเคนทีละตัว

ขั้นตอนโดยละเอียด • ตัวอย่างการคำนวณ 4.20 (FFN 2 → 4 → 2)

ข้อมูลมาจากไหน

- z, W1, W2 เป็นค่าสมมติในภาพที่ 4.24
- 2 → 4 → 2 ย่อจาก 768 → 3,072 → 768 (dff = 4d)
- ค่าเอนเอียงเป็นศูนย์

แถวของ W1	คูณกับ (1, −1)	ก่อน ReLU	หลัง ReLU
(1, 0)	1•1 + 0•(−1)	1	1
(0, 1)	0•1 + 1•(−1)	−1	0
(1, 1)	1•1 + 1•(−1)	0	0
(1, −1)	1•1 + (−1)•(−1)	2	2

W2 × (1, 0, 0, 2): (0.2 + 0.8, 0.5 − 1.0) = (1.0, −0.5)

4.4.6 Pooling และการฉาย : จากหลายเวกเตอร์สู่เวกเตอร์เดียว

$$\mathbf{v}_{mean} = \frac{1}{n}\sum_{t=1}^{n}\mathbf{h}_t, \qquad \mathbf{v}_{\max}[j] = \max_t h_t[j] \tag{4.28}$$

$$\mathbf{v}_{cls} = \mathbf{h}_{[CLS]}, \qquad \mathbf{v}_w = \sum_{t=1}^{n} a_t \mathbf{h}_t \tag{4.29}$$

$$\mathbf{q} = \mathbf{W}\mathbf{v} + \mathbf{b}, \qquad \hat{\mathbf{v}} = \frac{\mathbf{q}}{\|\mathbf{q}\|} \tag{4.30}$$

ระบบผู้แนะนำต้องการเวกเตอร์ **หนึ่งเส้นต่อหนึ่งรายการ** แต่ Encoder ให้ **หนึ่งเส้นต่อหนึ่งโทเคน**

เลือกวิธี Pooling แบบใด

- **[CLS]** เมื่อแบบจำลองถูกฝึกให้ [CLS] สรุปทั้งลำดับ (ViT, BERT จำแนก)
- **Mean** ค่าเริ่มต้นที่ดีสำหรับเวกเตอร์ประโยค (Sentence-BERT ดีกว่า [CLS])
- **Max** เมื่อสนใจว่ามีคุณลักษณะเด่นปรากฏหรือไม่ (Text CNN)
- **Weighted** เมื่อรู้ว่าบางโทเคนสำคัญกว่า เช่น ถ่วงด้วย IDF

ตัวอย่างการคำนวณ 4.21 : Pooling และการฉาย (Toy Story 5)

เวกเตอร์โทเคน 4 มิติ (ภาพที่ 4.26)

โทเคน	d1	d2	d3	d4
toys	0.1	0.4	0.2	0.3
tablet	0.5	0.2	0.1	0.0
playtime	0.3	0.8	0.6	0.1
friendship	0.0	0.1	0.3	0.7
Mean	0.225	0.375	0.300	0.275
Max	0.5	0.8	0.6	0.7

Mean ให้ทุกโทเคนมีส่วนเท่ากัน • Max เก็บสัญญาณเด่นสุด • ฉาย + L^2 ⟶ วัดโคไซน์ได้ทันที

1 Mean Pooling

- d1: (0.1 + 0.5 + 0.3 + 0.0)/4 = 0.225 • d2: (0.4 + 0.2 + 0.8 + 0.1)/4 = 0.375

2 Max Pooling: เลือกค่ามากสุดทีละมิติ = (0.5, 0.8, 0.6, 0.7)

3 ฉาย $W = [1\ 0\ 1\ 0\ ;\ 0\ 1\ 0\ 1]$, $b = (0.05, -0.05)$

- $q_1 = 0.225 + 0.300 + 0.05 = 0.575$ • $q_2 = 0.375 + 0.275 - 0.05 = 0.600$

4 ปรับความยาว: $\|q\| = \sqrt{(0.575^2 + 0.600^2)} = 0.8310$ ⟶ $v = (0.6919, 0.7220)$

ที่มาของข้อมูล • ตัวอย่างการคำนวณ 4.21 (Pooling ของ Toy Story 5)

ข้อมูลมาจากไหน

- toys (3 ครั้ง), tablet, playtime, friendship พบจริงในเรื่องย่อ i1 (ตารางที่ 4.6)
- เวกเตอร์ h1–h4, W, b เป็นค่าสมมติ (ภาพที่ 4.26)
- $q = W\ v_mean + b \rightarrow \|q\| = 0.8310$

โทเคน	ม.1	ม.2	ม.3	ม.4
toys	0.1	0.4	0.2	0.3
tablet	0.5	0.2	0.1	0.0
playtime	0.3	0.8	0.6	0.1
friendship	0.0	0.1	0.3	0.7
Mean	0.225	0.375	0.300	0.275
Max	0.5	0.8	0.6	0.7

$v = (0.575, 0.600)/0.8310 = (0.6919, 0.7220)$

4.4.7 การประยุกต์ Transformer กับสี่สื่อ

$$N_{img} = \frac{H}{P} \times \frac{W}{P}, \quad N_{aud} = \left(\left\lfloor \frac{F-P}{s} \right\rfloor + 1\right)\left(\left\lfloor \frac{T-P}{s} \right\rfloor + 1\right), \quad N_{vid} = \frac{T_v}{t} \times \frac{H}{P} \times \frac{W}{P} \tag{4.31}$$

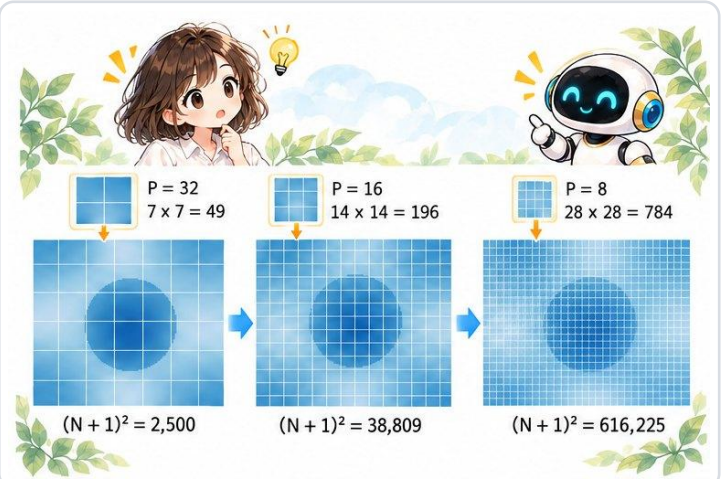
$$C_{attn} = (N+1)^2$$

สื่อ	แบ่งเป็นโทเคน	แบบจำลองที่นิยม	เวกเตอร์ของรายการ	ไลบรารี Python
ข้อความ	คำย่อย (WordPiece, BPE)	BERT, Sentence-BERT, MiniLM, E5	Mean หรือ [CLS]	transformers, sentence-transformers
ภาพ	แพตช์ P×P ฉายเชิงเส้น	ViT, Swin, DINOv2, CLIP	[CLS]	transformers, timm, open_clip
เสียง	แพตช์ของ Log-Mel	AST, wav2vec 2.0, HuBERT, CLAP	[CLS] หรือ Mean	transformers (ASTModel)
วิดีโอ	Tubelet เวลา × พื้นที่	ViViT, TimeSformer, VideoMAE	Mean หรือ [CLS]	transformers (VideoMAE), decord

บล็อก Transformer เหมือนกันทุกสื่อ ต่างกันที่ **วิธีแบ่งโทเคน** ซึ่งกำหนดจำนวนโทเคน N และต้นทุนของ Self-Attention

ตัวอย่างการคำนวณ 4.22 : จำนวนโทเคนและต้นทุน Attention

สื่อ	การคำนวณจำนวนโทเคน N	คู่ Attention $(N+1)^2$
ข้อความ	N = 60 โทเคน (+[CLS] = 61)	61^2 = 3,721
ภาพ (P = 16)	$(224/16)^2 = 14^2$ = 196 แพตช์ • 1 แพตช์ = 16×16×3 = 768 ค่า	197^2 = 38,809
เสียง (P = 16, s = 10)	ความถี่ (128−16)/10 ปัดลง + 1 = 12 • เวลา (1,024−16)/10 ปัดลง + 1 = 101 ⟶ 12×101 = 1,212	$1{,}213^2$ = 1,471,369
วิดีโอ Joint	Tubelet 2×16×16: (16/2) × 14 × 14 = 1,568	$1{,}568^2$ = 2,458,624
วิดีโอ Divided	เทียบ 196 ในเฟรม + 8 ข้ามเวลา	1,568 × 204 = 319,872



ตีความ ต้นทุนโตเป็นกำลังสองของ N • วิดีโอแบบทุกคู่แพงกว่าข้อความ **661 เท่า** • Divided Attention ลดได้ ≈ 7.7 เท่า • P เล็กลงครึ่ง ⟶ คู่เพิ่ม ≈ 16 เท่า

ขั้นตอนโดยละเอียด • ตัวอย่างการคำนวณ 4.22 (จำนวนโทเคน)

ข้อมูลมาจากไหน

- รูปแบบข้อมูลนำเข้า: ตารางที่ 4.8
- แพตช์ภาพ 16 (ViT-B/16) • แพตช์เสียง 16 ก้าว 10 (AST) • Tubelet 2×16×16 (ViViT)

สื่อ	จำนวนโทเคน	คู่ Attention
ข้อความ	60 (+1)	3,721
ภาพ	14 × 14 = 196 (+1)	38,809
เสียง	12 × 101 = 1,212 (+1)	1,471,369
วิดีโอ Joint	8 × 196 = 1,568	2,458,624
วิดีโอ Divided	1,568 × (196 + 8)	319,872

วิดีโอ Joint แพงกว่าข้อความ ≈ 661 เท่า • Divided ลดได้ ≈ 7.7 เท่า

4.4.8 เวกเตอร์ข้อความ 12 มิติจากเทคนิค Transformer (ตารางที่ 4.15)

รหัส	tech	mind	spac	rom	crim	act	fam	mus	fant	toys	soc	amb
i1	.2145	.1287	.0429	·	.0858	.1287	.3377	·	.0858	.8889	·	·
i2	·	.9879	·	·	·	.0335	.1488	·	·	·	·	.0288
i3	.3544	.1688	·	·	·	.1181	.7852	·	.4642	·	·	·
i4	·	.1940	·	.8027	.3051	.0555	.4528	·	·	·	.1294	·
i5	·	·	·	.2809	·	.0551	.4312	.1926	.5777	·	.1926	.5694
i6	·	.0553	·	.6780	·	·	.7314	·	·	·	.0475	·
i7	·	.0330	·	·	.9065	.4121	.0769	.0384	·	·	·	·
i8	·	·	.0898	·	.0449	.9755	.0772	·	·	·	.1797	·
i9	·	.1355	·	·	.1734	.0758	·	·	.0379	·	.9473	.2167
i10	·	.2024	.4424	·	·	.5770	·	·	.1574	·	.6294	.0975
i11	.5949	.2511	.4850	.0511	.0595	.4591	·	·	.0595	·	.3569	·
i12	.2449	.2627	.8892	·	·	.0911	.2524	·	·	·	.0911	·

ค่าจำลองของ Text Transformer Encoder ปรับความยาวเป็น 1 แล้ว · แถบเหลือง = เรื่องที่ u1 ชอบ (ใช้บ่อยในหัวข้อ 4.5)

เวกเตอร์ภาพ เสียง วิดีโอ 3 มิติจากเทคนิค Transformer (ตารางที่ 4.16)

รหัส	I1	I2	I3	A1	A2	A3	V1	V2	V3
i1	0.9	0.6	0.9	0.5	0.7	0.1	0.6	0.5	1.0
i2	0.8	0.8	0.9	0.5	1.0	0.2	0.5	0.4	1.0
i3	0.7	0.5	0.8	0.3	0.8	0.4	0.4	0.3	1.0
i4	0.5	0.7	0.2	0.3	0.8	0.3	0.4	0.3	0.0
i5	0.7	0.6	0.5	0.4	1.0	0.1	0.3	0.4	0.0
i6	0.6	0.8	0.3	0.2	0.8	0.1	0.2	0.2	0.0
i7	0.2	0.3	0.1	0.9	0.2	0.8	0.9	0.9	0.0
i8	0.6	0.9	0.3	1.0	0.3	0.7	1.0	1.0	0.0
i9	0.4	0.3	0.1	0.3	0.5	0.6	0.3	0.4	0.0
i10	0.5	0.8	0.1	0.8	0.3	0.8	0.7	0.6	0.0
i11	0.3	0.2	0.1	0.6	0.3	0.7	0.6	0.5	0.0
i12	0.3	0.3	0.1	0.5	0.6	0.7	0.5	0.4	0.0
μ คลัง	0.5417	0.5667	0.3667	0.5250	0.6083	0.4583	0.5333	0.4917	0.2500

อ่านตาราง

- ค่าเป็นบวกทั้งหมด ⟶ โคไซน์ระหว่างสองเรื่องใด ๆ จะสูงเกือบเสมอ
- แถวล่าง μ **คลัง** = ค่าเฉลี่ย 12 เรื่อง ใช้ลบออกก่อน (สมการ 4.33)
- ตัวอย่าง: μ^{I}_{1} = 6.5/12 = 0.5417

ตัวอย่างการคำนวณ 4.23 : บริบทเปลี่ยนความคล้าย i2–i5

โจทย์ เทียบโคไซน์ข้อความของ i2 Inside Out 2 กับ i5 Wicked จาก CNN (ตาราง 4.12) และ Transformer (ตาราง 4.15)

1 เทคนิค CNN (Static)

มิติ	i2	i5	ผลคูณ
mind	0.9641	0.1189	0.1146
romance	0.0266	0.3029	0.0081
action	0.0391	0.1189	0.0046
family	0.2001	0.4649	0.0930
ambition	0.1679	0.4356	0.0731
cos			0.2935

2 เทคนิค Transformer (ตามบริบท)

มิติ	i2	i5	ผลคูณ
mind	0.9879	0	0
action	0.0335	0.0551	0.0018
family	0.1488	0.4312	0.0642
ambition	0.0288	0.5694	0.0164
cos			0.0824

ตีความ Static: dream ตัวเดียวให้ทั้ง mind และ ambition ⟶ คล้ายเกินจริง • Transformer: dream ใน i2 ⟶ mind, ใน i5 ⟶ ambition

ที่มาของข้อมูล • ตัวอย่างการคำนวณ 4.23 (dream ใน i2 และ i5)

ข้อมูลมาจากไหน

- i2, i5 ไม่มีแนวร่วมกัน (ตารางที่ 4.4) แต่ u6 ชอบทั้งคู่ (ตารางที่ 4.5)
- dream: i2 ×4 (night, memory, frightening) • i5 ×2 (chase a dream, better land)
- Static: dream = mind 0.5 + ambition 0.5 ทุกที่
- Transformer: i2 → mind • i5 → ambition

มิติ	CNN i2×i5	TF i2×i5
mind	0.1146	0.0000
romance	0.0081	0.0000
action	0.0046	0.0018
family	0.0930	0.0642
ambition	0.0731	0.0164
รวม	0.2935	0.0824

cos ลดจาก 0.2935 เป็น 0.0824 เมื่ออ่านบริบท

เปรียบเทียบ CNN กับ Transformer

ประเด็น	CNN	Transformer
หน่วยประมวลผล	ตัวกรองเลื่อนบนหน้าต่างเฉพาะที่	Self-Attention ระหว่างทุกคู่โทเคน
ระยะมองเห็น	ขยายทีละชั้น r = 1 + L(k−1)	ทั้งลำดับตั้งแต่ชั้นแรก
ลำดับและตำแหน่ง	รู้โดยธรรมชาติ	ต้องเพิ่มการฝังตำแหน่ง
ต้นทุนต่อชั้น	เชิงเส้นตามขนาดข้อมูล	กำลังสองของจำนวนโทเคน
ข้อมูลที่ต้องใช้ฝึก	น้อยถึงปานกลาง	มาก (ฝึกล่วงหน้าบนข้อมูลใหญ่)
ความหมายตามบริบท	จำกัด (คำเดียวความหมายเดียว)	ดี (ขึ้นกับคำรอบข้าง)
ข้ามสื่อ	แยกแบบจำลองตามสื่อ	มีแบบจำลองร่วม เช่น CLIP, CLAP

cos(CNN, TF) ของเรื่องเดียวกัน

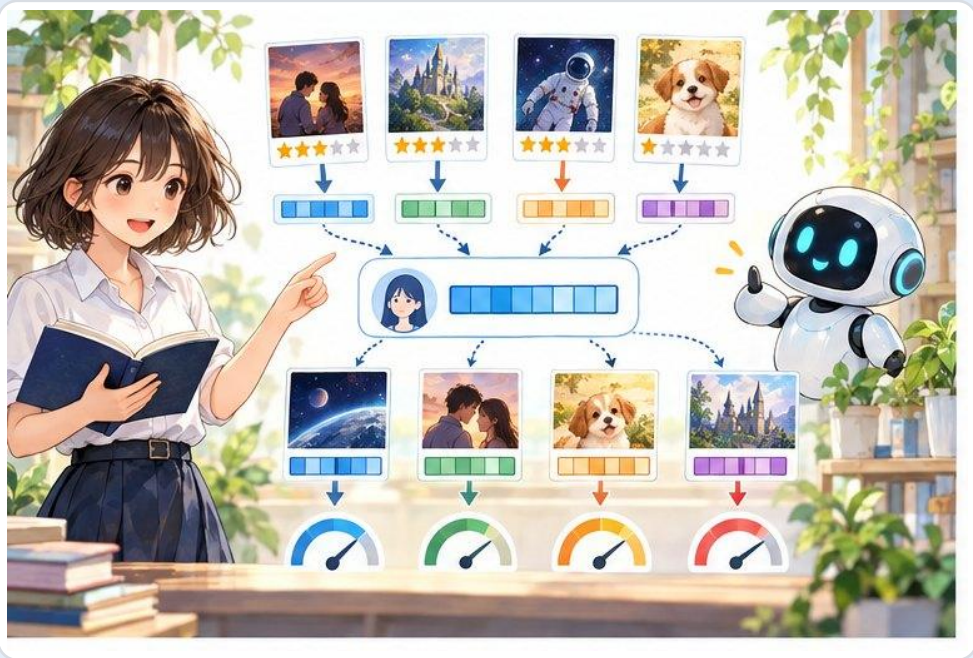
สื่อ	cos เฉลี่ย	ต่ำสุด
ข้อความ	0.9960	0.9799 (i5)
ภาพ	0.9926	0.9895 (i10)
เสียง	0.9745	0.9561 (i3)
วิดีโอ	0.9779	0.9541 (i5)

เลือกใช้

- CNN ข้อมูลน้อย ต้องเร็ว ลวดลายเฉพาะที่
- Transformer ความหมายขึ้นกับบริบทกว้าง / ต้องเชื่อมหลายสื่อ

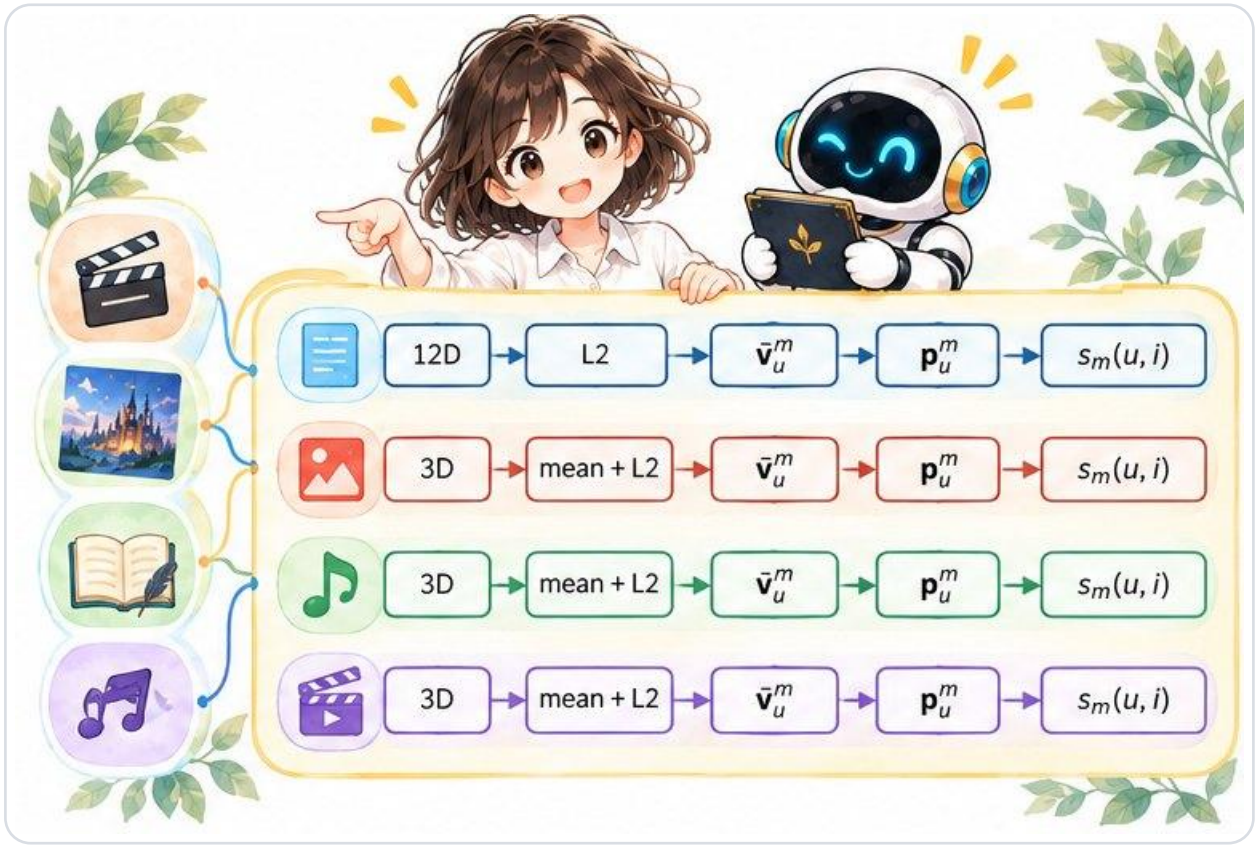
4.5

การสร้างโปรไฟล์ผู้ใช้ (User Profile)



Mean Centering • ค่าเฉลี่ย • ถ่วงคะแนน • Rocchio • ลดตามเวลา • หลายจุดสนใจ • Attention • Caser/SASRec

ขั้นตอนเดียวกันสำหรับทุกสื่อ : เวกเตอร์รายการ ⟶ โปรไฟล์ ⟶ คะแนน



ภาพ เสียง วิดีโอ ต้อง “เตรียมเวกเตอร์” ก่อน ส่วนข้อความใช้ตามเดิม

$$s_m(u,i) = \mathbf{p}_u^m \cdot \mathbf{v}_i^m = \cos(\mathbf{p}_u^m, \mathbf{v}_i^m) \qquad (4.35)$$

สิ่งที่ต้องรู้

- p_u^m โปรไฟล์ของผู้ใช้ u ในสื่อ m (ความยาว 1)
- $s_m(u,i)$ คะแนนรายสื่อ อยู่ระหว่าง −1 ถึง 1
- ทุกวิธีในหัวข้อนี้ **ต่างกันเพียงวิธีสร้าง** p_u^m
- ตัวอย่างใช้เทคนิค Transformer เป็นหลัก แล้วเทียบ CNN ใน 4.5.8

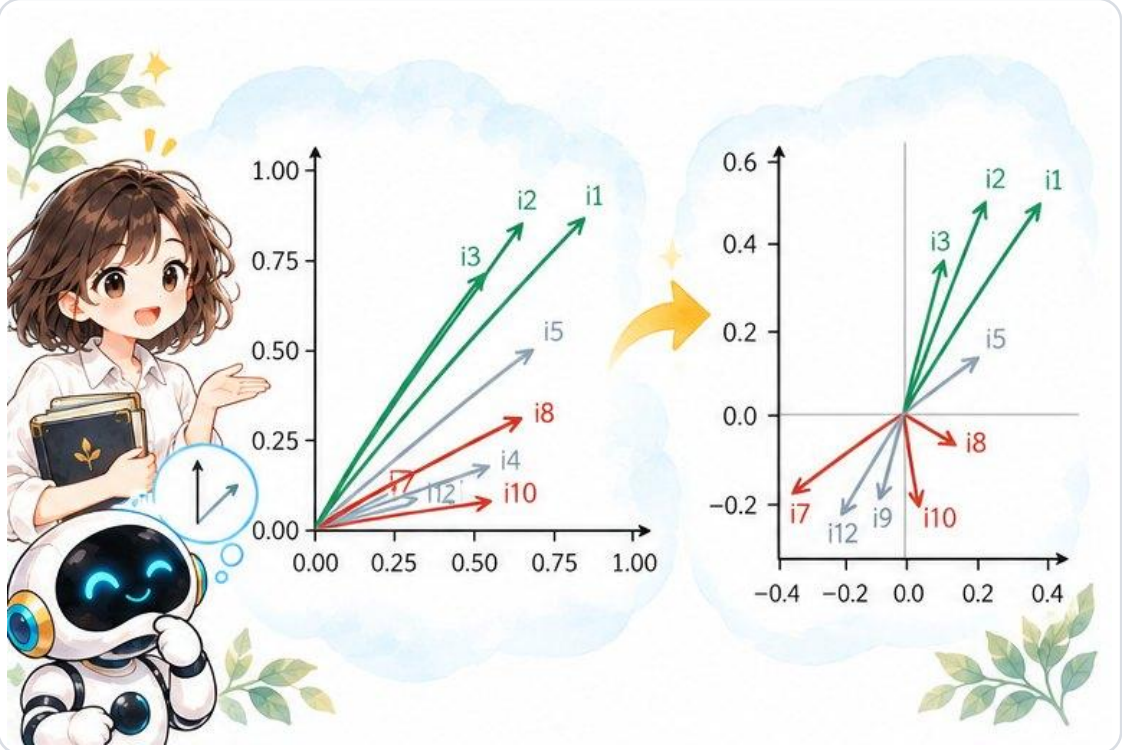
4.5.1 เตรียมเวกเตอร์ : ลบค่าเฉลี่ยของคลัง แล้วปรับความยาว

$$\boldsymbol{\mu}^m = \frac{1}{|\mathcal{I}|} \sum_{i \in \mathcal{I}} \mathbf{x}_i^m, \qquad \tilde{\mathbf{x}}_i^m = \mathbf{x}_i^m - \boldsymbol{\mu}^m \tag{4.33}$$

$$\mathbf{v}_i^m = \frac{\tilde{\mathbf{x}}_i^m}{\|\tilde{\mathbf{x}}_i^m\|} \tag{4.34}$$

ทำไมต้องลบค่าเฉลี่ย

- ค่าในตาราง 4.16 เป็นบวกทั้งหมด ⟶ ทุกเวกเตอร์อยู่มุมบวกเดียวกัน โคไซน์สูงเกือบเสมอ
- หลังลบ μ เวกเตอร์กระจายรอบศูนย์กลาง แยกแอนิเมชันออกจากแอ็กชันได้ชัด
- ข้อความ (ปรับความยาวแล้ว มีศูนย์มาก) **ไม่ต้องลบ**



เปลี่ยนคำถามจาก “มีค่ามากเหมือนกันไหม” เป็น “**ต่างจากเรื่องทั่วไปไปทางเดียวกันไหม**”

ตัวอย่างการคำนวณ 4.24 : เตรียมเวกเตอร์ภาพ i1 และ i7

โจทย์ เตรียมเวกเตอร์ภาพ (Transformer) ของ i1 Toy Story 5 = (0.9, 0.6, 0.9) และ i7 Ballerina = (0.2, 0.3, 0.1) แล้วเทียบโคไซน์ก่อน–หลัง

1 โคไซน์ก่อนเตรียม

- ผลคูณจุด = 0.18 + 0.18 + 0.09 = 0.45
- $\|i1\|$ = √1.98 = 1.4071 · $\|i7\|$ = √0.14 = 0.3742
- cos = 0.45/(1.4071 × 0.3742) = **0.8547** (สูงเกินจริง)

2 ค่าเฉลี่ยของคลัง 12 เรื่อง

- มิติ 1: (0.9 + 0.8 + ••• + 0.3)/12 = 6.5/12 = 0.5417
- μ = (0.5417, 0.5667, 0.3667)

3 ลบค่าเฉลี่ยแล้วปรับความยาว

- $\tilde{x}_{i1}$ = (0.3583, 0.0333, 0.5333) ยาว 0.6434 ⟶ v_{i1} = (0.5569, 0.0518, 0.8289)
- $\tilde{x}_{i7}$ = (−0.3417, −0.2667, −0.2667) ยาว 0.5089 ⟶ v_{i7} = (−0.6714, −0.5240, −0.5240)

4 โคไซน์หลังเตรียม

- $v_{i1} \cdot v_{i7}$ = −0.8355
- โปสเตอร์ต่างกันจริง

ขั้นตอนโดยละเอียด • ตัวอย่างการคำนวณ 4.24 (ลบค่าเฉลี่ยของคลัง)

ข้อมูลมาจากไหน

- เวกเตอร์ภาพ 12 แถว: ตารางที่ 4.16
- μ ต้องเฉลี่ยจากทุกเรื่องในคลัง
- i1 โปสเตอร์แอนิเมชันสว่าง • i7 แอ็กชันโทนมืด (ตารางที่ 4.4)

เรื่อง	ผลรวม/12 หรือ $x - \mu$	ความยาว	v
μ	(6.5, 6.8, 4.4)/12 = (0.5417, 0.5667, 0.3667)		
i1	(0.3583, 0.0333, 0.5333)	0.6434	(0.5569, 0.0518, 0.8289)
i7	(−0.3417, −0.2667, −0.2667)	0.5089	(−0.6714, −0.5240, −0.5240)

cos ก่อนเตรียม 0.8547 → หลังเตรียม −0.8355

4.5.2 โปรไฟล์ค่าเฉลี่ย และตัวอย่าง 4.25 (1/2)

$$\mathbf{p}_u^m = \frac{\bar{\mathbf{p}}}{\|\bar{\mathbf{p}}\|}, \qquad \bar{\mathbf{p}} = \frac{1}{|L_u|} \sum_{i \in L_u} \mathbf{v}_i^m \qquad (4.36)$$

L_u เรื่องที่ผู้ใช้ชอบ (4–5) • แนวคิด “ผู้ใช้น่าจะชอบเรื่องที่อยู่ใกล้จุดศูนย์กลางของเรื่องที่เคยชอบ”

โจทย์ u1 ชอบ i1, i2, i3 • สร้างโปรไฟล์ข้อความ (Transformer)

1 เฉลี่ยทีละมิติ (ตัวอย่าง)

- technology: (0.2145 + 0 + 0.3544)/3 = 0.1896
- mind: (0.1287 + 0.9879 + 0.1688)/3 = 0.4285
- family: 0.4239 • toys: 0.8889/3 = 0.2963

มิติ	i1	i2	i3	$\bar{p}$	p (÷0.7283)
technology	.2145	0	.3544	.1896	.2604
mind	.1287	.9879	.1688	.4285	.5883
space	.0429	0	0	.0143	.0196
romance	0	0	0	0	0
crime	.0858	0	0	.0286	.0393
action	.1287	.0335	.1181	.0934	.1283
family	.3377	.1488	.7852	.4239	.5820
music	0	0	0	0	0
fantasy	.0858	0	.4642	.1833	.2517
toys	.8889	0	0	.2963	.4068
society	0	0	0	0	0
ambition	0	.0288	0	.0096	.0132

ขั้นที่ 2: $\|\bar{p}\| = 0.7283 \longrightarrow$ หารทุกมิติ

ตัวอย่างการคำนวณ 4.25 : คะแนน i5 และ Top-3 (2/2)

3 คะแนนของ i5 Wicked (สมการ 4.35)

มิติ	p_u1	v_i5	ผลคูณ
action	0.1283	0.0551	0.0071
family	0.5820	0.4312	0.2510
fantasy	0.2517	0.5777	0.1454
ambition	0.0132	0.5694	0.0075
p · v_i5			≈ 0.4109

4 จัดอันดับผู้สมัคร 7 เรื่อง

อันดับ	รายการ	คะแนน
1	i5 Wicked	0.4109
2	i12 Project Hail Mary	0.3943
3	i11 Mickey 17	0.3883
4	i10 Dune: Part Two	0.2427
5	i8 Furiosa	0.1736
6	i7 Ballerina	0.1526
7	i9 Conclave	0.1086



ตีความ โปรไฟล์เด่นที่ mind, family, toys • i5 ได้อันดับ 1 จาก family + fantasy แต่ i12 (0.3943) กับ i11 (0.3883) ใกล้กันมาก ⟶ ข้อความอย่างเดียวแยกไม่ชัด

ที่มาของข้อมูล • ตัวอย่างการคำนวณ 4.25 (โปรไฟล์ค่าเฉลี่ยของ u1)

ข้อมูลมาจากไหน

- คะแนน u1 (ตารางที่ 4.5): ชอบ i1 = 5, i2 = 5, i3 = 4 • ไม่ชอบ i4 = 1, i6 = 1
- L_u1 = {i1, i2, i3} (คะแนน 4–5)
- ผู้สมัคร = 12 − 5 เรื่องที่ให้คะแนนแล้ว = i5, i7–i12
- เวกเตอร์ข้อความ: ตารางที่ 4.15 (จากเรื่องย่อ ตารางที่ 4.6)

มิติ	i1	i2	i3	$\bar{p}$	p_u1
tech	0.2145	0.0000	0.3544	0.1896	0.2604
mind	0.1287	0.9879	0.1688	0.4285	0.5883
space	0.0429	0.0000	0.0000	0.0143	0.0196
crime	0.0858	0.0000	0.0000	0.0286	0.0393
action	0.1287	0.0335	0.1181	0.0934	0.1283
family	0.3377	0.1488	0.7852	0.4239	0.5820
fantasy	0.0858	0.0000	0.4642	0.1833	0.2517
toys	0.8889	0.0000	0.0000	0.2963	0.4068
ambition	0.0000	0.0288	0.0000	0.0096	0.0132

$\|\bar{p}\| = 0.7283 \rightarrow p_u1 = \bar{p} / 0.7283$

ขั้นตอนโดยละเอียด • ตัวอย่างการคำนวณ 4.25 (คะแนนเจ็ดเรื่อง)

วิธีคิดคะแนน

- s(u1, i) = p_u1 • v_i (ทั้งสองยาว 1 → เท่ากับโคไซน์)
- i5: family 0.2510 + fantasy 0.1454 + อื่น ๆ = 0.4109
- ทำแบบเดียวกันกับผู้สมัครทุกเรื่อง

อันดับ	รายการ	s	มิติที่ช่วยมาก
1	i5	0.4109	family 0.2510, fantasy 0.1454
2	i12	0.3943	mind 0.1545, family 0.1469
3	i11	0.3883	tech 0.1549, mind 0.1477
4	i10	0.2427	mind 0.1191, action 0.0740
5	i8	0.1736	action 0.1251, family 0.0449
6	i7	0.1526	action 0.0529, family 0.0448
7	i9	0.1086	mind 0.0797, action 0.0097

Top-3: i5 (0.4109), i12 (0.3943), i11 (0.3883)

คะแนนรายสื่อของ u1 จากโปรไฟล์ค่าเฉลี่ย (ตารางที่ 4.19)

	เทคนิค Transformer				เทคนิค CNN			
รหัส	T	I	A	V	T	I	A	V
i5	0.4109	0.9180	0.9929	−0.6417	0.5504	0.9145	0.9954	−0.8430
i7	0.1526	−0.8154	−0.9609	−0.5177	0.1555	−0.8020	−0.9157	−0.6405
i8	0.1736	−0.0053	−0.8548	−0.4474	0.1871	0.0829	−0.7904	−0.5635
i9	0.1086	−0.8084	−0.3154	−0.6417	0.1200	−0.7527	−0.0276	−0.8430
i10	0.2427	−0.6555	−0.9799	−0.8390	0.2850	−0.5229	−0.9549	−0.9369
i11	0.3883	−0.7392	−0.9816	−0.9727	0.4170	−0.7019	−0.9844	−0.9925
i12	0.3943	−0.8268	−0.6487	−0.8781	0.4786	−0.7960	−0.6943	−0.9305

แต่ละสื่อเห็นต่างกัน

ภาพ/เสียงชอบ i5 • วิดีโอชอบ i8

มาตราคะแนนต่างกัน

T: 0.1–0.4 • I/A: −0.9 ถึง 1

สองเทคนิคอันดับใกล้กัน

T ของ i5: 0.4109 vs 0.5504

4.5.3 ถ่วงด้วยคะแนน และตัวอย่างการคำนวณ 4.26

$$\mathbf{p}_u^m \propto \sum_{i \in R_u} (r_{ui} - \bar{r}_u)\, \mathbf{v}_i^m, \qquad \bar{r}_u = \frac{1}{|R_u|} \sum_{i \in R_u} r_{ui} \tag{4.37}$$

1 ค่าเฉลี่ยและน้ำหนัก

- $\bar{r}_{u1}$ = (5+5+4+1+1)/5 = 3.2
- i1 = i2: 5 − 3.2 = **1.8**
- i3: **0.8** • i4 = i6: 1 − 3.2 = **−2.2**

2 รวมเวกเตอร์ถ่วงน้ำหนัก (มิติที่เปลี่ยนมาก)

- romance: −2.2(0.8027) − 2.2(0.6780) = **−3.2577**
- family: 1.8(0.3377) + 1.8(0.1488) + 0.8(0.7852) − 2.2(0.4528) − 2.2(0.7314) = **−1.1015**
- mind: 1.8(0.1287) + 1.8(0.9879) + 0.8(0.1688) − 2.2(0.1940) − 2.2(0.0553) = 1.5964

3

อันดับ	1	2	3	4	5	6	7
รายการ	i11	i12	i10	i8	i9	i7	i5
คะแนน	0.1533	0.0851	0.0828	0.0203	−0.0448	−0.0919	−0.2622

ตีความ เรื่องไม่ชอบมี family สูง ⟶ กด family ติดลบ i5 ตกไปอันดับ 7

ขั้นตอนโดยละเอียด • ตัวอย่างการคำนวณ 4.26 (ถ่วงด้วยคะแนน)

ข้อมูลมาจากไหน

- ใช้คะแนนครบ 5 เรื่องของ u1 (ตารางที่ 4.5)
- $\bar{r}$ = 16/5 = 3.2 → น้ำหนัก (1.8, 1.8, 0.8, −2.2, −2.2)
- i4, i6 เป็น Drama, Romance และเรื่องย่อมีคำ love → romance, family สูง
- $\|p\|$ = 4.2610

มิติ	ผลรวม p	p/‖p‖
tech	0.6696	0.1572
mind	1.5964	0.3747
space	0.0772	0.0181
romance	−3.2577	−0.7645
crime	−0.5168	−0.1213
action	0.2643	0.0620
family	−1.1015	−0.2585
fantasy	0.5258	0.1234
toys	1.6001	0.3755
society	−0.3892	−0.0913
ambition	0.0518	0.0122

Top-3: i11 (0.1533), i12 (0.0851), i10 (0.0828) • i5 = −0.2622

4.5.4 Rocchio และตัวอย่างการคำนวณ 4.27 ($\beta = 0.5$)

$$\mathbf{p}_u^m \propto \alpha \frac{1}{|L_u|} \sum_{i \in L_u} \mathbf{v}_i^m - \beta \frac{1}{|D_u|} \sum_{j \in D_u} \mathbf{v}_j^m \qquad (4.38)$$

D_u เรื่องไม่ชอบ (1–2) · $\alpha = 1$ · $\beta = 0$ คือโปรไฟล์ค่าเฉลี่ย

1 ค่าเฉลี่ยเรื่องไม่ชอบ

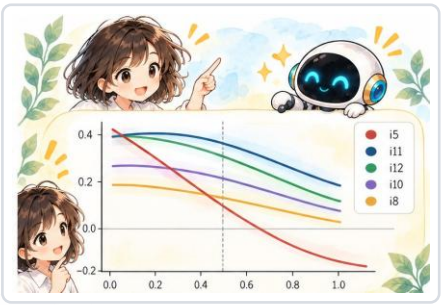
- romance: (0.8027+0.6780)/2 = 0.7404
- family: (0.4528+0.7314)/2 = 0.5921

2 ลบด้วย $\beta = 0.5$

- family: 0.4239 − 0.5(0.5921) = 0.1278
- romance: 0 − 0.5(0.7404) = **−0.3702**

3 ปรับความยาว + จัดอันดับ

- i5: 0.4109 ⟶ 0.0865
- Top-3: **i11 (0.3282), i12 (0.2827), i10 (0.1901)**



β	0.00	0.25	0.50	0.75	1.00
HR@3 (T)	0.70	0.70	0.65	0.60	0.60
MRR (T)	0.6044	0.6794	0.6453	0.6440	0.6153
MRR (4 สื่อ)	0.7635	0.7283	0.7300	0.6325	0.6110

ตีความ ผลักน้อยกว่าถ่วงคะแนน แต่ i5 หลุด Top-3 · เลือก β บนชุดตรวจสอบ

ขั้นตอนโดยละเอียด • ตัวอย่างการคำนวณ 4.27 (Rocchio $\beta = 0.5$)

ข้อมูลมาจากไหน

- ชอบ {i1, i2, i3} → $\bar{p}$ จากตัวอย่าง 4.25
- ไม่ชอบ {i4, i6} (คะแนน 1, ตารางที่ 4.5)
- $\alpha = 1$, $\beta = 0.5$
- family: $0.4239 - 0.5 \times 0.5921 = 0.1278$

มิติ	$\bar{p}$	ไม่ชอบ	$\bar{p} - 0.5 \cdot$ไม่ชอบ	L^2
tech	0.1896	0.0000	0.1896	0.2809
mind	0.4285	0.1247	0.3661	0.5424
space	0.0143	0.0000	0.0143	0.0212
romance	0.0000	0.7404	−0.3702	−0.5484
crime	0.0286	0.1526	−0.0477	−0.0706
action	0.0934	0.0278	0.0796	0.1179
family	0.4239	0.5921	0.1278	0.1894
fantasy	0.1833	0.0000	0.1833	0.2716
toys	0.2963	0.0000	0.2963	0.4390
society	0.0000	0.0885	−0.0442	−0.0655
ambition	0.0096	0.0000	0.0096	0.0142

Top-3: i11 (0.3282), i12 (0.2827), i10 (0.1901) • i5 = 0.0865

4.5.5 โปรไฟล์ที่ลดน้ำหนักตามเวลา

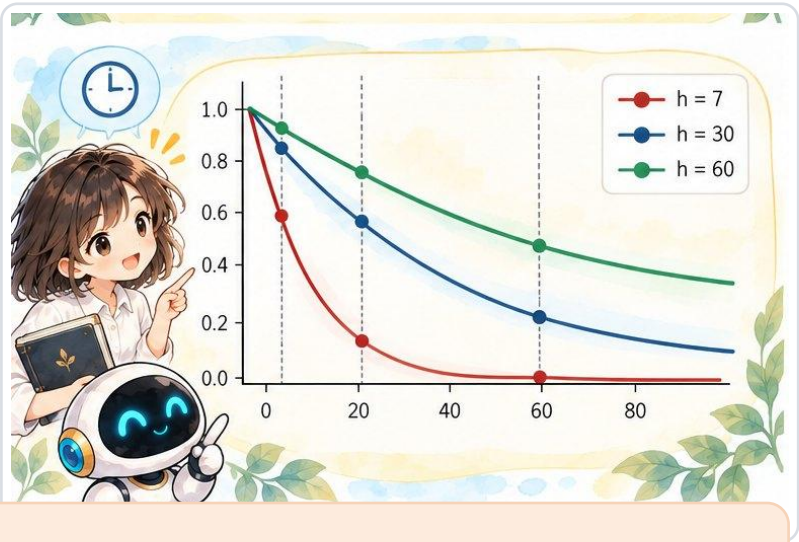
$$w_i = 2^{-\Delta t_i / h} \qquad (4.39)$$

$$\mathbf{p}_u^m \propto \frac{\sum_{i \in L_u} w_i \, \mathbf{v}_i^m}{\sum_{i \in L_u} w_i} \qquad (4.40)$$

Δt อายุคะแนน (วัน) • h ครึ่งชีวิต

รหัส	ชื่อเรื่อง	วันที่ให้คะแนน	Δt	$\Delta t/h$	w	สัดส่วน
i2	Inside Out 2	26 ก.ย. 2569	5	0.1667	0.8909	50.3%
i1	Toy Story 5	11 ก.ย. 2569	20	0.6667	0.6300	35.6%
i3	The Wild Robot	2 ส.ค. 2569	60	2.0000	0.2500	14.1%

ตารางที่ 4.21 วันที่สร้างคำแนะนำ 1 ต.ค. 2569 และ h = 30 วัน (วันที่สมมติ)



ครึ่งชีวิต h มาจากไหน

- ข่าว: ชั่วโมง–วัน • ภาพยนตร์/สินค้า: สัปดาห์–เดือน • h เล็ก = ไวต่อพฤติกรรมล่าสุด ลืมเร็ว • h $\longrightarrow \infty$ = โปรไฟล์ค่าเฉลี่ย

ตัวอย่างการคำนวณ 4.28 : โปรไฟล์ข้อความของ u1 ลดตามเวลา

โจทย์ ใช้วันที่ในตาราง 4.21 และ h = 30 วัน จงหาน้ำหนัก โปรไฟล์ข้อความ และ Top-3 ของ u1

1 อายุ ⟶ น้ำหนัก

- i2: $2^{-5/30} = 0.8909$
- i1: $2^{-20/30} = 0.6300$
- i3: $2^{-60/30} = 2^{-2} = 0.2500$

2 สัดส่วนอิทธิพล

- รวม = 1.7709
- i2 : i1 : i3
- = 50.3% : 35.6% : 14.1%

3 เฉลี่ยถ่วงน้ำหนัก

- mind = (0.8909×0.9879 + 0.6300×0.1287 + 0.2500×0.1688)/1.7709 = **0.5666**
- family 0.3058 • toys 0.3162

4 จัดอันดับ

อันดับ	1	2	3	4	5	6	7
รายการ	i12	i11	i5	i10	i8	i7	i9
คะแนน	0.3754	0.3633	0.2703	0.2483	0.1402	0.1386	0.1282

ตีความ i2 (ล่าสุด) มีอิทธิพลครึ่งหนึ่ง ⟶ mind เด่นขึ้น ⟶ i12 Project Hail Mary (mind สูง) ขึ้นอันดับ 1 แทน i5

ขั้นตอนโดยละเอียด • ตัวอย่างการคำนวณ 4.28 (ลดน้ำหนักตามเวลา)

ข้อมูลมาจากไหน

- เรื่องที่ชอบของ u1: ตารางที่ 4.5
- วันที่ให้คะแนน (สมมติ): ตารางที่ 4.21
- Δt ถึง 1 ต.ค. 2569: 5, 20, 60 วัน
- h = 30 วัน

เรื่อง	Δt/h	w	สัดส่วน
i2	0.1667	0.8909	50.3%
i1	0.6667	0.6300	35.6%
i3	2.0000	0.2500	14.1%
รวม		1.7709	100%

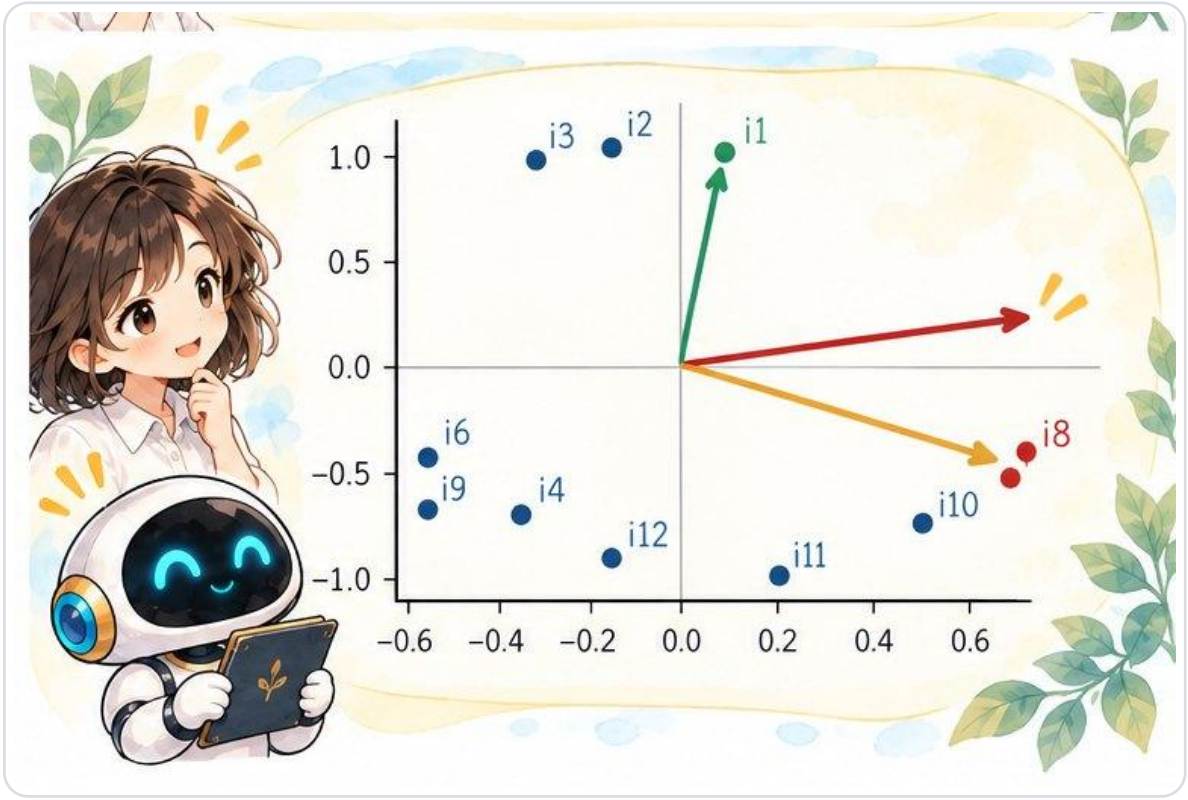
Top-3: i12 (0.3754), i11 (0.3633), i5 (0.2703)

4.5.6 ผู้ใช้ที่ชอบหลายแบบ : หลายจุดสนใจ และ Attention

$$s_m(u,i) = \max_{k=1,\ldots,K} \mathbf{p}_u^{m,k} \cdot \mathbf{v}_i^m \qquad (4.41)$$

$$\mathbf{p}_u^m(i) \propto \sum_{j \in L_u} \alpha_{ij} \mathbf{v}_j^m, \quad \alpha_{ij} = \frac{\exp(\mathbf{v}_i^m \cdot \mathbf{v}_j^m / \tau)}{\sum_{l \in L_u} \exp(\mathbf{v}_i^m \cdot \mathbf{v}_l^m / \tau)} \qquad (4.42)$$

- u4 ชอบ i1 (แอนิเมชัน) และ i7, i8 (แอ็กชัน) ⟶ ค่าเฉลี่ยตก “กลางทาง”
- **Multi-interest** แบ่งกลุ่มเรื่องที่ชอบ ใช้จุดสนใจที่ใกล้สุด (MIND ใช้ K = 4–8)
- **Attention** สร้างโปรไฟล์ใหม่ต่อผู้สมัครแต่ละเรื่อง • τ = 0.5



τ เล็ก (0.1) ≈ เลือกเรื่องที่คล้ายสุดเรื่องเดียว • τ ใหญ่ (10) ≈ ค่าเฉลี่ย

ตัวอย่างการคำนวณ 4.29 : โปรไฟล์วิดีโอสองจุดสนใจของ u4

โจทย์ u4: i1 = 4, i7 = 5, i8 = 5 • เทียบ Top-3 ของโปรไฟล์ค่าเฉลี่ยกับสองจุดสนใจ (K = 2) บนวิดีโอ (Transformer)

1 เวกเตอร์ที่เตรียมแล้ว

- v_{i1} = (0.0885, 0.0111, 0.9960)
- v_{i7} = (0.6080, 0.6771, −0.4146) • v_{i8} = (0.6358, 0.6926, −0.3406)
- cos i7–i8 = 0.9968 • i1–i7 = −0.3516

2 โปรไฟล์ค่าเฉลี่ย

- p = (0.6890, 0.7140, 0.1245)
- i2 Inside Out 2 ได้เพียง 0.0066
- Top-3: i10 (0.5042), i11 (0.0801), i2 (0.0066)

3 แบ่งเป็นสองจุดสนใจ

- จุด 1 = {i1}: p^1 = (0.0885, 0.0111, 0.9960)
- จุด 2 = {i7, i8}: p^2 = (0.6224, 0.6854, −0.3779)

4

ผู้สมัคร	$p^1 \cdot v$	$p^2 \cdot v$	max
i2	0.9825	−0.4852	0.9825
i3	0.9333	−0.6337	0.9333
i10	−0.7297	0.8531	0.8531

ที่มาของข้อมูล • ตัวอย่างการคำนวณ 4.29 (u4 สองจุดสนใจ)

ข้อมูลมาจากไหน

- u4 (ตารางที่ 4.5): ชอบ i1 = 4, i7 = 5, i8 = 5 • ไม่ชอบ i5 = 1
- i1 แอนิเมชัน • i7, i8 แอ็กชัน (ตารางที่ 4.4)
- ผู้สมัคร 8 เรื่อง: i2, i3, i4, i6, i9, i10, i11, i12
- วิดีโอ: ตารางที่ 4.16 • μV = (0.5333, 0.4917, 0.2500)

เรื่อง	V	V − μV	v
i1	(0.6, 0.5, 1.0)	(0.0667, 0.0083, 0.7500)	(0.0885, 0.0111, 0.9960)
i7	(0.9, 0.9, 0.0)	(0.3667, 0.4083, −0.2500)	(0.6080, 0.6771, −0.4146)
i8	(1.0, 1.0, 0.0)	(0.4667, 0.5083, −0.2500)	(0.6358, 0.6926, −0.3406)
i2	(0.5, 0.4, 1.0)	(−0.0333, −0.0917, 0.7500)	(−0.0441, −0.1212, 0.9916)
i3	(0.4, 0.3, 1.0)	(−0.1333, −0.1917, 0.7500)	(−0.1697, −0.2440, 0.9548)
i4	(0.4, 0.3, 0.0)	(−0.1333, −0.1917, −0.2500)	(−0.3898, −0.5603, −0.7308)
i6	(0.2, 0.2, 0.0)	(−0.3333, −0.2917, −0.2500)	(−0.6554, −0.5735, −0.4915)
i9	(0.3, 0.4, 0.0)	(−0.2333, −0.0917, −0.2500)	(−0.6591, −0.2589, −0.7061)
i10	(0.7, 0.6, 0.0)	(0.1667, 0.1083, −0.2500)	(0.5218, 0.3392, −0.7827)
i11	(0.6, 0.5, 0.0)	(0.0667, 0.0083, −0.2500)	(0.2575, 0.0322, −0.9657)
i12	(0.5, 0.4, 0.0)	(−0.0333, −0.0917, −0.2500)	(−0.1242, −0.3416, −0.9316)

cos(i7, i8) = 0.9968 • cos(i1, i7) = −0.3516

ขั้นตอนโดยละเอียด • ตัวอย่างการคำนวณ 4.29 (คะแนนแปดเรื่อง)

สองวิธีให้คะแนน

- ค่าเฉลี่ย: p = (0.6890, 0.7140, 0.1245)
- จุดสนใจ 1 = v_i1 • จุดสนใจ 2 = l2(v_i7 + v_i8)
- คะแนนสองจุดสนใจ = max(p1•v, p2•v)

ผู้สมัคร	ค่าเฉลี่ย	p1•v	p2•v	max
i2	0.0066	0.9825	−0.4852	0.9825
i3	−0.1723	0.9333	−0.6337	0.9333
i4	−0.7596	−0.7686	−0.3505	−0.3505
i6	−0.9222	−0.5539	−0.6152	−0.5539
i9	−0.7269	−0.7645	−0.3208	−0.3208
i10	0.5042	−0.7297	0.8531	0.8531
i11	0.0801	−0.9387	0.5473	0.5473
i12	−0.4455	−0.9427	0.0406	0.0406

ค่าเฉลี่ย: i10, i11, i2 → สองจุดสนใจ: i2, i3, i10

ตัวอย่างการคำนวณ 4.30 : โปรไฟล์แบบ Attention ของ u4

โจทย์ สร้างโปรไฟล์วิดีโอแบบ Attention ของ u4 สำหรับผู้สมัคร i2 Inside Out 2 และ i10 Dune: Part Two เมื่อ $\tau = 0.5$

1 i2 ถามประวัติ

	i1	i7	i8
cos กับประวัติ	0.9825	−0.5200	−0.4497
÷ $\tau = 0.5$	1.9650	−1.0400	−0.8994
α (softmax)	0.9037	0.0448	0.0515

โปรไฟล์ ≈ i1 ⟶ คะแนน i2 = 0.9577

2 i10 ถามประวัติ

	i1	i7	i8
cos กับประวัติ	−0.7297	0.8714	0.8333
÷ $\tau = 0.5$	−1.4594	1.7428	1.6666
α (softmax)	0.0207	0.5083	0.4710

โปรไฟล์เอียงไป i7, i8 ⟶ คะแนน i10 = 0.8438

ตีความ ไม่ต้องกำหนดจำนวนจุดสนใจล่วงหน้า แต่ต้องคำนวณโปรไฟล์ใหม่ **ทุกครั้ง** ที่ให้คะแนนผู้สมัคร ⟶ ใช้การคำนวณมากกว่า

ขั้นตอนโดยละเอียด • ตัวอย่างการคำนวณ 4.30 (โปรไฟล์แบบ Attention)

ข้อมูลมาจากไหน

- ประวัติ u4 = {i1, i7, i8} (ตารางที่ 4.5)
- เวกเตอร์วิดีโอที่เตรียมแล้ว (ตัวอย่าง 4.29)
- i2 แอนิเมชัน • i10 แอ็กชัน (ตารางที่ 4.4) • $\tau = 0.5$

ผู้สมัคร	cos กับ i1, i7, i8	α	คะแนน
i2	(0.9825, −0.5200, −0.4497)	(0.9037, 0.0448, 0.0515)	0.9577
i10	(−0.7297, 0.8714, 0.8333)	(0.0207, 0.5083, 0.4710)	0.8438

i2 ดึงจาก i1 ($\alpha = 0.9037$) • i10 ดึงจาก i7, i8

4.5.7 ลำดับผู้ใช้ด้วย CNN (Caser) และ Transformer (SASRec)

$$\mathbf{p}_u^{Caser} = \sum_{t=1}^{L} c_t \, \mathbf{v}_{(t)}, \qquad \mathbf{p}_u^{SAS} = \sum_{t=1}^{L} \alpha_{L,t} \, \mathbf{v}_{(t)}, \qquad \alpha_{L,t} = \mathrm{softmax}_t\left(\frac{\mathbf{v}_{(L)} \cdot \mathbf{v}_{(t)}}{\sqrt{d}}\right) \tag{4.43}$$

โจทย์ ตัวอย่าง 4.31 : u4 ดู i1 ⟶ i7 ⟶ i8 ตามลำดับ • Caser ใช้ c = (0.2, 0.3, 0.5) • SASRec ใช้ตำแหน่งสุดท้าย (i8) ถามทุกตำแหน่ง

1 Caser : ถ่วงตามตำแหน่งเวลา

- $p = 0.2v_{i1} + 0.3v_{i7} + 0.5v_{i8} = (0.5180, 0.5516, -0.0955)$
- Top-3: **i10 (0.6977)**, i11 (0.3191), i12 (−0.2148)

2 SASRec : Causal Self-Attention

- ผลคูณจุด ÷ √3: (−0.1589, 0.5755, 0.5774) ⟶ α = (0.1933, 0.4030, 0.4037)
- p = (0.5188, 0.5546, −0.1120) • Top-3: **i10 (0.7119)**, i11, i12

ตีความ ทั้งสองให้ความสำคัญเรื่องล่าสุด (แอ็กชัน) ⟶ แนะนำ i10 • ต่างจากหลายจุดสนใจที่ยังเก็บแอนิเมชันไว้ • ระบบจริงต้องฝึก c_t และ W บนผู้ใช้จำนวนมาก

ขั้นตอนโดยละเอียด • ตัวอย่างการคำนวณ 4.31 (Caser และ SASRec)

ข้อมูลมาจากไหน

- เรื่องที่ชอบของ u4: ตารางที่ 4.5
- ลำดับ i1 → i7 → i8 เป็นค่าสมมติ
- c = (0.2, 0.3, 0.5) แทนน้ำหนักที่ Caser เรียนรู้
- SASRec หารด้วย $\sqrt{3}$ (d = 3)

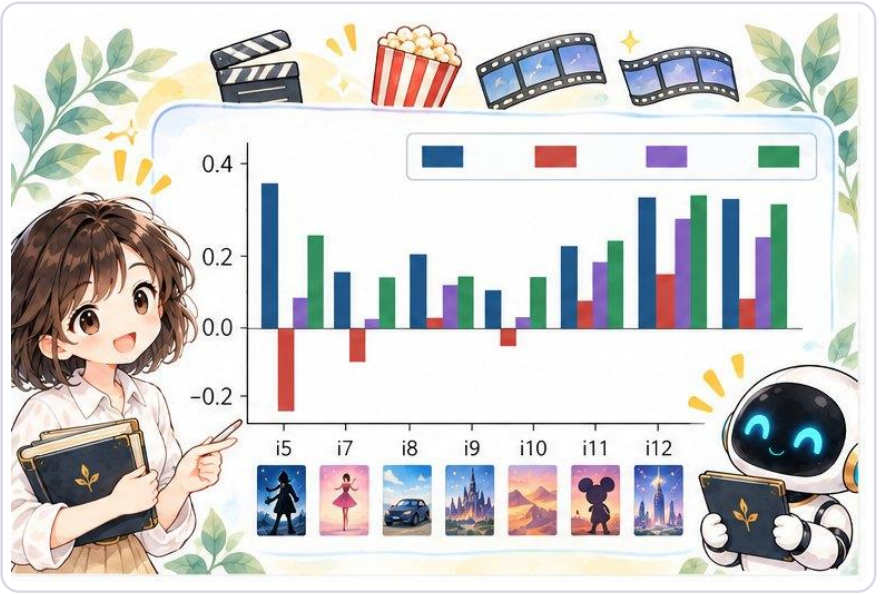
ผู้สมัคร	Caser	SASRec
i2	−0.2417	−0.2620
i3	−0.4113	−0.4303
i4	−0.5785	−0.5616
i6	−0.7983	−0.7855
i9	−0.5465	−0.5294
i10	0.6977	0.7119
i11	0.3191	0.3382
i12	−0.2148	−0.1948

ทั้งสองแบบ: i10 อันดับ 1 เพราะเน้นเรื่องล่าสุด (แอ็กชัน)

4.5.8 เปรียบเทียบโปรไฟล์จากเทคนิค CNN และ Transformer

วิธีสร้างโปรไฟล์	cos โปรไฟล์ CNN–TF (ต่ำสุด)	Top-3 CNN	Top-3 Transformer
ค่าเฉลี่ยเรื่องที่ชอบ	0.9781	i5, i8, i9	i5, i8, i9
ถ่วงด้วยคะแนน	0.9825	i8, i7, i5	i8, i7, i5
Rocchio $\beta = 0.5$	0.9738	i5, i8, i7	i5, i8, i7
ลดตามเวลา h = 30	0.9837	i5, i8, i9	i5, i8, i12

Top-3 หลังรวมสี่สื่อด้วยน้ำหนัก (0.4, 0.2, 0.2, 0.2)



เลือกวิธีสร้างโปรไฟล์อย่างไร

- เริ่มจาก **ค่าเฉลี่ย** (ไม่มีไฮเปอร์พารามิเตอร์ และ MRR ดีสุดเมื่อรวมสี่สื่อ) • Rocchio เมื่อเรื่องไม่ชอบชัดเจน (เลือก β บนชุดตรวจสอบ)
- **ลดตามเวลา** เมื่อรสนิยมเปลี่ยนเร็ว • **หลายจุดสนใจ/Attention** เมื่อชอบหลายแนว • Caser/SASRec เมื่อมีลำดับยาวและผู้ใช้มาก

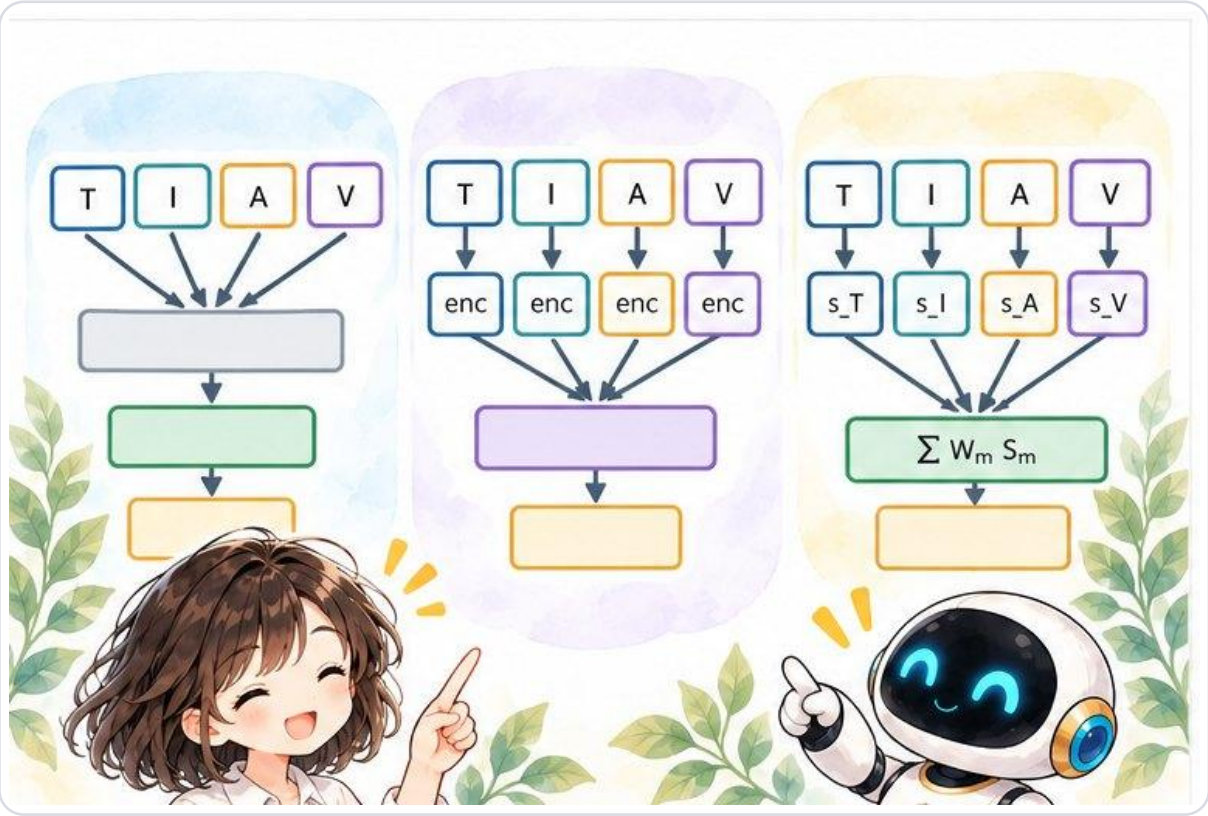
4.6

Multimodal Fusion : รวมหลักฐานจากหลายสื่อ

Late Fusion • z-score • Borda/RRF • Grid Search • Early • Gate • สื่อหาย • CLIP • VBPR



4.6.1 สามระดับของการรวม



E Early Fusion

ต่อเวกเตอร์ทุกสื่อเป็นเส้นเดียว ก่อนสร้างโปรไฟล์และให้คะแนน (4.6.6)

I Intermediate Fusion

แต่ละสื่อผ่านตัวเข้ารหัสของตน แล้วรวมตรงกลางด้วย Gate/Attention (4.6.7)

L Late Fusion

ให้คะแนนแยกสื่อจนจบ แล้วรวมคะแนนหรืออันดับ — ง่ายที่สุด แนะนำให้เริ่ม (4.6.2–4.6.5)

กรรมการสี่คน Late = ให้คะแนนแยกแล้วถ่วงน้ำหนัก • Early = นั่งดูพร้อมกันแล้วสรุปความเห็นเดียว • Intermediate = ประธานตัดสินว่าผู้ชมคนนี้ควรเชื่อกรรมการคนใด

4.6.2 Late Fusion และตัวอย่างการคำนวณ 4.32

$$s(u,i) = \sum_{m \in \{T,I,A,V\}} w_m \, s_m(u,i), \qquad \sum_m w_m = 1, \; w_m \geq 0 \tag{4.44}$$

โจทย์ น้ำหนักตั้งต้น $(w_T, w_I, w_A, w_V) = (0.4, 0.2, 0.2, 0.2)$ • คะแนนรายสื่อ (Transformer) จากตาราง 4.19 • หา Top-3 ของ u1

รหัส	T	I	A	V	0.4T + 0.2I + 0.2A + 0.2V	s	อันดับ
i5	0.4109	0.9180	0.9929	−0.6417	0.1644 + 0.1836 + 0.1986 − 0.1283	0.4182	1
i8	0.1736	−0.0053	−0.8548	−0.4474	0.0694 − 0.0011 − 0.1710 − 0.0895	−0.1921	2
i9	0.1086	−0.8084	−0.3154	−0.6417	0.0434 − 0.1617 − 0.0631 − 0.1283	−0.3097	3
i12	0.3943	−0.8268	−0.6487	−0.8781	0.1577 − 0.1654 − 0.1297 − 0.1756	−0.3130	4
i11	0.3883	−0.7392	−0.9816	−0.9727	0.1553 − 0.1478 − 0.1963 − 0.1945	−0.3833	5
i7	0.1526	−0.8154	−0.9609	−0.5177	0.0610 − 0.1631 − 0.1922 − 0.1035	−0.3978	6
i10	0.2427	−0.6555	−0.9799	−0.8390	0.0971 − 0.1311 − 0.1960 − 0.1678	−0.3978	7

ตีความ i8 ขึ้นอันดับ 2 เพราะภาพ+วิดีโอ ทั้งที่ข้อความให้ i8 อยู่อันดับ 5 • i7 กับ i10 ต่างกันที่ทศนิยมตำแหน่งที่ 5

ขั้นตอนโดยละเอียด • ตัวอย่างการคำนวณ 4.32 (Late Fusion ครบเจ็ดเรื่อง)

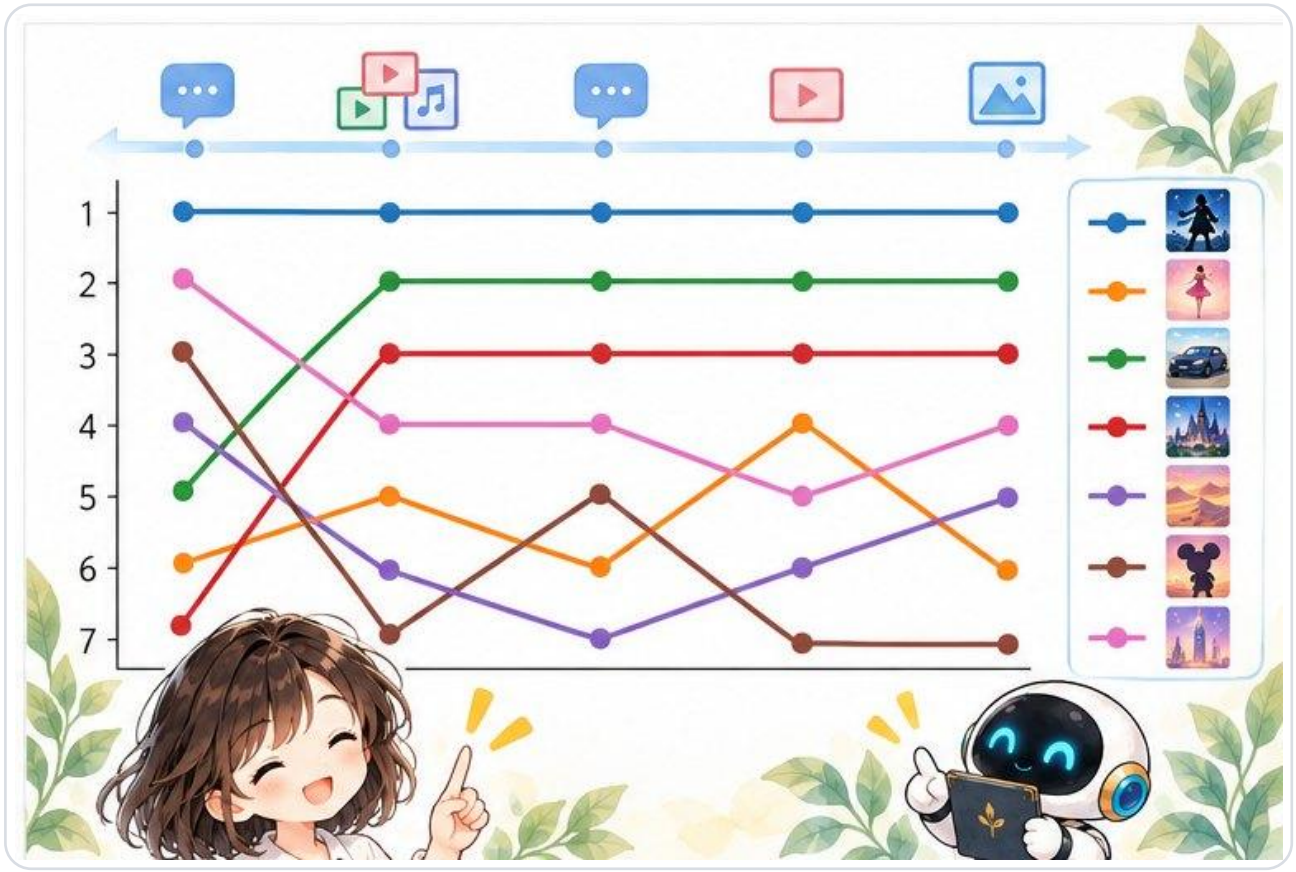
ข้อมูลมาจากไหน

- คะแนนรายสื่อ: ตารางที่ 4.19 (โปรไฟล์ค่าเฉลี่ยของ u1)
- u1 ชอบ i1, i2, i3 (ตารางที่ 4.5)
- ผู้สมัคร 7 เรื่องที่ u1 ยังไม่ให้คะแนน
- w = (0.4, 0.2, 0.2, 0.2)

เรื่อง	T	I	A	V	S
i5	0.4109	0.9180	0.9929	−0.6417	0.4182
i8	0.1736	−0.0053	−0.8548	−0.4474	−0.1921
i9	0.1086	−0.8084	−0.3154	−0.6417	−0.3097
i12	0.3943	−0.8268	−0.6487	−0.8781	−0.3130
i11	0.3883	−0.7392	−0.9816	−0.9727	−0.3834
i7	0.1526	−0.8154	−0.9609	−0.5177	−0.3978
i10	0.2427	−0.6555	−0.9799	−0.8390	−0.3978

Top-3: i5 (0.4182), i8 (−0.1921), i9 (−0.3097)

น้ำหนักตั้งต้นมาจากไหน และผลเมื่อเปลี่ยนน้ำหนัก



ภาพที่ 4.44 อันดับของเจ็ดเรื่องภายใต้น้ำหนักห้าชุด

ที่มาของไฮเปอร์พารามิเตอร์

- **ที่มา** ผู้เขียนตั้งให้ข้อความ = 2 เท่าของสื่ออื่น เพราะบทที่ 3 พิสูจน์แล้ว (ไม่ได้มาจากการค้นหา)
- **ผลเมื่อเปลี่ยน** อันดับ 1 คงที่ แต่อันดับ 2–7 เปลี่ยนมาก โดยเฉพาะ i12 และ i11 ที่ข้อความชอบแต่สื่ออื่นไม่ชอบ
- **วิธีเลือก** Grid Search บนชุดตรวจสอบ (4.6.5) หรือ Gate ตามผู้ใช้ (4.6.7)

4.6.3 ปรับมาตราคะแนน และตัวอย่าง 4.33 (1/2)

$$z_m(u,i) = \frac{s_m(u,i) - \mu_m}{\sigma_m} \qquad (4.45)$$

$$n_m(u,i) = \frac{s_m(u,i) - \min_j s_m(u,j)}{\max_j s_m(u,j) - \min_j s_m(u,j)} \qquad (4.46)$$

ข้อควรระวัง

- μ, σ คำนวณจากผู้สมัครของผู้ใช้คนเดียว ถ้าน้อยเรื่องจะไม่เสถียร
- ถ้า $\sigma = 0$ ให้ $z = 0$ (กันหารศูนย์)
- Min-Max ไวต่อค่าผิดปกติเรื่องเดียว

1 สถิติของคะแนนข้อความ (7 เรื่อง)

รหัส	s_T	s_T − μ	(s_T − μ)²
i5	0.4109	0.1436	0.02063
i7	0.1526	−0.1147	0.01315
i8	0.1736	−0.0937	0.00878
i9	0.1086	−0.1587	0.02518
i10	0.2427	−0.0246	0.00060
i11	0.3883	0.1210	0.01464
i12	0.3943	0.1270	0.01613
รวม	1.8710	0	0.09912

μ_T = 1.8710/7
= 0.2673

σ^2_T = 0.09912/7
= 0.01416

σ_T = √0.01416
= 0.1190
(หารด้วย 7)

ตัวอย่างการคำนวณ 4.33 : z-score ทุกสื่อแล้วรวม (2/2)

รหัส	z_T	z_I	z_A	z_V	รวม 0.4z_T + 0.2(z_I + z_A + z_V)	อันดับ
i5	1.2070	2.1970	2.3054	0.3526	1.4538	1
i12	1.0676	−0.6702	−0.1707	−0.9544	0.0680	2
i8	−0.7875	0.6797	−0.4817	1.4267	0.0100	3
i11	1.0170	−0.5262	−0.6729	−1.4774	−0.1285	4
i7	−0.9637	−0.6515	−0.6417	1.0380	−0.4365	5
i10	−0.2070	−0.3888	−0.6704	−0.7381	−0.4422	6
i9	−1.3333	−0.6400	0.3320	0.3526	−0.5244	7
μ / σ	0.2673 / 0.1190	−0.4190 / 0.6085	−0.5355 / 0.6630	−0.7055 / 0.1809		

- i5: (0.4109 − 0.2673)/0.1190 = 1.2070
- i12 รวม: 0.4(1.0676) + 0.2(−0.6702 − 0.1707 − 0.9544) = 0.0680

ตีความ ความต่างเล็กของข้อความถูกขยายเป็นหลายเท่าของ σ ⟶ ข้อความ “เสียงดัง” เท่าสื่ออื่น ⟶ i12 ขึ้นอันดับ 2

ขั้นตอนโดยละเอียด • ตัวอย่างการคำนวณ 4.33 (z-score ทุกสื่อ)

ข้อมูลมาจากไหน

- คะแนนรายสื่อ: ตารางที่ 4.19
- μ, σ คิดจาก 7 ผู้สมัครของ u1 (หารด้วย 7)
- $z = (s - \mu)/\sigma$ แล้วรวมด้วย (0.4, 0.2, 0.2, 0.2)

สื่อ	μ	σ	z(i5)	z(i8)	z(i12)
ข้อความ	0.2673	0.1190	1.2070	−0.7874	1.0676
ภาพ	−0.4190	0.6085	2.1970	0.6797	−0.6702
เสียง	−0.5355	0.6630	2.3054	−0.4817	−0.1707
วิดีโอ	−0.7055	0.1809	0.3526	1.4267	−0.9544
รวม			1.4538	0.0100	0.0680

Top-3: i5 (1.4538), i12 (0.0680), i8 (0.0100)

4.6.4 รวมด้วยอันดับ : Borda Count และ RRF (ตัวอย่าง 4.34)

$$B(i) = \sum_{m}(n - rank_m(i)) \qquad (4.47)$$

$$RRF(i) = \sum_{m}\frac{1}{k + rank_m(i)} \qquad (4.48)$$

- n จำนวนผู้สมัคร (= 7)
- $rank_m(i)$ อันดับในสื่อ m เริ่มจาก 1
- k ลดอิทธิพลอันดับต้น Cormack et al. (2009) ใช้ k = 60
- ทิ้งค่าคะแนน ใช้เพียง **ลำดับ** ⟶ ไม่มีปัญหามาตรา

1 อันดับรายสื่อ (จากตาราง 4.19)

อันดับ	ข้อความ T	ภาพ I	เสียง A	วิดีโอ V
1	i5	i5	i5	i8
2	i12	i8	i9	i7
3	i11	i10	i12	i5
4	i10	i11	i8	i9
5	i8	i9	i7	i10
6	i7	i7	i10	i12
7	i9	i12	i11	i11

ฟ้า = i5 อันดับ (1, 1, 1, 3) • ส้ม = i8 อันดับ (5, 2, 4, 1)

ตัวอย่างการคำนวณ 4.34 : คะแนน Borda และ RRF

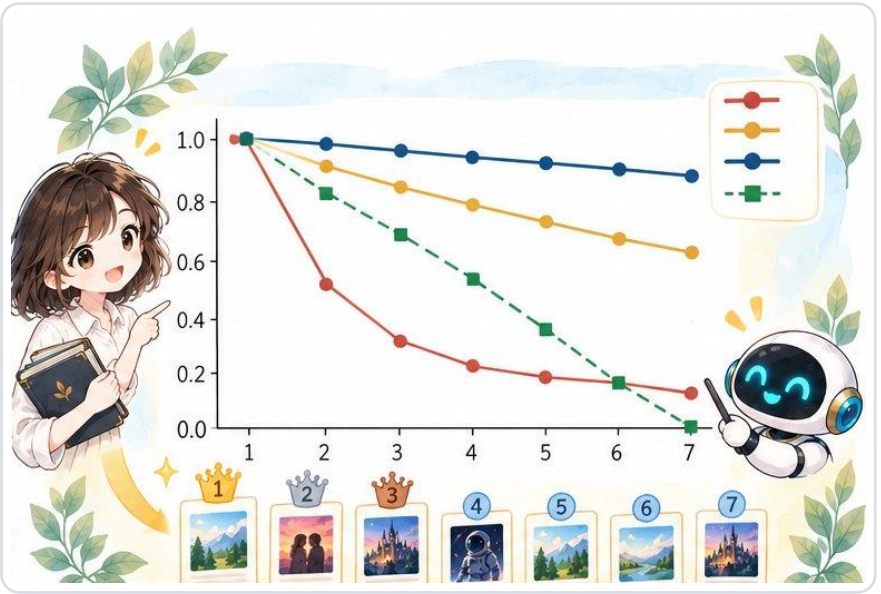
2 Borda (n = 7)

- i5 (1,1,1,3): 6 + 6 + 6 + 4 = 22
- i8 (5,2,4,1): 2 + 5 + 3 + 6 = 16

3 RRF (k = 60)

- i5: 3/61 + 1/63 = 0.0651
- i8: 1/65 + 1/62 + 1/64 + 1/61 = 0.0635

รหัส	อันดับ (T, I, A, V)	Borda	RRF k=60	RRF k=0
i5	(1, 1, 1, 3)	22	0.065053	3.3333
i8	(5, 2, 4, 1)	16	0.063532	1.9500
i12	(2, 7, 3, 6)	10	0.062079	1.1429
i9	(7, 5, 2, 4)	10	0.062064	1.0929
i10	(4, 3, 6, 5)	10	0.062034	0.9500
i7	(6, 6, 5, 2)	9	0.061817	1.0333
i11	(3, 4, 7, 7)	7	0.061349	0.8690



ขั้นตอนโดยละเอียด • ตัวอย่างการคำนวณ 4.34 (อันดับครบเจ็ดเรื่อง)

ข้อมูลมาจากไหน

- อันดับรายสื่อ = เรียงคะแนนแต่ละคอลัมน์ของตารางที่ 4.19
- n = 7 ผู้สมัครของ u1 • k = 60
- Borda = $\Sigma(7 - \text{rank})$ • RRF = $\Sigma\, 1/(60 + \text{rank})$

เรื่อง	T	I	A	V	Borda	RRF
i5	1	1	1	3	22	0.0651
i8	5	2	4	1	16	0.0635
i12	2	7	3	6	10	0.0621
i9	7	5	2	4	10	0.0621
i10	4	3	6	5	10	0.0620
i7	6	6	5	2	9	0.0618
i11	3	4	7	7	7	0.0613

RRF Top-3: i5, i8, i12

ตีความ Borda / RRF และค่า k

ผลที่ได้

- Top-3 ของ RRF (k = 60): **i5, i8, i12**
- i8 ขึ้นอันดับ 2 เพราะอยู่อันดับต้นในภาพและวิดีโอ
- ไม่สนว่าคะแนนห่างกันเท่าใด สนเพียงลำดับ

ปัญหาของ k = 60 เมื่อรายการสั้น

- คะแนนทุกเรื่องใกล้กันมาก (0.0613–0.0651)
- อันดับต้นมีอิทธิพลไม่มากกว่าอันดับท้ายนัก
- k = 0 ให้รางวัลอันดับ 1 สูง: i5 = 3.3333, i8 = 1.95

ที่มาของ k = 60

- Cormack et al. (2009) พบว่า k = 60 ให้ผลดีสม่ำเสมอเมื่อรวมผลค้นหาจากหลายระบบ ⟶ ค่าตั้งต้นในไลบรารีส่วนใหญ่
- MovieMini-12 มีผู้สมัครเพียง 7–9 เรื่อง ⟶ **k = 0 ให้ผลดีกว่า** (HR@3 0.85, MRR 0.7921 บน Transformer)
- k = 60 เหมาะกับรายการยาวหลายร้อย ถ้ารายการสั้นควรลองค่าเล็กบนชุดตรวจสอบ

4.6.5 Grid Search และตัวอย่างการคำนวณ 4.35

$$N_{grid} = \binom{G+M-1}{M-1} \qquad (4.49)$$

G จำนวนก้าว (1/0.1 = 10) • M จำนวนสื่อ (= 4)

1 นับชุดน้ำหนัก

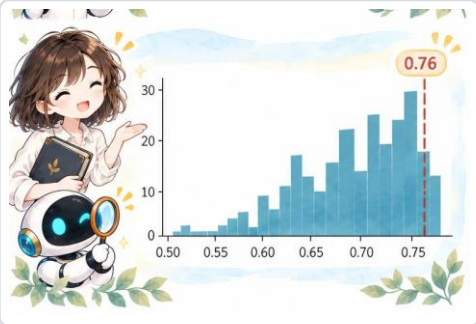
- C(13, 3) = 286 ชุด

2 ค้นหาบนผู้ใช้ทั้งแปดคน

- ดีที่สุด (0.60, 0.10, 0.20, 0.10): MRR 0.7755 vs ตั้งต้น 0.7635 • มี 20/286 ชุดที่ดีกว่า

3 เลือกบน u1–u4 แล้วทดสอบบน u5–u8

- ดีสุดบน u1–u4: (0.00, 0.00, 0.10, 0.90) MRR **0.9333**
- นำไปใช้กับ u5–u8: MRR เพียง 0.5192 < ตั้งต้น 0.6095



ตีความ เลือกน้ำหนักบนข้อมูลที่ใช้วัดผล ⟶ ตัวเลขดูดีเกินจริง (Overfitting) โดยเฉพาะข้อมูลเล็ก ⟶ ต้องแยกชุดเลือกน้ำหนักออกจากชุดทดสอบเสมอ (บทที่ 2)

ที่มาของข้อมูล • ตัวอย่างการคำนวณ 4.35 (Grid Search)

ข้อมูลมาจากไหน

- 20 กรณีจากตารางที่ 4.5: u1–u4 คนละ 3 (12 กรณี) • u5–u8 คนละ 2 (8 กรณี)
- จำนวนชุด = C(13, 3) = 286
- เลือกบน u1–u4 → ทดสอบบน u5–u8

ชุดน้ำหนัก	MRR (8 คน)
(0.60, 0.10, 0.20, 0.10)	0.7755
(0.50, 0.10, 0.20, 0.20)	0.7738
(0.10, 0.10, 0.20, 0.60)	0.7713
(0.20, 0.10, 0.20, 0.50)	0.7713
(0.30, 0.10, 0.20, 0.40)	0.7713
ตั้งต้น	0.7635

เลือกบน u1–u4 ได้ 0.9333 แต่ทดสอบบน u5–u8 เหลือ 0.5192

4.6.6 Early Fusion และตัวอย่างการคำนวณ 4.36

$$\mathbf{x}_i = [\sqrt{w_T}\,\mathbf{v}_i^T \| \sqrt{w_I}\,\mathbf{v}_i^I \| \sqrt{w_A}\,\mathbf{v}_i^A \| \sqrt{w_V}\,\mathbf{v}_i^V], \qquad \mathbf{q}_u \cdot \mathbf{x}_i = \sum_m w_m\, \mathbf{p}_u^m \cdot \mathbf{v}_i^m \tag{4.50}$$

โจทย์ ต่อเวกเตอร์ i5 และโปรไฟล์ u1 ยาว 12 + 3 + 3 + 3 = 21 มิติ ด้วยน้ำหนักตั้งต้น แล้วตรวจว่าผลคูณจุด = Late Fusion

1 ตัวคูณของแต่ละสื่อ

- √0.4 = 0.6325 (ข้อความ)
- √0.2 = 0.4472 (สื่ออื่น)

2 ผลคูณจุดของเวกเตอร์ 21 มิติ

- √w × √w = w ⟶ ส่วนข้อความ: 0.4 × 0.4109 = 0.1644
- ภาพ เสียง วิดีโอ: 0.2 × [0.9180 + 0.9929 + (−0.6417)] = 0.2538 • รวม = 0.4182 = Late Fusion

ตีความ ต่อเวกเตอร์ตรง ๆ + ผลคูณจุด ไม่ได้อะไรเพิ่มจาก Late Fusion • ข้อได้เปรียบเกิดเมื่อส่งเวกเตอร์ต่อเข้า **แบบจำลองที่เรียนรู้ได้** (MLP) จับปฏิสัมพันธ์ข้ามสื่อ แต่ต้องการข้อมูลมาก

ขั้นตอนโดยละเอียด • ตัวอย่างการคำนวณ 4.36 (Early Fusion 21 มิติ)

ข้อมูลมาจากไหน

- โปรไฟล์ u1 สี่สื่อ: ตัวอย่าง 4.25 และภาพที่ 4.37 (เฉลี่ยจาก i1, i2, i3)
- เวกเตอร์ i5: ตารางที่ 4.15 + 4.14 → 12 + 3 + 3 + 3 = 21 มิติ
- คูณแต่ละส่วนด้วย $\sqrt{w}$ ทั้งโปรไฟล์และรายการ

ส่วน	$\sqrt{w}$	โคไซน์รายสื่อ	w × โคไซน์
ข้อความ 1–12	0.6325	0.4109	0.1644
ภาพ 13–15	0.4472	0.9180	0.1836
เสียง 16–18	0.4472	0.9929	0.1986
วิดีโอ 19–21	0.4472	−0.6417	−0.1283
รวม			0.4182

$\sqrt{w} \times \sqrt{w} = w$ → ผลคูณจุด 21 มิติ = Late Fusion = 0.4182

4.6.7 Gate : น้ำหนักเฉพาะผู้ใช้ และตัวอย่าง 4.37 (1/2)

$$g_m(u) = \frac{2}{|L_u|(|L_u|-1)} \sum_{i<j \in L_u} \mathbf{v}_i^m \cdot \mathbf{v}_j^m, \qquad w_m(u) = \frac{\exp(g_m(u)/\tau)}{\sum_k \exp(g_k(u)/\tau)} \tag{4.51}$$

g_m ความสม่ำเสมอ = ค่าเฉลี่ยโคไซน์ของทุกคู่เรื่องที่ชอบ • τ = 0.5 • ชอบน้อยกว่า 2 เรื่อง ⟶ ให้น้ำหนักเท่ากัน 0.25

1 ความสม่ำเสมอของ u1 (เวกเตอร์ที่เตรียมแล้ว)

สื่อ	i1–i2	i1–i3	i2–i3	g_m = เฉลี่ย 3 คู่
T ข้อความ	0.1817	0.4179	0.2876	0.2957
I ภาพ	0.9390	0.9523	0.8638	0.9184
A เสียง	0.7412	0.3948	0.6765	0.6042
V วิดีโอ	0.9825	0.9333	0.9839	0.9665

ตัวอย่างภาพ

- g_I = (0.9390 + 0.9523 + 0.8638)/3
- = 0.9184
- แอนิเมชันสามเรื่องคล้ายกันมากในภาพและวิดีโอ แต่เรื่องย่อต่างกัน

ตัวอย่างการคำนวณ 4.37 : softmax เป็นน้ำหนัก แล้วรวม (2/2)

2 softmax หลังหารด้วย $\tau = 0.5$

	T	I	A	V	รวม
g_m	0.2957	0.9184	0.6042	0.9665	
g_m / 0.5	0.5914	1.8368	1.2084	1.9330	
exp(•)	1.8066	6.2765	3.3479	6.9107	18.3417
w_m(u1)	0.0985	0.3422	0.1825	0.3768	1.0000

3 รวมคะแนนของ i5

- 0.0985(0.4109) + 0.3422(0.9180) + 0.1825(0.9929) + 0.3768(−0.6417)
- = 0.2941
- Top-3: i5 (0.2941), i8 (−0.3093), i9 (−0.5653)

- ตีความ Gate ลดน้ำหนักข้อความเหลือ 0.0985 เพราะเรื่องย่อของสามเรื่องต่างกัน
- **ข้อควรระวัง** ความสม่ำเสมอสูง ≠ แยกแยะผู้สมัครได้ดี — ผู้สมัครทั้งเจ็ดไม่มีแอนิเมชันเลย ⟶ MRR ของ Gate (0.7175) ต่ำกว่า Late (0.7635)

ขั้นตอนโดยละเอียด • ตัวอย่างการคำนวณ 4.37 (น้ำหนัก Gate ของ u1)

ข้อมูลมาจากไหน

- สามเรื่องที่ u1 ชอบ = แอนิเมชันทั้งหมด (ตารางที่ 4.4, 4.5)
- g = เฉลี่ยโคไซน์ 3 คู่ในแต่ละสื่อ
- w = softmax(g/0.5)
- คะแนนรายสื่อ: ตารางที่ 4.19

สื่อ	i1–i2	i1–i3	i2–i3	g	w
ข้อความ	0.1817	0.4179	0.2875	0.2957	0.0985
ภาพ	0.9390	0.9523	0.8638	0.9184	0.3422
เสียง	0.7412	0.3948	0.6765	0.6042	0.1825
วิดีโอ	0.9825	0.9333	0.9839	0.9665	0.3768

Top-3: i5 (0.2941), i8 (−0.3093), i9 (−0.5653)

4.6.8 เมื่อบางสื่อหายไป และตัวอย่างการคำนวณ 4.38

$$s(u,i) = \frac{\sum_{m \in A_i} w_m \, s_m(u,i)}{\sum_{m \in A_i} w_m} \qquad (4.52)$$

A_i สื่อที่รายการ i มีอยู่ • i12 Project Hail Mary (2026) อาจยังไม่มีตัวอย่างภาพยนตร์

ตีความ

- แทนด้วย 0 ⟶ i12 **ได้ประโยชน์** จากการไม่มีข้อมูล (คะแนนวิดีโอจริงติดลบ)
- กระจายใหม่ = “ประเมินจากหลักฐานที่มี” ยุติธรรมกว่า และควรบันทึกว่าคะแนนมาจากสื่อใดบ้าง

1 มีครบสี่สื่อ : −0.3130 (จากตัวอย่าง 4.32)

2 แทนวิดีโอด้วย 0

- 0.4(0.3943) + 0.2(−0.8268) + 0.2(−0.6487) + 0.2(0) = **−0.1373**

3 กระจายน้ำหนักใหม่

- −0.1373 / 0.8 = **−0.1717** = ใช้น้ำหนัก (0.5, 0.25, 0.25)

ขั้นตอนโดยละเอียด • ตัวอย่างการคำนวณ 4.38 (i12 ไม่มีวิดีโอ)

ข้อมูลมาจากไหน

- i12 Project Hail Mary ฉายปี 2026 (ตารางที่ 4.4) → สมมติว่ายังไม่มีตัวอย่างภาพยนตร์
- คะแนนรายสื่อของ i12: ตารางที่ 4.19
- น้ำหนักตั้งต้น (0.4, 0.2, 0.2, 0.2)

กรณี	การคำนวณ	s(u1, i12)
มีครบ	0.1577 − 0.1654 − 0.1297 − 0.1756	−0.3130
แทน V ด้วย 0	0.1577 − 0.1654 − 0.1297 + 0	−0.1373
กระจายน้ำหนักใหม่	−0.1373 / 0.8 (w = 0.5, 0.25, 0.25)	−0.1717

แทนด้วย 0 ทำให้ i12 ขึ้นอันดับ 2 โดยไม่ตั้งใจ • กระจายใหม่ = ประเมินจากหลักฐานที่มี

4.6.9 จัดแนวข้ามสื่อแบบ CLIP

$$p_{ij} = \frac{\exp(S_{ij}/\tau)}{\sum_{k=1}^{B} \exp(S_{ik}/\tau)}, \qquad S_{ij} = \cos(f_{img}(x_i), f_{txt}(t_j)) \tag{4.53}$$

$$\mathcal{L}_{CLIP} = \frac{1}{2}\left(-\frac{1}{B}\sum_{i} \ln p_{ii} - \frac{1}{B}\sum_{j} \ln p_{jj}^{txt}\right) \tag{4.54}$$

- ฝึกตัวเข้ารหัสภาพ (ViT) และข้อความพร้อมกันบนคู่ภาพ–คำบรรยาย **400 ล้านคู่** (Contrastive Learning)
- คู่ (x_i, t_i) คือคู่ถูก • B ขนาดชุด • τ อุณหภูมิ (เริ่ม 0.07, B = 32,768)
- ผล: **ปริภูมิร่วม** ⟶ พิมพ์ “แอนิเมชันอบอุ่นเรื่องครอบครัว” แล้วค้นโปสเตอร์ได้



τ = 1 แยกคู่ถูก–ผิดแทบไม่ออก • τ เล็กเกิน softmax แหลมจนเกรเดียนต์คู่ผิด ≈ 0

ตัวอย่างการคำนวณ 4.39 : ค่าสูญเสียของ CLIP บนชุดสามคู่

โคไซน์ภาพ–ข้อความ S (สมมติ) · $\tau = 0.07$

ภาพ \ ข้อความ	t_i1	t_i8	t_i12
x_i1	0.31	0.12	0.05
x_i8	0.10	0.28	0.09
x_i12	0.04	0.11	0.30

1 แถว i1 : หาร τ แล้ว softmax (ขั้นที่ 1–2)

- (0.31, 0.12, 0.05)/0.07 = (4.4286, 1.7143, 0.7143)
- $e^{4.4286} = 83.81$ · $e^{1.7143} = 5.55$ · $e^{0.7143} = 2.04$ · รวม 91.40
- $p_{1,\cdot}$ = (**0.9169**, 0.0607, 0.0223)

3 ทิศภาพ ⟶ ข้อความ (softmax ตามแถว)

- คู่ถูก (0.9169, 0.8751, 0.9169)
- $\mathcal{L}_{img} = -(\ln 0.9169 + \ln 0.8751 + \ln 0.9169)/3 = \mathbf{0.1023}$

4 ทิศข้อความ ⟶ ภาพ (ตามคอลัมน์)

- คู่ถูก (0.9338, 0.8404, 0.9277)
- $\mathcal{L}_{txt} = \mathbf{0.1058} \longrightarrow \mathcal{L}_{CLIP} = (0.1023 + 0.1058)/2 = \mathbf{0.1040}$

ตีความ โคไซน์ 0.31 กับ 0.12 ต่างไม่มาก แต่หลังหาร 0.07 ความน่าจะเป็นคู่ถูกสูงถึง 0.9169 · การฝึกจะผลักคู่ถูกให้สูงขึ้น คู่ผิดต่ำลง

ขั้นตอนโดยละเอียด • ตัวอย่างการคำนวณ 4.39 (CLIP ครบสองทิศ)

ข้อมูลมาจากไหน

- i1 แอนิเมชัน • i8 แอ็กชัน • i12 ดราม่าวิทยาศาสตร์ (ตารางที่ 4.4)
- โปสเตอร์คู่กับเรื่องย่อเรื่องเดียวกัน (ตารางที่ 4.6)
- โคไซน์เป็นค่าสมมติ (ภาพที่ 4.48) • τ = 0.07

คู่ถูก	p ภาพ→ข้อความ	p ข้อความ→ภาพ
i1	0.9169	0.9338
i8	0.8751	0.8404
i12	0.9169	0.9277

L_img = 0.1023 • L_txt = 0.1058 • L_CLIP = 0.1040

4.6.10 รวมสื่อกับพฤติกรรมผู้ใช้ : VBPR

$$\hat{x}_{ui} = \beta_i + \boldsymbol{\gamma}_u \cdot \boldsymbol{\gamma}_i + \boldsymbol{\theta}_u \cdot (\mathbf{E}\,\mathbf{f}_i) \qquad (4.55)$$

$$\mathcal{L}_{BPR} = -\sum_{(u,i,j)} \ln \sigma(\hat{x}_{ui} - \hat{x}_{uj}), \quad \sigma(z) = \frac{1}{1+e^{-z}} \qquad (4.56)$$

- $\boldsymbol{\beta}_i$ ความนิยมพื้นฐาน · $\boldsymbol{\gamma}_u$, $\boldsymbol{\gamma}_i$ เวกเตอร์แฝงจากพฤติกรรม (บทที่ 5)
- $\mathbf{f}_i$ เวกเตอร์ภาพจาก CNN/Transformer (ไม่ต้องเรียนรู้)
- $\mathbf{E}$ ฉายภาพลงมิติแฝง · $\boldsymbol{\theta}_u$ รสนิยมด้านภาพ
- (u, i, j) i = ชอบ, j = ไม่ชอบ/ยังไม่ดู (Rendle et al., 2009)

ทำไม VBPR สำคัญ

- รวม **เนื้อหา** เข้ากับ **การกรองแบบร่วมมือ**
- รายการใหม่ที่ยังไม่มีใครดู (Cold Start) ยังได้คะแนนจากพจน์ภาพ $\boldsymbol{\theta}_u \cdot (\mathbf{E}\,\mathbf{f}_i)$
- การกรองแบบร่วมมือล้วน ทำแบบนี้ไม่ได้
- ใช้เมื่อมีพฤติกรรมมากพอ และต้องรับมือรายการใหม่

ตัวอย่างการคำนวณ 4.40 : คะแนน VBPR และค่าสูญเสีย BPR

โจทย์ ϵ = [0.5 0 0.5 ; 0 1 0] · θ_{u1} = (1.0, 0.2) · γ_{u1} = (0.3, 0.1) · γ_{i5} = (0.2, 0.4) · γ_{i7} = (−0.1, 0.3) · β_{i5} = 0.1 · β_{i7} = 0.2

1 ฉายเวกเตอร์ภาพ ϵ f

- f_{i5} = (0.7552, 0.1590, 0.6359) ⟶ ϵf = (0.5(0.7552) + 0.5(0.6359), 0.1590) = **(0.6956, 0.1590)**
- f_{i7} = (−0.6714, −0.5240, −0.5240) ⟶ **(−0.5977, −0.5240)**

2 คะแนนของ i5

- พฤติกรรม: $\gamma_{u1} \cdot \gamma_{i5}$ = 0.06 + 0.04 = 0.1000
- ภาพ: $\theta \cdot \epsilon f$ = 1.0(0.6956) + 0.2(0.1590) = 0.7274
- $\hat{x}_{u1,i5}$ = 0.1 + 0.1000 + 0.7274 = **0.9274**

3 คะแนน i7 และค่าสูญเสีย

- $\hat{x}_{u1,i7}$ = 0.2 + (−0.03 + 0.03) + (−0.5977 − 0.1048) = **−0.5025**
- ผลต่าง 1.4299 ⟶ σ = 0.8069 ⟶ $-\ln\sigma$ = 0.2146

4 รายการใหม่ i12 (Cold Start)

- ยังไม่มีพฤติกรรม: β = 0, γ = 0
- เหลือพจน์ภาพ = **−0.6865**
- ค่าสูญเสียต่ำ = จัด i5 เหนือ i7 ถูกแล้ว

ที่มาของข้อมูล • ตัวอย่างการคำนวณ 4.40 (VBPR)

ข้อมูลมาจากไหน

- f_i = ภาพจากตารางที่ 4.16 หลังลบ μ_I และปรับ L^2
- i5 อันดับ 1 ของ u1 • i7 อันดับท้าย (ตารางที่ 4.19)
- i12 ฉายปี 2026 → แทนรายการใหม่
- ϵ, θ, γ, β เป็นค่าสมมติ

เรื่อง	f_i	ϵ f_i	θ·ϵ f_i
i5	(0.7552, 0.1590, 0.6359)	(0.6956, 0.1590)	0.7274
i7	(−0.6714, −0.5240, −0.5240)	(−0.5977, −0.5240)	−0.7025
i12	(−0.5395, −0.5954, −0.5954)	(−0.5674, −0.5954)	−0.6865

x(u1,i5) = 0.9274 • x(u1,i7) = −0.5025 • L_BPR = 0.2146

4.6.11 ผลของเทคนิค Fusion และแนวทางเลือกใช้

เทคนิค Fusion	CNN HR@3	CNN MRR	TF HR@3	TF MRR
Late Fusion ตั้งต้น	0.75	0.7671	0.75	0.7635
z-score	0.80	0.7506	0.75	0.7384
Min-Max	0.80	0.7256	0.75	0.7410
Borda	0.80	0.7546	0.75	0.7463
RRF $k = 60$	0.80	0.7535	0.80	0.7521
RRF $k = 0$	0.85	0.8185	0.85	0.7921
Gate $\tau = 0.5$	0.75	0.7392	0.75	0.7175

ตารางที่ 4.23 กันออกทีละรายการ 20 กรณี โปรไฟล์ค่าเฉลี่ยทุกระบบ

เลือกเทคนิค Fusion อย่างไร

- เริ่มจาก Late Fusion ง่าย อธิบายได้ เพิ่ม/ถอดสื่อได้ทันที
- **ปรับมาตรา/ใช้อันดับ** เมื่อช่วงคะแนนต่างกันมาก
- เลือกน้ำหนักบน **ชุดตรวจสอบ** ไม่ใช่ชุดทดสอบ
- Gate/Attention เมื่อผู้ใช้หลากหลายและมีข้อมูลพอ
- CLIP ค้นข้ามสื่อ • VBPR มีพฤติกรรมมาก + รายการใหม่

4.7

การประเมินผล

Leave-One-Out • HR@3 • MRR • nDCG@3 • ถอดสื่อทีละชนิด (Ablation)



4.7.1 โปรโตคอลซ่อนคำตอบทีละเรื่อง (Leave-One-Out)

$$HR@K = \frac{1}{N}\sum_{c=1}^{N} \mathbb{1}[r_c \le K], \quad MRR = \frac{1}{N}\sum_{c=1}^{N} \frac{1}{r_c}, \quad nDCG@K = \frac{1}{N}\sum_{c=1}^{N} \frac{\mathbb{1}[r_c \le K]}{\log_2(r_c + 1)} \tag{4.57}$$

1. ซ่อนเรื่องที่ชอบหนึ่งเรื่อง h
2. สร้างโปรไฟล์จากประวัติที่เหลือ
3. ให้คะแนนผู้สมัคร + h แล้วจัดอันดับ
4. บันทึกอันดับ r_c แล้วคำนวณตัวชี้วัด

ตั้งค่าในบทนี้

- ผู้ใช้ 8 คน ทุกเรื่องที่ชอบถูกซ่อนหนึ่งครั้ง ⟶ N = 20 กรณี
- K = 3 • $\mathbb{1}[\cdot]$ = 1 เมื่อเงื่อนไขจริง

ความหมาย

- HR@3 ติด Top-3 บ่อยแค่ไหน
- MRR เรื่องที่ถูกอยู่ใกล้อันดับ 1 แค่ไหน
- nDCG@3 ให้รางวัลตามตำแหน่งใน Top-3

ตัวอย่างการคำนวณ 4.41 : ซ่อน i2 ของ u1 และซ่อน i1 ของ u4

โจทย์ โปรไฟล์ค่าเฉลี่ย + Late Fusion น้ำหนักตั้งต้น (Transformer) จงหาอันดับของเรื่องที่ซ่อนและค่าตัวชี้วัดแต่ละกรณี

1 u1 ซ่อน i2 Inside Out 2

- โปรไฟล์จาก i1, i3 • ผู้สมัคร 7 + i2 = 8 เรื่อง
- Top-3: **i2 (0.6637)**, i5 (0.4350), i8 (−0.2203)
- r = 1 ⟶ HR@3 = 1 • 1/r = 1.0000
- nDCG@3 = $1/\log_2(2)$ = 1.0000

2 u4 ซ่อน i1 Toy Story 5

- โปรไฟล์จาก i7, i8 (แอ็กชันทั้งคู่)
- Top-3: i10 (0.7706), i11 (0.5290), i12 (0.2421)
- i1 อยู่ **อันดับ 7** ⟶ HR@3 = 0 • 1/r = 0.1429
- nDCG@3 = 0

3 เฉลี่ยสองกรณี

- HR@3 = (1 + 0)/2 = **0.5** • MRR = (1.0000 + 0.1429)/2 = 0.5714

ตีความ ประวัติที่เหลือของ u4 ไม่มีร่องรอยแอนิเมชันเลย ไม่มี Fusion ใดกู้ได้ ⟶ ต้องใช้พฤติกรรมผู้ใช้อื่น (บทที่ 5)

ขั้นตอนโดยละเอียด • ตัวอย่างการคำนวณ 4.41 (u1 ซ่อน i2)

ข้อมูลมาจากไหน

- u1 (ตารางที่ 4.5) ชอบ i1, i2, i3
- ซ่อน i2 → โปรไฟล์จาก i1, i3
- ผู้สมัคร 8 เรื่อง (ยังไม่ให้คะแนน + เรื่องที่ซ่อน)

อันดับ	รายการ	T	I	A	V	s
1	i2 ← ซ่อน	0.2786	0.9124	0.8488	1.0000	0.6637
2	i5	0.4843	0.9013	0.9480	−0.6427	0.4350
3	i8	0.1990	−0.1399	−0.9136	−0.4462	−0.2203
4	i9	0.0563	−0.7181	−0.1603	−0.6427	−0.2817
5	i12	0.3335	−0.7472	−0.6586	−0.8786	−0.3235
6	i11	0.3474	−0.6424	−0.9214	−0.9726	−0.3683
7	i7	0.1637	−0.7468	−0.9768	−0.5166	−0.3826
8	i10	0.1830	−0.7483	−0.9970	−0.8383	−0.4435

i2 อยู่อันดับ 1

ขั้นตอนโดยละเอียด • ตัวอย่างการคำนวณ 4.41 (u4 ซ่อน i1)

ข้อมูลมาจากไหน

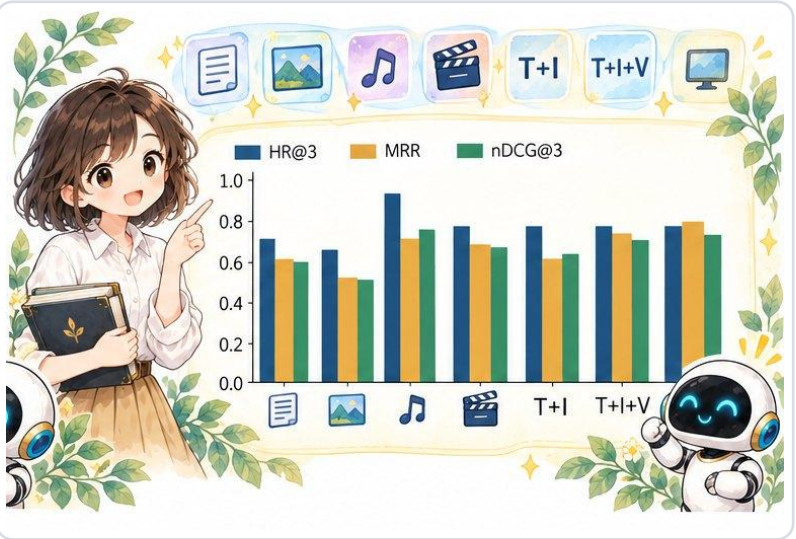
- u4 (ตารางที่ 4.5) ชอบ i1, i7, i8
- ซ่อน i1 → โปรไฟล์จาก i7, i8
- ผู้สมัคร 9 เรื่อง (ยังไม่ให้คะแนน + เรื่องที่ซ่อน)

อันดับ	รายการ	T	I	A	V	s
1	i10	0.5641	0.9012	0.9708	0.8531	0.7706
2	i11	0.4757	0.2985	0.8480	0.5473	0.5290
3	i12	0.1587	0.4430	0.4096	0.0406	0.2421
4	i9	0.2614	0.3655	−0.0897	−0.3208	0.0955
5	i4	0.2742	0.9349	−0.9999	−0.3505	0.0266
6	i6	0.0723	0.4972	−0.9528	−0.6152	−0.1852
7	i1 ← ซ่อน	0.1882	−0.8678	−0.6338	−0.3137	−0.2878
8	i2	0.0599	−0.6547	−0.7649	−0.4852	−0.3570
9	i3	0.1706	−0.9181	−0.9593	−0.6337	−0.4340

i1 อยู่อันดับ 7

4.7.3 ผลของสื่อเดี่ยว การรวมสื่อ และสองเทคนิค

ระบบ	น้ำหนัก (T, I, A, V)	CNN HR@3	CNN MRR	TF HR@3	TF MRR	TF nDCG@3
ข้อความ	(1, 0, 0, 0)	0.65	0.6818	0.70	0.6044	0.5946
ภาพ	(0, 1, 0, 0)	0.65	0.4876	0.65	0.5035	0.5012
เสียง	(0, 0, 1, 0)	0.95	0.7683	0.90	0.7033	0.7393
วิดีโอ	(0, 0, 0, 1)	0.60	0.5883	0.75	0.6673	0.6577
T+I	(0.5, 0.5, 0, 0)	0.75	0.6094	0.75	0.6082	0.6196
T+I+V	(0.4, 0.2, 0, 0.4)	0.75	0.6969	0.75	0.7247	0.7000
สี่สื่อ	(0.4, 0.2, 0.2, 0.2)	0.75	0.7671	0.75	0.7635	0.7250



เสียงคือสื่อเดี่ยวที่ดีสุด

HR@3 = 0.90 (TF) ดีกว่าข้อความ

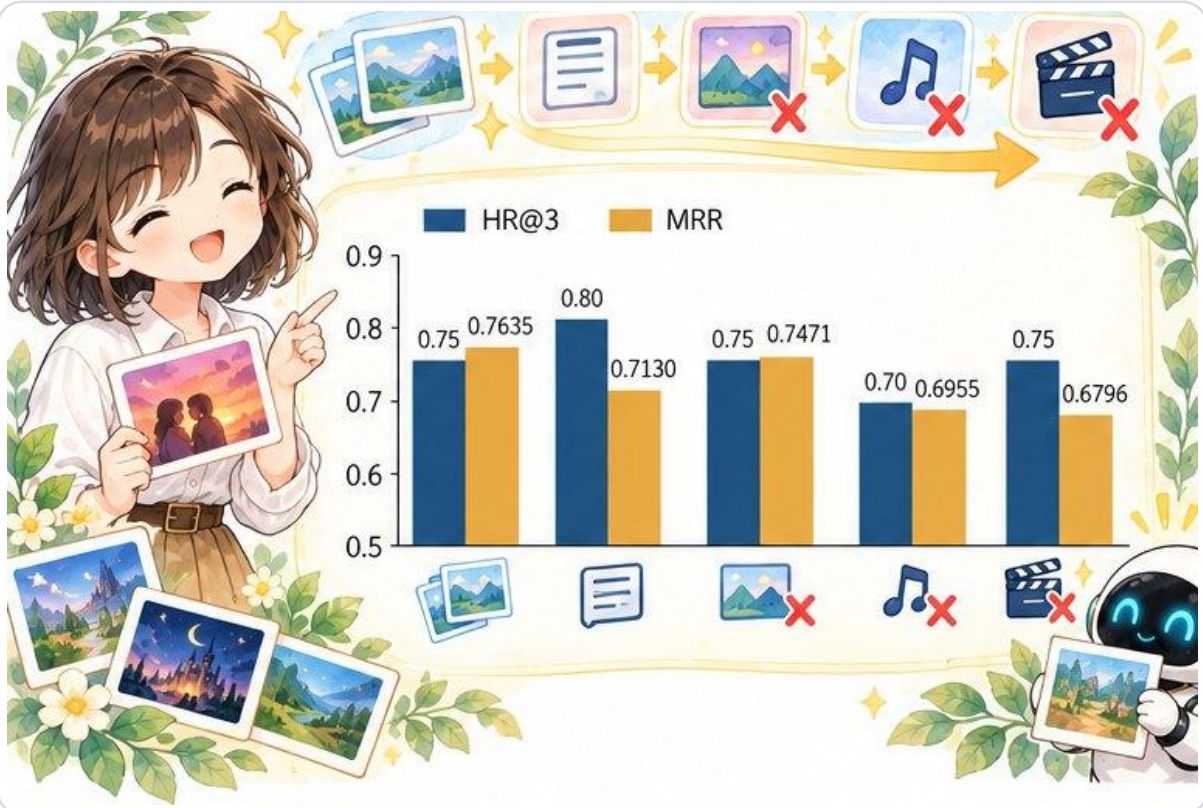
รวมสี่สื่อให้ MRR สูงสุด

0.7635 ⟶ ดันเรื่องถูกขึ้นอันดับ 1 บ่อยขึ้น

สองเทคนิคใกล้กันมาก

MRR หลังรวมต่าง < 0.01

4.7.4 ถอดสื่อทีละชนิด (Ablation)



ระบบ	HR@3	MRR	nDCG@3
สี่สื่อ (ตั้งต้น)	0.75	0.7635	0.7250
ถอด T	0.80	0.7130	0.7065
ถอด I	0.75	0.7471	0.7131
ถอด A	0.70	0.6955	0.6565
ถอด V	0.75	0.6796	0.6631

- **ถอดวิดีโอ MRR ลดมากสุด** (0.7635 ⟶ 0.6796) ⟶ วิดีโอให้ข้อมูล **ไม่ซ้ำ** สื่ออื่น
- **ถอดข้อความ** HR@3 เพิ่มเป็น 0.80 แต่ MRR ลด ⟶ ข้อความช่วยจัดอันดับต้น
- **ถอดภาพ** แทบไม่เปลี่ยน เพราะซ้ำกับวิดีโอ

ระวัง มีเพียง 20 กรณี เปลี่ยนอันดับหนึ่งเรื่อง ⟶ HR@3 เปลี่ยน 0.05 • ควรรายงาน Bootstrap CI